

Faculdade de Tecnologia Professor Jose Camargo - Fatec Jales

João Pedro Epaminondas do Carmo

Projeto de Extensão

Sistema de Gerenciamento da Prefeitura de Jales

Jales-SP

2026

João Pedro Epaminondas do Carmo

Sistema de Gerenciamento da Prefeitura de Jales

Documentação comprobatória das atividades realizadas durante o Projeto de Extensão desenvolvido junto a Faculdade de Tecnologia Professor José Camargo - Fatec Jales, como parte dos requisitos para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistema.

Faculdade de Tecnologia Professor Jose Camargo - Fatec Jales

Orientador: Prof. Me. Jefferson Antonio Ribeiro Passerini

Jales-SP

2026

Resumo

O projeto *Civitas* consiste em um sistema web desenvolvido para apoiar a gestão pública municipal, com foco no controle administrativo e financeiro de órgãos públicos. A proposta surge diante da necessidade de centralizar informações, reduzir inconsistências cadastrais, melhorar o acompanhamento de despesas e ampliar a rastreabilidade das operações relacionadas ao orçamento público. O sistema oferece uma plataforma digital para organização de cadastros administrativos, registro e acompanhamento de orçamentos, despesas, documentos, fluxos e auditorias, além de recursos de consulta, atualização e controle da situação dos registros. A solução também disponibiliza um painel com indicadores financeiros, permitindo melhor visualização das informações e apoio à tomada de decisão pelos gestores. Em seus principais aspectos técnicos, o *Civitas* utiliza tecnologias voltadas ao desenvolvimento web, integrando interface de usuário, serviços de aplicação, banco de dados e mecanismos de autenticação, de forma a proporcionar segurança, organização e possibilidade de evolução do sistema. Como resultado, espera-se que o projeto contribua para maior agilidade nos processos administrativos, redução de erros, melhoria do controle interno, fortalecimento da transparência e gestão mais eficiente dos recursos públicos municipais.

Palavras-chave: gestão pública municipal; controle orçamentário; despesas públicas; transparência administrativa.

Abstract

The *Civitas* project consists of a web-based system developed to support municipal public management, focusing on administrative and financial control of public agencies. The proposal arises from the need to centralize information, reduce data inconsistencies, improve expense tracking, and enhance the traceability of operations related to the public budget. The system provides a digital platform for organizing administrative records, registering and monitoring budgets, expenses, documents, workflows, and audits, as well as offering features for querying, updating, and controlling the status of records. The solution also includes a dashboard with financial indicators, enabling better visualization of information and supporting decision-making by managers. In its main technical aspects, the *Civitas* uses technologies aimed at web development, integrating user interface, application services, database, and authentication mechanisms, in order to provide security, organization, and scalability for the system. As a result, the project is expected to contribute to greater agility in administrative processes, reduction of errors, improvement of internal control, strengthening of transparency, and more efficient management of municipal public resources.

Keywords: municipal public management; budget control; public expenses; administrative transparency.

Lista de ilustrações

Figura 2.1 – Interface do sistema SILPI – Sistemas Humanizados.	15
Figura 3.1 – Diagrama de Classe	22
Figura 3.2 – Diagrama de Atores	46
Figura 3.3 – Diagrama de Caso de Uso Geral - Administrador	53
Figura 3.4 – Diagrama de Caso de Uso Geral - Enfermeiro	54
Figura 3.5 – Diagrama de Caso de Uso Geral - Profissional	55
Figura 3.6 – Diagrama de Caso: Prontuário SOAP	57
Figura 3.7 – Diagrama de sequência para listagem de parente de residente.	63
Figura 3.8 – Diagrama de sequência para cadastro de parente de residente.	64
Figura 3.9 – Diagrama de sequência para edição de parente de residente.	65
Figura 3.10–Diagrama de sequência para remoção de parente de residente.	66
Figura 3.11–Diagrama de atividade para o processo de autenticação do usuário. . .	67
Figura 4.1 – Persona 1 — Enfermeira-chefe (Carla Regina de Souza)	71
Figura 4.2 – Persona 2 — Recepcionista (Fernando Costa)	72
Figura 4.3 – Wireframes — (a) Tela Inicial para Módulo de Cuidado (Home Care) e (b) Tela Inicial para Módulo de Estoque (Home Stock)	73
Figura 4.4 – Wireframes — (a) Tela de Login para o Módulo de Cuidado (Care) e (b) Tela de Login para o Módulo de Estoque (Stock)	74
Figura 4.5 – Wireframes — (a) Tela de Cadastro de Instituição e (b) Tela de Gestão de Religião	75
Figura 4.6 – SeniorCareManager — Tela de Abertura	76
Figura 4.7 – SeniorStockManager — Tela de Abertura	77
Figura 4.8 – SeniorCareManager — Tela de Login	78
Figura 4.9 – SeniorStockManager — Tela de Login	78
Figura 4.10–SeniorCareManager — Gestão de Religiões (listagem, busca e manutenção)	79
Figura 4.11–SeniorStockManager — Tela de Manutenção de Produtos	80
Figura 4.12–Alto Contraste — (a) Tela de login com alto contraste ativado e (b) Tela de login sem alto contraste	83
Figura 4.13–Alto Contraste — Dispositivos Móveis — (a) Tela de login com alto contraste ativado e (b) Tela de login sem alto contraste	84
Figura 4.14–Fonte Redimensionada - (a) Tela principal (b) Menu Lateral com fonte redimensinada.	85
Figura 5.1 – Mapeamento Objeto-Relacional	88

Lista de tabelas

Tabela 4.1 – Teclas de Atalho Definidas no Sistema	82
Tabela 7.1 – Horas Trabalhadas no Projeto.	102

Lista de quadros

2.1	Requisitos Não Funcionais	18
3.1	Entidade Product	23
3.2	Entidade ProductBatch	24
3.3	Entidade ProductType	24
3.4	Entidade UnitOfMeasure	24
3.5	Entidade Manufacturer	25
3.6	Entidade StockMovement	25
3.7	Entidade Supplier	26
3.8	Enum StockMovementType	26
3.9	Enum StockRequestType	27
3.10	Entidade PurchaseRequest	27
3.11	Entidade PurchaseRequestItem	28
3.12	Entidade Employee	29
3.13	Entidade Position	29
3.14	Enum PurchaseStatus	30
3.15	Enum YesNo	30
3.16	Entidade StockRequest	30
3.17	Entidade StockRequestItem	31
3.18	Entidade Invoice	31
3.19	Entidade InvoiceItem	32
3.20	Entidade ProductGroup	32
3.21	Interface IStockDocument	33
3.22	Entidade Carrier	33
3.23	Entidade User	34
3.24	Enum UserType	34
3.25	Enum UserStatus	34
3.26	Entidade Religion	35
3.27	Enum MaritalStatus	35
3.28	Enum EducationLevel	35
3.29	Enum Ethnicity	36
3.30	Enum Sex	36
3.31	Entidade Resident	37
3.32	Entidade HealthInsurancePlan	37
3.33	Enum HealthPlanType	38
3.34	Entidade ResidentAllergy	38

3.35	Entidade Allergy	38
3.36	Entidade ResidentRelative	39
3.37	Enum Relationship	39
3.38	Enum AllergyType	40
3.39	Entidade NursingCarePlan	40
3.40	Entidade NursingService	40
3.41	Entidade Prescription	41
3.42	Entidade Schedule	41
3.43	Entidade CID10	41
3.44	Entidade Company	42
3.45	Entidade TechnicalResponsibility	42
3.46	Entidade SystemParameters	43
3.47	Entidade SystemModule	43
3.48	Entidade SystemModuloScreen	44
3.49	Entidade UserTypeDefaultAccess	44
3.50	Enum StatusEmployee	44
3.51	Casos de Uso — Administrador	47
3.52	Casos de Uso — Funcionário	49
3.53	Casos de Uso — Enfermeiro	50
3.54	UC: Acessar SOAP	58
3.55	UC: Registrar Anamnese	58
3.56	UC: Registrar Avaliação	59
3.57	UC: Registrar Exames	60
3.58	UC: Registrar Plano de Cuidado	61
4.1	Cenário — Registro de Religião	70
4.2	Cenário — Registro e Consulta de Alergias	70
5.1	Script SQL — Tabela address	89
5.2	Script SQL — Tabela allergy	89
5.3	Script SQL — Tabela carrier	89
5.4	Script SQL — Tabela city	90
5.5	Script SQL — Tabela health_insurance	90
5.6	Script SQL — Tabela health_insurance_plan	90
5.7	Script SQL — Tabela login	91
5.8	Script SQL — Tabela manufacturer	91
5.9	Script SQL — Tabela product	91
5.10	Script SQL — Tabela product_category	92
5.11	Script SQL — Tabela product_group	92
5.12	Script SQL — Tabela product_type	92

5.13	Script SQL — Tabela <code>profile</code>	92
5.14	Script SQL — Tabela <code>profile_claim</code>	93
5.15	Script SQL — Tabela <code>profile_resource</code>	93
5.16	Script SQL — Tabela <code>resident</code>	93
5.17	Script SQL — Tabela <code>resident_allergy</code>	94
5.18	Script SQL — Tabela <code>resident_health_insurance</code>	94
5.19	Script SQL — Tabela <code>resident_relative</code>	94
5.20	Script SQL — Tabela <code>resource</code>	95
5.21	Script SQL — Tabela <code>state</code>	95
5.22	Script SQL — Tabela <code>unitofmeasure</code>	95
5.23	Script SQL — Tabela <code>user</code>	95

Sumário

1	INTRODUÇÃO	10
2	LEVANTAMENTO DE REQUISITOS DE SOFTWARE	12
2.1	Descrição dos Objetivos do Sistema	13
2.2	Análise dos Sistemas Existentes	13
2.2.1	Levantamento de Sistemas no Mercado	14
2.3	Descrição dos Principais Problemas	16
2.3.1	Descrição dos Requisitos Não Funcionais	17
2.3.2	Descrição dos Requisitos Funcionais	19
3	VISÃO DE CASO DE USO - UML	20
3.1	Diagrama de Classes	20
3.2	Dicionário de Classes	23
3.3	Definição de Atores	44
3.4	Casos de Uso	46
3.4.1	Casos de Uso do Administrador	46
3.4.2	Casos de Uso do Funcionário	48
3.4.3	Casos de Uso do Enfermeiro	50
3.5	Diagrama de Casos de Uso	52
3.6	Diagrama de Casos de Uso - Individuais	56
3.6.1	Caso de Uso Individual - Cadastro de SOAP	56
3.7	Diagrama de Sequência	61
3.7.1	Diagrama de Sequência: Listar Parente de Residente	62
3.7.2	Diagrama de Sequência: Cadastrar Parente de Residente	64
3.7.3	Diagrama de Sequência: Editar Parente de Residente	65
3.7.4	Diagrama de Sequência: Remover Parente de Residente	66
3.8	Diagrama de Atividade	67
4	DEFINIÇÃO DA INTERFACE COM O USUÁRIO	69
4.1	Descrição de Cenário	69
4.2	Descrição de Personas	70
4.3	Esboços de tela (wireframes)	72
4.4	Protótipos de tela	75
4.5	Acessibilidade e Inclusão de Pessoas com Deficiência	80
4.5.1	Práticas de Acessibilidade Implementadas	81

5	BANCO DE DADOS	87
5.1	Modelo Entidade-Relacionamento	87
5.2	Script das Tabelas	88
5.3	Mapeamento Objeto Relacional	96
6	ARQUITETURA DE SOFTWARE	97
6.1	Arquitetura de Desenvolvimento	97
6.1.1	Back-End	98
6.1.2	Front-End	99
6.2	Segurança da Informação	100
7	CONTROLE DE PARTICIPAÇÃO NO PROJETO	102
7.1	Relatório de Sprints	102
7.1.1	Sprint 01 - Task 02 e 03	103
7.1.2	Sprint 02 - Task 04	117
7.1.3	Sprint 03 - Task 15	135
7.1.4	Sprint 04 - Task 16 e 20	137
7.1.5	Sprint 05 - Task 25	146
8	CONCLUSÃO	155

1 Introdução

O avanço da tecnologia e da informatização tem transformado significativamente a maneira como organizações públicas e privadas realizam seus processos administrativos, financeiros e operacionais. Atualmente, os sistemas de informação desempenham papel fundamental no armazenamento, processamento, controle e gerenciamento de dados, contribuindo diretamente para maior eficiência, segurança e organização das informações. A utilização de recursos tecnológicos permite automatizar tarefas que anteriormente eram executadas manualmente, reduzindo falhas humanas, aumentando a produtividade e melhorando a tomada de decisão nas instituições. De acordo com Sommerville(2018), os sistemas de software tornaram-se essenciais para o funcionamento das organizações modernas, auxiliando no gerenciamento de processos cada vez mais complexos e contribuindo para maior confiabilidade e eficiência nas atividades desenvolvidas.

A ausência de ferramentas integradas pode ocasionar atrasos em processos administrativos, dificuldades no controle de despesas e limitações no monitoramento de documentos e registros financeiros. A falta de centralização das informações também dificulta a tomada de decisão, uma vez que gestores públicos necessitam de dados organizados e atualizados para planejar ações e administrar adequadamente os recursos municipais. Conforme afirmam Pressman(2021), a engenharia de software possui papel importante no desenvolvimento de sistemas capazes de atender às necessidades organizacionais de forma estruturada, segura e confiável, contribuindo para maior qualidade e eficiência nos processos informatizados.

Pensando nas dificuldades enfrentadas pelos órgãos públicos municipais em relação ao controle administrativo e financeiro, surgiu o desenvolvimento do *Civitas*, um sistema web criado para apoiar a gestão pública municipal por meio da centralização e organização das informações administrativas e financeiras. O sistema foi projetado para oferecer funcionalidades relacionadas ao gerenciamento de cadastros administrativos, controle orçamentário, registro e acompanhamento de despesas, gerenciamento documental e registro de auditorias, permitindo maior controle das operações realizadas pelos órgãos públicos municipais.

A proposta do sistema busca atender à necessidade de modernização administrativa, oferecendo uma plataforma organizada e eficiente para gerenciamento das informações institucionais. O *Civitas* também disponibiliza recursos de consulta, atualização de registros e acompanhamento da situação dos processos administrativos, permitindo maior agilidade nas rotinas internas e melhoria no fluxo das informações entre os setores responsáveis. Dessa forma, o sistema contribui para reduzir inconsistências cadastrais, minimizar falhas

operacionais e facilitar o acompanhamento das atividades relacionadas ao orçamento público.

Outro aspecto relevante do projeto está relacionado à contribuição que a informatização pode oferecer para a melhoria dos serviços públicos municipais. A adoção de sistemas informatizados favorece maior controle dos processos internos, redução de documentos físicos, melhor armazenamento das informações e maior rapidez no acesso aos dados administrativos. Dessa forma, a tecnologia torna-se uma importante aliada na busca por uma administração pública mais eficiente, transparente e alinhada às necessidades da população.

A intenção do projeto é oferecer uma solução tecnológica capaz de auxiliar os órgãos públicos municipais no gerenciamento de informações administrativas e financeiras, contribuindo para maior agilidade nos processos internos, fortalecimento da transparência pública e melhoria do controle interno. Dessa forma, o *Civitas* busca unir tecnologia, organização e controle administrativo em uma única plataforma, atendendo às necessidades da gestão pública municipal de maneira eficiente, segura e estruturada, além de contribuir para a modernização dos processos administrativos e fortalecimento da gestão dos recursos públicos.

2 Levantamento de Requisitos de Software

O levantamento de requisitos constitui uma das etapas mais relevantes do processo de desenvolvimento de sistemas, uma vez que é nesse momento que se identificam, analisam e documentam as necessidades reais dos usuários e da instituição. Essa fase estabelece os fundamentos para a definição precisa do que será construído, garantindo que o produto final esteja em conformidade com as expectativas e os objetivos dos envolvidos no projeto.

De acordo com **Pressman2011ES7**, “a engenharia de requisitos oferece o suporte adequado para compreender o que o cliente precisa, avaliando as demandas, verificando a viabilidade, negociando condições possíveis e especificando funcionalidades que serão transformadas em um sistema de software operacional”. Assim, a atividade de levantamento de requisitos assume papel central no processo de engenharia de software, pois viabiliza a tradução das demandas organizacionais em especificações técnicas claras e executáveis.

No contexto do sistema proposto para o Lar dos Velhinhos São Vicente de Paulo e para a Associação de Pais e Amigos dos Excepcionais (APAE) de Jales, o levantamento de requisitos permitiu identificar pontos críticos a serem solucionados, tais como a necessidade de centralizar informações médicas em um prontuário eletrônico, organizar e registrar o controle de vacinas, aprimorar a gestão de compras e doações, e automatizar os processos de controle de estoque. Essas funcionalidades refletem as principais demandas das instituições, promovendo maior eficiência, segurança e transparência nas rotinas administrativas e assistenciais.

Conforme destaca **Sommerville2018ES10**, a definição de requisitos vai além do simples registro de informações iniciais, constituindo um passo essencial para estruturar o ciclo de vida do software e assegurar clareza no que será desenvolvido. Dessa forma, o levantamento realizado neste projeto fornece as bases para a construção de uma solução tecnológica capaz de atender simultaneamente às necessidades administrativas e assistenciais das instituições envolvidas, contribuindo para o aprimoramento do gerenciamento institucional e da qualidade dos serviços prestados aos residentes.

Portanto, o processo de levantamento de requisitos configura-se como elemento fundamental para garantir que o sistema proposto cumpra seu propósito, ao alinhar as expectativas dos usuários com as funcionalidades implementadas, resultando em uma ferramenta tecnológica que agrega valor e efetivamente soluciona as fragilidades identificadas nas entidades.

2.1 Descrição dos Objetivos do Sistema

Os fundamentos que orientaram o desenvolvimento do projeto Civitas concretizam-se na criação de um sistema informatizado e integrado, voltado ao gerenciamento da gestão pública municipal. O sistema foi concebido com o propósito de solucionar dificuldades relacionadas ao controle de despesas, orçamentos, instituições e fornecedores da administração pública, especialmente quando esses processos são realizados de forma manual, descentralizada ou pouco eficiente. Tais limitações podem comprometer a organização das informações, a transparência dos dados, o acompanhamento dos recursos públicos e a agilidade na tomada de decisões.

A proposta principal do Civitas consiste em oferecer uma plataforma tecnológica capaz de centralizar e organizar informações essenciais para o planejamento, o controle e o acompanhamento financeiro e administrativo do município. Seus usuários serão, principalmente, gestores públicos, servidores municipais e profissionais responsáveis pelos setores administrativos, financeiros e orçamentários, os quais poderão utilizar o sistema para registrar, consultar e acompanhar dados referentes às despesas, aos orçamentos, às instituições vinculadas e aos fornecedores da administração pública.

A implementação do sistema proporciona benefícios significativos à gestão municipal, como maior controle dos recursos públicos, redução de falhas operacionais, melhoria na rastreabilidade das informações e maior transparência nos processos administrativos. Além disso, o sistema contribui para tornar as atividades internas mais ágeis, organizadas e seguras, favorecendo uma administração pública mais eficiente e responsável perante a sociedade.

A análise das implicações decorrentes do aprimoramento promovido pelo sistema evidencia impactos positivos nos âmbitos organizacional, social e administrativo. Ao modernizar processos antes fragmentados ou dependentes de controles manuais, o Civitas fortalece a capacidade de planejamento e gestão dos órgãos públicos municipais. Como diferencial, destaca-se a integração entre o controle de despesas, orçamentos, instituições e fornecedores em uma única solução, promovendo maior confiabilidade, transparência e eficiência na administração pública.

2.2 Análise dos Sistemas Existentes

O Lar São Vicente de Paulo é uma instituição filantrópica localizada na cidade de Jales, São Paulo, que presta diversos serviços voltados ao acolhimento e cuidado de idosos em situação de vulnerabilidade social. A gestão dessas atividades, atualmente, é realizada de forma predominantemente manual ou com o auxílio de ferramentas genéricas, como planilhas eletrônicas e documentos físicos, o que pode comprometer a eficiência

operacional, a segurança das informações e a qualidade das decisões estratégicas.

Diante desse contexto, torna-se essencial a análise de sistemas já existentes que possam servir como referência para o desenvolvimento de um software personalizado, capaz de atender às demandas específicas da instituição. Essa etapa é fundamental para identificar boas práticas, recursos tecnológicos e funcionalidades aplicáveis ao cenário do Lar São Vicente de Paulo.

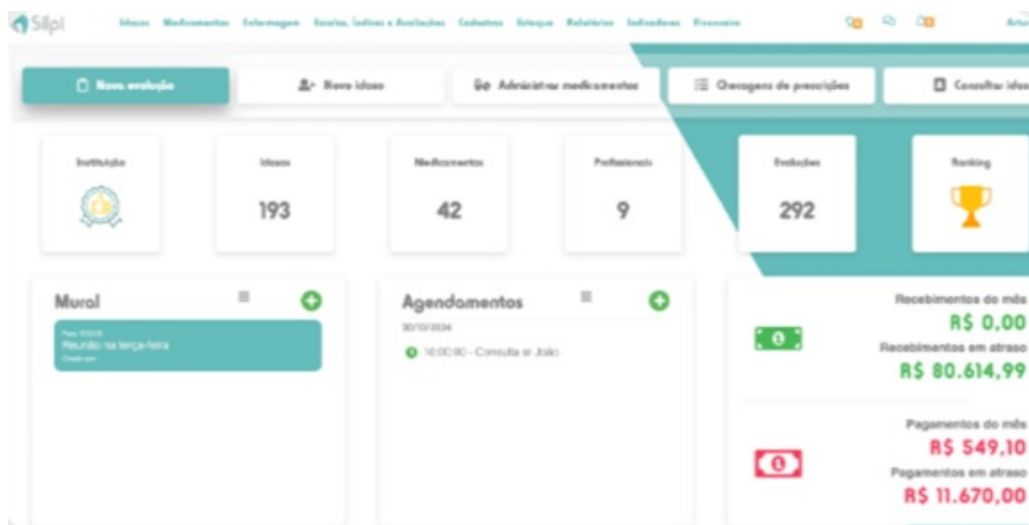
2.2.1 Levantamento de Sistemas no Mercado

Com o avanço das tecnologias de informação e comunicação, as instituições de caráter social e filantrópico têm buscado soluções digitais que otimizem seus processos administrativos e assistenciais. Nesse contexto, torna-se fundamental realizar um levantamento de sistemas disponíveis no mercado, a fim de identificar plataformas que possam servir de referência para o desenvolvimento de soluções personalizadas e adequadas às necessidades específicas de cada organização. Essa análise comparativa possibilita compreender as funcionalidades, limitações e boas práticas adotadas em softwares já consolidados, fornecendo subsídios técnicos e estratégicos para a concepção de sistemas mais eficientes, integrados e alinhados às demandas do setor social.

Entre as soluções analisadas, destaca-se o sistema SILPI – Sistemas Humanizados (**silpi2025**), pode-se observar um exemplo de sua interface na Figura 2.1, desenvolvido para a gestão de Instituições de Longa Permanência para Idosos (ILPIs). O sistema oferece funcionalidades voltadas ao controle assistencial, incluindo prontuário eletrônico, administração de medicamentos, registro da evolução clínica dos residentes e gerenciamento de atividades diárias. Além disso, contempla módulos administrativos, como controle de estoque, contas a pagar e a receber, e relatórios gerenciais.

A plataforma apresenta elevado grau de personalização, podendo ser adaptada às necessidades específicas de cada instituição e operando tanto em ambiente de nuvem quanto em servidores locais. Outro aspecto relevante é o suporte técnico especializado e a interface intuitiva, que facilitam a adoção e o uso por equipes multiprofissionais, promovendo maior integração e eficiência na gestão institucional.

Figura 2.1 – Interface do sistema SILPI – Sistemas Humanizados.



Fonte: Adaptado de SILPI (2025).

O sistema **Gestão 360 Social** (*gestao360*), desenvolvido pela empresa SETA Sistemas, constitui uma plataforma voltada a instituições de assistência social, com foco na gestão integrada de acolhidos, voluntários, projetos e recursos financeiros. O sistema possibilita o cadastro detalhado de beneficiários, o controle de atendimentos, o acompanhamento da evolução dos acolhidos e a emissão de relatórios técnicos. Além disso, oferece integração com o CadÚnico e disponibiliza ferramentas para a prestação de contas, o planejamento de programas sociais e a conciliação bancária. Trata-se de uma solução escalável, capaz de ser adaptada a diferentes portes institucionais, promovendo maior transparência, eficiência e confiabilidade na gestão dos serviços oferecidos.

O sistema **ONG Fácil** (*ongfacil2025*) é uma plataforma on-line voltada à gestão de organizações não governamentais e instituições filantrópicas. A ferramenta oferece funcionalidades como controle de doações financeiras e materiais, cadastro de doadores, emissão de recibos, gestão de contas a pagar e a receber, e geração de relatórios financeiros. O sistema também dispõe de módulos para comunicação com doadores, campanhas de arrecadação e prestação de contas. Sua operação é integralmente baseada em nuvem, permitindo acesso remoto e seguro às informações. A interface apresenta estrutura simples e intuitiva, o que facilita o uso por equipes administrativas com diferentes níveis de familiaridade tecnológica.

Embora os sistemas analisados — *Gestão 360 Social*, *ONG Fácil* e *SILPI – Sistemas Humanizados* — apresentem funcionalidades relevantes para a administração e o apoio assistencial em instituições sociais e filantrópicas, verificou-se que nenhum deles contempla integralmente as necessidades operacionais do Lar São Vicente de Paulo. As soluções existentes priorizam, em maior ou menor grau, aspectos relacionados à gestão administrativa, como controle financeiro, gerenciamento de doações e emissão de relatórios

institucionais.

No caso do *SILPI*, embora se destaque por ser uma plataforma voltada especificamente à gestão de Instituições de Longa Permanência para Idosos (ILPIs), oferecendo recursos assistenciais como prontuário eletrônico, controle de medicamentos e registro da evolução clínica, ainda assim não atende plenamente às demandas particulares do Lar São Vicente de Paulo. Entre as lacunas observadas, ressaltam-se a ausência de funcionalidades voltadas à personalização do acompanhamento dos residentes, à integração com processos internos da instituição e à adaptação a fluxos de trabalho locais.

Dessa forma, evidencia-se a necessidade de desenvolver um sistema que una a eficiência administrativa à gestão humanizada e individualizada dos residentes, atendendo de forma abrangente às especificidades funcionais e assistenciais de uma ILPI.

2.3 Descrição dos Principais Problemas

A ausência de um sistema informatizado específico para o gerenciamento das atividades do Lar São Vicente de Paulo tem ocasionado diversas dificuldades operacionais e administrativas, comprometendo a eficiência dos serviços prestados e a qualidade do atendimento aos idosos acolhidos. Durante o levantamento de requisitos e as visitas técnicas à instituição, observou-se que a gestão das informações é predominantemente manual, sendo realizada por meio de planilhas eletrônicas e registros físicos. Esse modelo de controle favorece a ocorrência de falhas, como a perda de dados, a duplicidade de registros e o preenchimento incorreto de informações essenciais.

Outro problema identificado refere-se à falta de integração entre os diferentes setores da instituição, como saúde, administração, assistência social e almoxarifado. A atuação isolada desses departamentos dificulta a comunicação interna, retarda a execução de processos e inviabiliza a tomada de decisões baseadas em dados consolidados. Além disso, a gestão de medicamentos, prescrições e prontuários clínicos é realizada de forma fragmentada, sem o suporte de um sistema que permita o acompanhamento histórico e a rastreabilidade das informações relacionadas aos residentes.

No campo financeiro, a administração de receitas, despesas, doações e prestações de contas ocorre sem o auxílio de ferramentas automatizadas, o que dificulta a transparência, o controle contábil e a elaboração de relatórios gerenciais. Essa limitação reflete também na impossibilidade de gerar indicadores de desempenho, relatórios de ocupação e análises sobre o consumo de recursos, restringindo o planejamento estratégico e a otimização dos processos internos. Soma-se a isso a baixa eficiência na captação e fidelização de doadores, já que o registro e o acompanhamento das doações são pouco estruturados, comprometendo a comunicação com apoiadores e a execução de campanhas de arrecadação eficazes.

Outro aspecto relevante diz respeito às restrições financeiras enfrentadas pela instituição, que inviabilizam a adoção de soluções comerciais disponíveis no mercado. Os sistemas existentes, embora ofereçam funcionalidades avançadas, possuem custos de licenciamento, manutenção e suporte técnico incompatíveis com a realidade orçamentária de uma entidade filantrópica. Assim, torna-se inviável a contratação ou personalização de softwares pagos, o que reforça a necessidade do desenvolvimento de uma solução própria, gratuita e sob medida para o contexto institucional.

Além disso, constatou-se que a comunicação interna entre os colaboradores ainda depende de meios informais, como bilhetes, telefonemas ou reuniões presenciais. Essa prática pode gerar atrasos, ruídos de comunicação e retrabalho, especialmente em situações que exigem respostas imediatas. Diante desse conjunto de fragilidades, evidencia-se a necessidade de desenvolver um sistema informatizado integrado e personalizado, capaz de atender às demandas específicas do Lar São Vicente de Paulo, assegurando maior eficiência administrativa, segurança da informação e continuidade da assistência humanizada aos idosos acolhidos.

2.3.1 Descrição dos Requisitos Não Funcionais

Os requisitos não funcionais abrangem os atributos de qualidade e desempenho do sistema, sendo responsáveis por assegurar que o software opere de forma eficiente, segura e estável. Segundo **PaulaFilho2015ES**, esses requisitos devem ser expressos de maneira precisa e, sempre que possível, quantitativa, uma vez que sua mensuração contribui para o controle da qualidade do produto final.

Os requisitos não funcionais surgem das necessidades dos usuários, que se devem a restrições orçamentárias, políticas organizacionais, necessidade de interoperabilidade com outros sistemas de software ou hardware, ou fatores externos, como normas de segurança (*safety*) ou legislação relativa à privacidade. **Sommerville2018ES10**

O *SeniorCareManager* foi desenvolvido em linguagem *C#* com arquitetura orientada a serviços, garantindo robustez e escalabilidade. O sistema utiliza um banco de dados relacional gratuito, adequado à realidade financeira das instituições filantrópicas, assegurando o armazenamento eficiente de informações sem comprometer a performance.

Em relação ao desempenho, o sistema foi projetado para apresentar tempo de resposta inferior a três segundos para as principais operações e disponibilidade mínima de 99,8%, assegurando acesso contínuo às informações. Em casos de falhas, o tempo máximo para correção ou atualização é de sete dias úteis, o que garante a manutenção da estabilidade e a confiança dos usuários.

A segurança das informações constitui um dos pilares do projeto. O controle de acesso é realizado por meio de autenticação baseada em tokens JWT (JSON Web Token),

assegurando a integridade das sessões e a proteção contra acessos indevidos. Todas as transações são registradas em logs de auditoria, possibilitando o rastreamento de eventos, modificações e acessos ao sistema.

A privacidade dos dados dos residentes é preservada em conformidade com a Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais (LGPD) **brasil2018lgpd**, sendo vedado o compartilhamento de informações sensíveis sem autorização expressa.

O sistema é compatível com diferentes navegadores e dispositivos, permitindo seu uso em computadores, tablets e smartphones. Além disso, a interface foi desenvolvida segundo princípios de usabilidade e design responsivo, favorecendo a experiência do usuário e a inclusão digital de profissionais de diferentes níveis de familiaridade tecnológica.

A supervisão e validação de dados são realizadas por administradores autorizados, garantindo a qualidade das informações inseridas. Assim, o *SeniorCareManager* alia desempenho, segurança e acessibilidade, constituindo uma solução tecnológica estável, ética e socialmente responsável.

A fim de assegurar a qualidade, robustez e confiabilidade do sistema proposto, torna-se imprescindível a definição clara dos requisitos não funcionais. Esses requisitos estabelecem parâmetros essenciais relacionados ao desempenho, segurança, disponibilidade e integridade das informações, servindo como base para orientar decisões arquiteturais e garantir que o sistema atenda às expectativas de uso em ambientes reais. O Quadro 2.1 apresenta a consolidação dos principais requisitos não funcionais identificados ao longo do processo de análise, ressaltando aspectos fundamentais para o funcionamento adequado e sustentável da solução.

Quadro 2.1 – Requisitos Não Funcionais

Requisitos Não Funcionais	Descrição
Desempenho	Garantir tempos de resposta rápidos para consultas e transações, mantendo alta eficiência.
Segurança	Implementar medidas robustas, como controle de acesso e criptografia, para proteger dados sensíveis.
Escalabilidade	Capacidade de lidar com um aumento substancial no volume de dados e transações.
Disponibilidade	Assegurar que o banco de dados esteja acessível a maior parte do tempo, minimizando períodos de inatividade.
Consistência	Manter a integridade e a consistência dos dados, garantindo precisão e atualização constante.
Backup e Restauração	Estabelecer procedimentos eficientes para backup regular e recuperação rápida em caso de falhas.

Fonte: Elaborado pelos autores.

2.3.2 Descrição dos Requisitos Funcionais

Os requisitos funcionais representam uma das etapas mais importantes no processo de desenvolvimento de sistemas, uma vez que permitem identificar as necessidades dos usuários e definir as funcionalidades específicas que o software deverá executar. De acordo com **Sommerville2018ES10**, tais requisitos estabelecem uma base sólida para o planejamento, a implementação e os testes, garantindo que o produto final esteja alinhado às expectativas e aos objetivos do projeto.

O sistema *SeniorCareManager* foi projetado para otimizar o gerenciamento das atividades do Lar dos Velhinhos São Vicente de Paulo e da APAE de Jales, integrando processos administrativos e assistenciais em uma única plataforma. Dessa forma, seus requisitos funcionais delineiam os serviços e operações essenciais que devem ser disponibilizados aos usuários, assegurando eficiência, rastreabilidade e segurança das informações.

Entre as funcionalidades principais, destacam-se os módulos de cadastro e manutenção de dados, que permitem incluir, editar e excluir registros de residentes, colaboradores, familiares, fornecedores e doações. Essas operações são realizadas por meio de botões de ação intuitivos, que facilitam o uso por diferentes perfis de profissionais da instituição.

O sistema também dispõe de um módulo de prontuário eletrônico, no qual enfermeiros e cuidadores podem registrar e consultar informações clínicas, evoluções, prescrições, planos de cuidado e histórico vacinal. Essa estrutura segue a lógica dos modelos do *e-SUS Atenção Primária* **ministeriosaude2023esus**, adaptada às particularidades da assistência em Instituições de Longa Permanência para Idosos (ILPIs).

Outros recursos incluem o módulo de estoque, destinado ao controle de insumos, medicamentos e materiais de uso contínuo; o módulo de compras e doações, que organiza o recebimento, a distribuição e a prestação de contas dos recursos; e o módulo de vacinas, que gerencia as campanhas de imunização, alertas de vencimento e registros de aplicação.

O sistema incorpora ainda mecanismos de pesquisa e filtragem de informações, com campos dinâmicos e categorização por grupos e subgrupos, permitindo consultas rápidas e precisas. A acessibilidade também é um aspecto prioritário, assegurando que o *SeniorCareManager* possa ser utilizado por diferentes perfis de usuários, incluindo profissionais com limitações visuais ou motoras.

Por fim, o sistema mantém integração com serviços externos, como geolocalização para registro de visitas domiciliares e notificações automáticas via e-mail, ampliando a conectividade e a eficiência das rotinas institucionais.

3 Visão de Caso de Uso - UML

O entendimento profundo de um sistema é crucial para o sucesso de qualquer projeto de desenvolvimento de software. Nesse contexto, a Linguagem de Modelagem Unificada (UML) configura-se como uma ferramenta essencial para representar visualmente a estrutura e as interações entre os componentes de um sistema. Conforme afirmam **Guedes2018**, a UML possibilita a modelagem clara e estruturada de artefatos de software, contribuindo para o entendimento e a comunicação entre os envolvidos no projeto. De forma complementar, **Miro2024** destaca que “desenvolvedores criam diagramas UML para entender projetos, arquitetura de código e propostas de implementação de sistemas de software complexos.”

A análise apresentada a seguir dedica-se à exploração dos principais diagramas da UML aplicados no desenvolvimento do sistema. Esses diagramas desempenham um papel fundamental na compreensão da estrutura e na narrativa do processo de desenvolvimento, oferecendo uma visão aprofundada de sua arquitetura. Ao examinar cada diagrama, busca-se desvendar não apenas a organização técnica, mas também a lógica e a dinâmica subjacentes ao desenvolvimento, proporcionando uma base sólida para a compreensão abrangente do projeto.

3.1 Diagrama de Classes

O diagrama de classes é fundamental na UML, sendo essencial para a criação dos demais diagramas do conjunto. Seu principal enfoque está em permitir a visualização das classes que comporão o sistema, juntamente com seus respectivos atributos e métodos, além de demonstrar como essas classes se relacionam, complementam e transmitem informações entre si.

Seu principal enfoque está em permitir a visualização das classes que comporão o sistema com seus respectivos atributos e métodos, bem como em demonstrar como as classes do diagrama se relacionam, complementam e transmitem informações entre si. Esse diagrama apresenta uma visão estática de como as classes estão organizadas, preocupando-se em como definir a estrutura lógica delas. (**Guedes2018**)

De acordo com **Sommerville2018ES10**, os diagramas de classes são estruturados em torno de um retângulo simples que representa uma classe. No interior desse retângulo, o nome da classe é apresentado na parte superior, seguido pelos atributos e seus tipos na área intermediária. As relações entre as classes são representadas por linhas conectando

os retângulos, indicando as associações entre elas e demonstrando como os elementos se interligam no modelo estático.

O diagrama de classes apresentado descreve uma arquitetura robusta e modular que integra informações administrativas, assistenciais, clínicas e operacionais do sistema. No núcleo do modelo destaca-se a classe **Resident**, responsável por concentrar os dados pessoais, sociais e de saúde dos residentes. Essa classe se relaciona diretamente com entidades auxiliares, como **Religion**, **Sex**, **Ethnicity**, **HealthInsurancePlan** e **NursingCarePlan**, permitindo representar características individuais, plano de saúde, cuidados prescritos e dimensões culturais. A estrutura do residente também é estendida por classes especializadas, como **ResidentRelative**, que registra contatos familiares e vínculos de parentesco, **ResidentAllergy**, que mapeia alergias vinculadas à classe **Allergy**, e **CID10**, utilizada para classificação padronizada de diagnósticos médicos.

A dimensão assistencial do modelo é reforçada pelas classes **Employee**, **Position**, **User** e **TechnicalResponsibility**, que compõem o conjunto de profissionais do sistema. Essas classes representam funções institucionais, níveis de acesso, responsabilidades técnicas e informações de cadastro, evidenciando como os colaboradores interagem com o sistema e executam ações clínicas e administrativas. Elementos como **Prescription** e **NursingService** reforçam o subsistema de cuidado, permitindo o registro estruturado de intervenções, serviços e condutas.

A segunda grande área modelada refere-se ao *módulo de estoque e suprimentos*. Nessa seção do diagrama, encontram-se classes como **Product**, **ProductGroup**, **ProductType**, **ProductBatch**, **Supplier**, **Invoice**, **Carrier** e **UnitOfMeasure**, que juntas garantem o registro completo de produtos, fornecedores, unidades de medida, lotes e movimentações fiscais. O ciclo de solicitação e movimentação de materiais é detalhado por classes como **PurchaseRequest**, **PurchaseRequestItem**, **StockRequest**, **StockRequestItem**, **StockMovement** e **StockDocument**. Os enumeradores associados descrevem estados dos processos, tipos de movimentação e etapas de aprovação, permitindo rastreabilidade precisa de entradas, saídas, ajustes e solicitações internas.

De maneira geral, o diagrama revela um sistema amplamente integrado, no qual módulos assistenciais, administrativos e logísticos se articulam de maneira coesa. As associações, multiplicidades e enumerações indicam regras de negócio específicas, vínculos hierárquicos, dependências obrigatórias e processos padronizados. Assim, o diagrama de classes fornece uma visão estática abrangente da estrutura do sistema, desempenhando papel fundamental na compreensão dos componentes que o compõem e nas inter-relações que sustentam seu funcionamento, conforme enfatizado por **Guedes2018** e **Sommerville2018ES10**.

3.2 Dicionário de Classes

O *Quadro 3.1* apresenta o dicionário de dados da entidade *Product*, descrevendo seus atributos, respectivos tipos e funções no modelo de domínio do sistema. Esse mapeamento estrutural possibilita compreender como as informações sobre os produtos são armazenadas, organizadas e utilizadas pelas camadas de persistência e regras de negócio, garantindo padronização e consistência no desenvolvimento do software.

Quadro 3.1 – Dicionário de dados da entidade *Product*

Atributo	Tipo	Descrição
Id	long	Código único que identifica um produto.
Description	String	Nome comercial ou descrição principal do produto (ex.: “Dipirona 500mg Caixa com 20”).
GenericName	String	Nome genérico ou princípio ativo do produto (ex.: “Dipirona Monoidratada”).
MinimumStock	decimal	Quantidade mínima ideal que deve haver deste produto em estoque.
CurrentStock	decimal	Quantidade total atual deste produto em estoque (soma de todos os lotes).
StockValue	decimal	Valor monetário total do estoque atual para este produto.
UnitPrice	decimal	Preço de venda unitário do produto.
AverageCost	decimal	Custo médio de aquisição do produto, ponderado pelas compras.
LastPurchasePrice	decimal	Preço unitário da última compra realizada para este produto.
HighCost	YesNo	Enum (Sim/Não) que indica se o produto é classificado como de alto custo.
ExpirationControlled	YesNo	Enum (Sim/Não) que indica se o produto requer controle de data de validade.

Fonte: Elaborado pelos autores.

O *Quadro 3.2* apresenta o dicionário de dados da entidade *ProductBatch*, responsável por representar os lotes associados a cada produto. Seus atributos permitem o controle detalhado de identificação, validade, quantidade e valor em estoque, assegurando rastreabilidade e precisão nos processos de gestão de materiais.

Quadro 3.2 – Dicionário de dados da entidade ProductBatch

Atributo	Tipo	Descrição
Id	long	Código único que identifica um lote de produto.
BatchNumber	String	Código ou número que identifica o lote específico do fabricante.
ExpirationDate	DateTime	Data de validade do lote.
CurrentQuantity	decimal	Quantidade atual de itens restantes neste lote específico.
CurrentStockValue	decimal	Valor monetário total do estoque restante neste lote.

Fonte: Elaborado pelos autores.

O *Quadro 3.3* descreve a entidade *ProductType*, utilizada para classificar os produtos conforme sua natureza ou finalidade. Essa categorização é fundamental para organizar o catálogo do sistema, apoiar regras de negócio específicas e facilitar o gerenciamento de diferentes grupos de itens.

Quadro 3.3 – Dicionário de dados da entidade ProductType

Atributo	Tipo	Descrição
Id	int	Código único que identifica o tipo de produto.
Name	String	Nome que descreve o tipo do produto (ex.: “Medicamento”, “Material de Higiene”).

Fonte: Elaborado pelos autores.

O *Quadro 3.4* apresenta o dicionário de dados da entidade *UnitOfMeasure*, que define as unidades de medida utilizadas pelo sistema. A padronização dessas unidades garante consistência na movimentação de estoque, nos cálculos de valor e na integração entre diferentes módulos do software.

Quadro 3.4 – Dicionário de dados da entidade UnitOfMeasure

Atributo	Tipo	Descrição
Id	int	Código único que identifica a unidade de medida.
Description	String	Descrição por extenso da unidade de medida (ex.: “Quilograma”, “Unidade”).
Abbreviation	String	Abreviatura da unidade de medida (ex.: “Kg”, “Un”).

Fonte: Elaborado pelos autores.

O *Quadro 3.5* mostra o dicionário de dados da entidade *Manufacturer*, responsável por registrar as informações cadastrais de fabricantes e laboratórios. A correta identificação desses fornecedores é essencial para a rastreabilidade dos produtos e para assegurar conformidade documental e regulatória.

Quadro 3.5 – Dicionário de dados da entidade Manufacturer

Atributo	Tipo	Descrição
Id	int	Código único que identifica um fabricante ou laboratório.
CorporateName	String	Nome legal e formal do fabricante (Razão Social).
Tradename	String	Nome de marca ou comercial do fabricante (Nome Fantasia).
CpfCnpj	String	Número do CPF ou CNPJ do fabricante.

Fonte: Elaborado pelos autores.

O *Quadro 3.6* apresenta o dicionário de dados da entidade *StockMovement*, que registra todas as movimentações realizadas no estoque. Seus atributos permitem identificar o tipo de movimentação, sua origem, os valores envolvidos e o documento que deu causa à operação, fornecendo suporte essencial para auditoria, rastreabilidade e controle financeiro.

Quadro 3.6 – Dicionário de dados da entidade StockMovement

Atributo	Tipo	Descrição
Id	long	Código único que identifica uma movimentação de estoque.
MovementDate	DateTime	Data e hora em que a movimentação de estoque ocorreu.
StockMovementType	StockMovementType	Enum que define o tipo de movimentação (Entrada ou Saída).
StockRequestType	StockRequestType	Enum que define a origem ou motivo da movimentação (Compra, Consumo, Perda, etc.).
ReferenceDocument	String	Número do documento de referência que gerou a movimentação (ex.: Nota Fiscal).
Quantity	decimal	Quantidade de produto que foi movimentada.
UnitValue	decimal	Valor unitário do produto na movimentação.
TotalValue	decimal	Valor total da movimentação (Quantidade × Valor Unitário).

Fonte: Elaborado pelos autores.

O *Quadro 3.7* apresenta o dicionário de dados da entidade *Supplier*, que armazena as informações cadastrais de fornecedores. Esses dados incluem identificação jurídica, contatos

e endereço completo, permitindo ao sistema manter registros confiáveis e essenciais para processos de compras, auditorias e conformidade regulatória.

Quadro 3.7 – Dicionário de dados da entidade Supplier

Atributo	Tipo	Descrição
Id	int	Código único que identifica um fornecedor.
CorporateName	String	Nome legal e formal do fornecedor (Razão Social).
TradeName	String	Nome de marca pelo qual o fornecedor é conhecido (Nome Fantasia).
CpfCnpj	String	Número de CPF ou CNPJ do fornecedor.
Email	String	Endereço de e-mail principal de contato.
Phone	String	Número de telefone principal de contato.
PostalCode	String	Código de Endereçamento Postal (CEP).
Street	String	Rua ou logradouro do endereço do fornecedor.
Number	String	Número do endereço do fornecedor.
District	String	Bairro do endereço do fornecedor.
City	String	Cidade do endereço do fornecedor.
State	String	Sigla do estado (UF) do endereço do fornecedor.

Fonte: Elaborado pelos autores.

O Quadro 3.8 apresenta o enum *StockMovementType*, utilizado para classificar as movimentações de estoque como entrada ou saída. Essa distinção é fundamental para o correto funcionamento das rotinas de controle de estoque, garantindo consistência nas operações e nos registros históricos.

Quadro 3.8 – Dicionário de dados do enum StockMovementType

Valor	Tipo	Descrição
STOCKIN	int	Valor do enum que representa uma entrada de estoque.
STOCKOUT	int	Valor do enum que representa uma saída de estoque.

Fonte: Elaborado pelos autores.

O Quadro 3.9 descreve o enum *StockRequestType*, que identifica a origem da movimentação de estoque. Essa categorização é essencial para diferenciar compras, consumos, doações, perdas e ajustes, permitindo análises e controles precisos sobre o comportamento do estoque.

Quadro 3.9 – Dicionário de dados do enum StockRequestType

Valor	Tipo	Descrição
PURCHASE	int	Representa uma movimentação originada de compra.
CONSUMPTION	int	Representa uma movimentação originada do consumo interno.
DONATION	int	Representa uma movimentação originada de doação.
LOSS	int	Representa uma baixa no estoque por perda ou avaria.
ADJUSTMENT	int	Representa uma movimentação por ajuste de inventário.

Fonte: Elaborado pelos autores.

O *Quadro 3.10* apresenta o dicionário de dados da entidade *PurchaseRequest*, responsável por registrar o processo de requisição e fechamento de compras. Seus atributos contemplam as datas de abertura e conclusão, os valores obtidos nas cotações com diferentes fornecedores, o valor final contratado e o status da compra, oferecendo uma visão estruturada de todo o ciclo de aquisição.

Quadro 3.10 – Dicionário de dados da entidade PurchaseRequest

Atributo	Tipo	Descrição
Id	long	Código único que identifica uma requisição de compra.
RequestStartDate	DateTime	Data em que a requisição de compra foi iniciada.
PurchaseDate	DateTime	Data em que a compra foi efetivamente realizada.
PurchaseValueSupplierA	decimal	Valor total da cotação com o Fornecedor A.
PurchaseValueSupplierB	decimal	Valor total da cotação com o Fornecedor B.
PurchaseValueSupplierC	decimal	Valor total da cotação com o Fornecedor C.
PurchaseBestValue	decimal	Melhor valor (menor) entre as cotações realizadas.
PurchaseValue	decimal	Valor final pelo qual a compra foi fechada.
PurchaseStatus	PurchaseStatus	Enum que indica o status atual do processo de compra.
RequestCloseDate	DateTime	Data em que a requisição de compra foi concluída ou cancelada.

Fonte: Elaborado pelos autores.

O *Quadro 3.11* apresenta o dicionário de dados da entidade *PurchaseRequestItem*, que detalha os itens vinculados a uma requisição de compra. Nessa estrutura são descritas

as quantidades solicitadas e recebidas, bem como os preços unitários e valores totais obtidos nas cotações com diferentes fornecedores, permitindo análise comparativa e transparência no processo de aquisição.

Quadro 3.11 – Dicionário de dados da entidade PurchaseRequestItem

Atributo	Tipo	Descrição
Id	long	Código único que identifica um item dentro de uma requisição de compra.
RequestedQuantity	decimal	Quantidade do produto que foi solicitada.
ReceivedQuantity	decimal	Quantidade do produto que foi efetivamente recebida.
UnitPriceSupplierA	decimal	Preço unitário do produto na cotação com o Fornecedor A.
ProductValueSupplierA	decimal	Valor total do produto na cotação com o Fornecedor A.
UnitPriceSupplierB	decimal	Preço unitário do produto na cotação com o Fornecedor B.
ProductValueSupplierB	decimal	Valor total do produto na cotação com o Fornecedor B.
UnitPriceSupplierC	decimal	Preço unitário do produto na cotação com o Fornecedor C.
ProductValueSupplierC	decimal	Valor total do produto na cotação com o Fornecedor C.

Fonte: Elaborado pelos autores.

O Quadro 3.12 apresenta o dicionário de dados da entidade *Employee*, responsável por armazenar as informações cadastrais dos colaboradores da instituição. Os atributos contemplam dados pessoais, de contato, endereço e situação funcional, servindo de base para os módulos de gestão de recursos humanos e controle de acesso ao sistema.

Quadro 3.12 – Dicionário de dados da entidade Employee

Atributo	Tipo	Descrição
Id	int	Código único que identifica um funcionário.
Name	String	Nome completo do funcionário.
Cpf	String	Número do CPF do funcionário.
Phone	String	Telefone de contato do funcionário.
Email	String	Endereço de e-mail do funcionário.
HireDate	DateTime	Data em que o funcionário foi contratado.
State	String	Estado (UF) do endereço do empregado.
City	String	Cidade do endereço do empregado.
Street	String	Rua do endereço do empregado.
Cep	String	Código de Endereçamento Postal (CEP) do endereço do empregado.
Number	int	Número do endereço do empregado.
Neighborhood	String	Bairro ou vizinhança do endereço do empregado.
statusEmployee	StatusEmployee	Enum que indica a situação atual do funcionário (Ativo, Demitido, etc.).

Fonte: Elaborado pelos autores.

O *Quadro 3.13* apresenta o dicionário de dados da entidade *Position*, utilizada para representar os cargos existentes na instituição. Essa estrutura permite padronizar a nomenclatura de funções e apoiar rotinas de alocação de pessoal, hierarquia organizacional e controle de permissões no sistema.

Quadro 3.13 – Dicionário de dados da entidade Position

Atributo	Tipo	Descrição
Id	int	Código único que identifica um cargo na instituição.
Name	String	Nome do cargo (ex.: “Enfermeiro Chefe”, “Fisioterapeuta”).

Fonte: Elaborado pelos autores.

O *Quadro 3.14* apresenta o enum *PurchaseStatus*, utilizado para representar as etapas do processo de compra dentro do sistema. Esses estados permitem acompanhar a evolução da requisição, desde sua abertura até o encerramento, garantindo rastreabilidade e controle das movimentações de aquisição.

Quadro 3.14 – Dicionário de dados do enum PurchaseStatus

Valor	Tipo	Descrição
REQUESTED	int	Indica que a compra foi solicitada.
QUOTED	int	Indica que a compra está na fase de cotação.
APPROVED	int	Indica que a compra foi aprovada.
ORDERED	int	Indica que o pedido de compra foi enviado ao fornecedor.
CLOSED	int	Indica que o processo de compra foi finalizado.

Fonte: Elaborado pelos autores.

O *Quadro 3.15* descreve o enum *YesNo*, utilizado para representar respostas booleanas em formato textual dentro do sistema. Essa estrutura padroniza o uso das opções “Sim” e “Não” em diversos módulos e funcionalidades.

Quadro 3.15 – Dicionário de dados do enum YesNo

Valor	Tipo	Descrição
Yes	int	Representa a resposta “Sim”.
No	int	Representa a resposta “Não”.

Fonte: Elaborado pelos autores.

O *Quadro 3.16* apresenta o dicionário de dados da entidade *StockRequest*, que representa solicitações de saída de estoque. Seus atributos contemplam datas, descrição e informações essenciais para rastrear a operação e justificar a movimentação dos produtos.

Quadro 3.16 – Dicionário de dados da entidade StockRequest

Atributo	Tipo	Descrição
Id	long	Código único que identifica uma requisição de estoque (saída).
RequestDate	DateTime	Data em que a requisição foi feita.
RequestFullFillmentDate	DateTime	Data em que a requisição foi atendida.
Description	String	Descrição ou justificativa da requisição de estoque.

Fonte: Elaborado pelos autores.

O *Quadro 3.17* apresenta o dicionário de dados da entidade *StockRequestItem*, que descreve os itens incluídos em uma requisição de estoque. Os atributos indicam quantidades e valores associados a cada produto solicitado.

Quadro 3.17 – Dicionário de dados da entidade StockRequestItem

Atributo	Tipo	Descrição
Id	int	Código único que identifica um item da requisição.
Quantity	decimal	Quantidade do produto solicitada.
UnitValue	decimal	Valor unitário do produto no momento da requisição.
TotalItem	decimal	Valor total do item (Quantidade $\times$ Valor Unitário).

Fonte: Elaborado pelos autores.

O Quadro 3.18 apresenta o dicionário de dados da entidade *Invoice*, que registra todas as informações referentes às notas fiscais. Seus atributos contemplam identificação, datas, impostos, valores de produtos e serviços, seguro, frete e demais encargos, permitindo controle detalhado das operações fiscais e contábeis.

Quadro 3.18 – Dicionário de dados da entidade Invoice

Atributo	Tipo	Descrição
Id	long	Código único que identifica uma nota fiscal.
InvoiceNumber	String	Número da nota fiscal.
InvoiceSeries	String	Série da nota fiscal.
InvoiceDate	DateTime	Data de emissão da nota fiscal.
DepartureDate	DateTime	Data de envio da mercadoria.
pdfDanfe	Text	Conteúdo do PDF da DANFE.
AcessKeyCode	String	Chave de acesso (44 dígitos).
DiscountValue	decimal	Valor total do desconto.
FreightValue	decimal	Valor total do frete.
TotalProductAmount	decimal	Valor total dos produtos.
TotalServiceAmount	decimal	Valor total dos serviços.
TotalInvoiceAmount	decimal	Valor total da nota fiscal.
InsuranceValue	decimal	Valor do seguro da carga.
IPIValue	decimal	Valor total do IPI.
ICMSBaseValue	decimal	Base de cálculo do ICMS.
ICMSValue	decimal	Valor total do ICMS.
ICMSSubBaseValue	decimal	Base de cálculo do ICMS-ST.
ICMSSubValue	decimal	Valor do ICMS-ST.
FCPSTValue	decimal	Valor do FCP-ST.
ISSQNBaseValue	decimal	Base de cálculo do ISSQN.
ISSQNValue	decimal	Valor total do ISSQN.
OtherChargesValue	decimal	Outras despesas acessórias.

Fonte: Elaborado pelos autores.

O Quadro 3.19 apresenta o dicionário de dados da entidade *InvoiceItem*, que descreve cada item pertencente a uma nota fiscal. São detalhados valores unitários, quantidades, impostos, rateios e demais elementos fiscais que compõem o cálculo do item.

Quadro 3.19 – Dicionário de dados da entidade *InvoiceItem*

Atributo	Tipo	Descrição
Id	long	Código único do item na nota fiscal.
ProductSupplierId	int	ID do produto conforme cadastro do fornecedor.
CFOPCode	String	Código Fiscal de Operações e Prestações.
CSOSNCode	String	Código de Situação da Operação (Simples Nacional).
UnitPrice	decimal	Preço unitário na nota fiscal.
Quantity	decimal	Quantidade do produto na nota fiscal.
UnitCost	decimal	Custo unitário considerando impostos e despesas.
TotalPrice	decimal	Valor total do item.
TransportationAllocation	decimal	Rateio do frete.
InsuranceAllocation	decimal	Rateio do seguro.
OtherChargesAllocation	decimal	Rateio de outras despesas.
DiscountAllocation	decimal	Rateio do desconto.
ICMSTaxRate	decimal	Alíquota do ICMS.
IPITaxRate	decimal	Alíquota do IPI.
ICMSBaseValue	decimal	Base de cálculo do ICMS.
ICMSValue	decimal	Valor do ICMS.
IPIBaseValue	decimal	Base de cálculo do IPI.
IPIValue	decimal	Valor do IPI.

Fonte: Elaborado pelos autores.

O Quadro 3.20 apresenta o dicionário de dados da entidade *ProductGroup*, utilizada para classificar os produtos em categorias organizacionais dentro do sistema. Essa estrutura permite agrupar itens conforme sua natureza ou finalidade, facilitando consultas, controles e relatórios.

Quadro 3.20 – Dicionário de dados da entidade *ProductGroup*

Atributo	Tipo	Descrição
Id	int	Código único que identifica um grupo de produtos.
Name	String	Nome do grupo de produtos (ex.: "Remédios", "Materiais de Limpeza").

Fonte: Elaborado pelos autores.

O *Quadro 3.21* apresenta o dicionário de dados da interface *IStockDocument*, responsável por definir propriedades essenciais relacionadas à movimentação e requisição de estoque. Ela serve como contrato para classes que representam documentos transacionais no módulo de estoque.

Quadro 3.21 – Dicionário de dados da interface *IStockDocument*

Propriedade	Tipo	Descrição
StockMovementType	StockMovementType	Define o tipo de movimentação de estoque.
StockRequestType	StockRequestType	Define o tipo de requisição de estoque.

Fonte: Elaborado pelos autores.

O *Quadro 3.22* apresenta o dicionário de dados da entidade *Carrier*, que representa transportadoras responsáveis pelo envio e recebimento de mercadorias. Os atributos abrangem identificação jurídica, informações de contato e endereço completo, essenciais para integração logística e rastreabilidade.

Quadro 3.22 – Dicionário de dados da entidade *Carrier*

Atributo	Tipo	Descrição
Id	int	Código único que identifica uma transportadora.
CorporateName	String	Nome legal da transportadora (Razão Social).
TradeName	String	Nome comercial da transportadora (Nome Fantasia).
CpfCNPJ	String	Número do CPF ou CNPJ.
Street	String	Rua ou logradouro do endereço.
Number	String	Número do imóvel.
District	String	Bairro do endereço.
AddressComplement	String	Complemento do endereço.
City	String	Cidade do endereço.
State	String	Estado (UF) do endereço.
PostalCode	String	Código de Endereçamento Postal (CEP).
Phone	String	Telefone de contato.
Email	String	E-mail de contato.

Fonte: Elaborado pelos autores.

O *Quadro 3.23* apresenta o dicionário de dados da entidade *User*, responsável por armazenar as informações de autenticação e perfil de acesso dos usuários do sistema. Seus atributos incluem credenciais, tipos de perfil e status, fundamentais para controle de permissões e segurança.

Quadro 3.23 – Dicionário de dados da entidade User

Atributo	Tipo	Descrição
Id	int	Código único que identifica o usuário.
Email	String	Endereço de e-mail utilizado para login.
Password	String	Senha de acesso ao sistema.
UserType	UserType	Enum que define o perfil de permissões do usuário.
UserStatus	UserStatus	Enum que define a situação do usuário no sistema.

Fonte: Elaborado pelos autores.

O *Quadro 3.24* apresenta o enum *UserType*, que define os diferentes perfis de acesso disponíveis no sistema. Esses perfis determinam os níveis de permissão e funcionamento das regras de segurança e controle administrativo.

Quadro 3.24 – Dicionário de dados do enum UserType

Valor	Tipo	Descrição
ADMIN	int	Perfil com acesso total ao sistema.
MANAGER	int	Perfil com permissões de gerenciamento.
OPERATOR	int	Perfil com permissões operacionais padrão.
NURSE	int	Perfil para profissionais de enfermagem.
PHYSIOTHERAPIST	int	Perfil para fisioterapeutas.
NUTRITIONIST	int	Perfil para nutricionistas.
PSYCHOLOGIST	int	Perfil para psicólogos.
DOCTOR	int	Perfil para médicos.
NURSINGTECHNICIAN	int	Perfil para técnicos de enfermagem.

Fonte: Elaborado pelos autores.

O *Quadro 3.25* descreve o enum *UserStatus*, que representa os diferentes estados possíveis de um usuário no sistema, refletindo sua permissão de acesso e situação funcional.

Quadro 3.25 – Dicionário de dados do enum UserStatus

Valor	Tipo	Descrição
ACTIVE	int	Usuário ativo com acesso permitido.
INACTIVE	int	Usuário desativado sem acesso ao sistema.
BLOCKED	int	Usuário temporariamente bloqueado.

Fonte: Elaborado pelos autores.

O *Quadro 3.26* apresenta o dicionário de dados da entidade *Religion*, utilizada para registrar informações referentes às diferentes religiões cadastradas no sistema. Essa estrutura auxilia na identificação, organização e padronização dos dados pessoais dos residentes ou usuários.

Quadro 3.26 – Dicionário de dados da entidade Religion

Atributo	Tipo	Descrição
Id	int	Código único que identifica uma religião.
Name	String	Nome da religião.

Fonte: Elaborado pelos autores.

O *Quadro 3.27* apresenta o enum *MaritalStatus*, utilizado para representar os diferentes estados civis possíveis no sistema. Essa padronização permite organizar informações demográficas e garantir consistência nos registros pessoais.

Quadro 3.27 – Dicionário de dados do enum MaritalStatus

Valor	Tipo	Descrição
SINGLE	int	Estado civil "Solteiro(a)".
MARRIED	int	Estado civil "Casado(a)".
WIDOWED	int	Estado civil "Viúvo(a)".
DIVORCED	int	Estado civil "Divorciado(a)".
SEPARATED	int	Estado civil "Separado(a)".
CIVIL_UNION	int	Estado civil "União Estável".

Fonte: Elaborado pelos autores.

O *Quadro 3.28* descreve o enum *EducationLevel*, responsável por representar os diferentes níveis de escolaridade dos residentes ou usuários. Esse conjunto de valores permite a categorização padronizada de informações educacionais dentro do sistema.

Quadro 3.28 – Dicionário de dados do enum EducationLevel

Valor	Tipo	Descrição
NO_SCHOOLING	int	Analfabeto/Sem escolaridade.
ELEMENTARY_SCHOOL_COMPLETED	int	Ensino Fundamental Completo.
ELEMENTARY_SCHOOL_INCOMPLETE	int	Ensino Fundamental Incompleto.
HIGH_SCHOOL_COMPLETED	int	Ensino Médio Completo.
HIGH_SCHOOL_INCOMPLETE	int	Ensino Médio Incompleto.
HIGHER_EDUCATION_COMPLETED	int	Ensino Superior Completo.
HIGHER_EDUCATION_INCOMPLETE	int	Ensino Superior Incompleto.
POSTGRADUATE	int	Pós-graduado.

Fonte: Elaborado pelos autores.

O *Quadro 3.29* apresenta o enum *Ethnicity*, utilizado para indicar a autodeclaração de cor ou etnia do residente. Essa informação segue classificações amplamente utilizadas em sistemas oficiais de saúde e registros populacionais.

Quadro 3.29 – Dicionário de dados do enum *Ethnicity*

Valor	Tipo	Descrição
WHITE	int	Etnia/cor "Branca".
BLACK	int	Etnia/cor "Preta".
MIXED	int	Etnia/cor "Parda".
ASIAN	int	Etnia/cor "Amarela".
INDIGENOUS	int	Etnia/cor "Indígena".

Fonte: Elaborado pelos autores.

O *Quadro 3.30* descreve o enum *Sex*, utilizado para representar o sexo biológico declarado pelo residente. Essa padronização é fundamental para registros administrativos e clínicos.

Quadro 3.30 – Dicionário de dados do enum *Sex*

Valor	Tipo	Descrição
MALE	int	Sexo "Masculino".
FEMALE	int	Sexo "Feminino".

Fonte: Elaborado pelos autores.

O *Quadro 3.31* apresenta o dicionário de dados da entidade *Resident*, que representa os residentes assistidos pela instituição. Seus atributos abrangem informações pessoais, documentos, contatos, dados demográficos e características físicas, permitindo um cadastro completo e padronizado.

Quadro 3.31 – Dicionário de dados da entidade Resident

Atributo	Tipo	Descrição
Id	int	Código único do residente.
RegisteredName	String	Nome civil completo.
SocialName	String	Nome pelo qual o residente prefere ser chamado.
DateOfBirth	DateTime	Data de nascimento.
Age	String	Idade do residente.
Cpf	String	Número do CPF.
Rg	String	Número do RG.
IssuingBody	String	Órgão emissor do RG.
IssuingState	String	Estado emissor do RG.
PisPasep	String	Número do PIS/PASEP.
Sex	Sex	Enum que define o sexo do residente.
MaritalStatus	MaritalStatus	Enum que define o estado civil.
Ethnicity	Ethnicity	Enum que define a etnia/cor.
FatherName	String	Nome completo do pai.
MotherName	String	Nome completo da mãe.
SpouseName	String	Nome do cônjuge, se aplicável.
NationalHealthCardNumber	String	Número do Cartão Nacional de Saúde (CNS).
PrivateHealthCardNumber	String	Número do plano de saúde privado.
MobileNumber	String	Telefone celular.
HomePhoneNumber	String	Telefone fixo.
Height	decimal	Altura em metros (ex.: 1.75).
Weight	decimal	Peso em kg (ex.: 70.5).

Fonte: Elaborado pelos autores.

O *Quadro 3.32* apresenta o dicionário de dados da entidade *HealthInsurancePlan*, que registra os diferentes planos de saúde utilizados pelos residentes. Seus atributos classificam o plano, definem sua natureza e organizam informações para uso administrativo e clínico.

Quadro 3.32 – Dicionário de dados da entidade HealthInsurancePlan

Atributo	Tipo	Descrição
Id	int	Código único do plano de saúde.
Name	String	Nome do plano de saúde.
Type	HealthPlanType	Enum que define o plano como público ou privado.
Abbreviation	String	Sigla ou abreviação do plano.

Fonte: Elaborado pelos autores.

O *Quadro 3.33* apresenta o enum *HealthPlanType*, utilizado para classificar planos de saúde conforme sua natureza institucional: público ou privado.

Quadro 3.33 – Dicionário de dados do enum *HealthPlanType*

Valor	Tipo	Descrição
PUBLIC	int	Representa um plano de saúde público.
PRIVATE	int	Representa um plano de saúde privado.

Fonte: Elaborado pelos autores.

O *Quadro 3.34* apresenta o dicionário de dados da entidade *ResidentAllergy*, utilizada para registrar alergias específicas de cada residente. Essa entidade consolida informações descritivas, datas de detecção e encerramento, permitindo acompanhamento clínico.

Quadro 3.34 – Dicionário de dados da entidade *ResidentAllergy*

Atributo	Tipo	Descrição
Id	int	Código único do registro de alergia do residente.
Description	String	Descrição detalhada da reação alérgica.
DetectionDate	DateTime	Data de diagnóstico ou registro da alergia.
ReleaseDate	DateTime	Data de descontinuação do registro (se aplicável).

Fonte: Elaborado pelos autores.

O *Quadro 3.35* apresenta o dicionário de dados da entidade *Allergy*, que define os tipos gerais de alergias registráveis no sistema. Essa entidade permite categorização padronizada para utilização em registros clínicos e administrativos.

Quadro 3.35 – Dicionário de dados da entidade *Allergy*

Atributo	Tipo	Descrição
Id	int	Código único da alergia.
Name	String	Nome da alergia.
Type	AllergyType	Enum que classifica o tipo da alergia (ex.: Medicação, Alimento).

Fonte: Elaborado pelos autores.

O *Quadro 3.36* apresenta o dicionário de dados da entidade *ResidentRelative*, responsável por registrar informações completas sobre os familiares ou responsáveis legais vinculados ao residente. Essa estrutura permite rastrear dados pessoais, endereço, documentos oficiais e grau de parentesco.

Quadro 3.36 – Dicionário de dados da entidade ResidentRelative

Atributo	Tipo	Descrição
Id	int	Código único que identifica um familiar de um residente.
Name	String	Nome completo do familiar.
Cpf	String	CPF do familiar.
Rg	String	RG do familiar.
IssuingBody	String	Órgão emissor do RG do familiar.
IssuingState	String	Estado (UF) de emissão do RG.
Relationship	Relationship	Enum que define o grau de parentesco.
MobileNumber	String	Número do telefone celular.
HomePhoneNumber	String	Número do telefone fixo.
Email	String	Endereço de e-mail do familiar.
Street	String	Logradouro do endereço.
Number	String	Número do imóvel.
District	String	Bairro do endereço.
AddressComplement	String	Complemento do endereço.
City	String	Cidade do endereço.
State	String	Sigla do estado (UF).
PostalCode	String	Código de Endereçamento Postal (CEP).

Fonte: Elaborado pelos autores.

O Quadro 3.37 apresenta o enum *Relationship*, que padroniza os diferentes graus de parentesco aceitos pelo sistema. Ele é empregado nos registros de familiares de residentes, garantindo consistência e categorização adequada das relações familiares.

Quadro 3.37 – Dicionário de dados do enum Relationship

Valor	Tipo	Descrição
FATHER	int	Parentesco: Pai.
MOTHER	int	Parentesco: Mãe.
BROTHER_SISTER	int	Parentesco: Irmão(ã).
SON_DAUGHTER	int	Parentesco: Filho(a).
GRANDFATHER_GRANDMOTHER	int	Parentesco: Avô(ó).
UNCLE_AUNT	int	Parentesco: Tio(a).
NEPHEW_NIECE	int	Parentesco: Sobrinho(a).
COUSIN	int	Parentesco: Primo(a).
SPOUSE	int	Parentesco: Cônjuge.

Fonte: Elaborado pelos autores.

O Quadro 3.38 descreve o enum *AllergyType*, utilizado para classificar os diferentes

tipos de alergias registráveis no sistema. Essa categorização permite organizar adequadamente alergias medicamentosas, alimentares, ambientais e outros tipos relevantes no contexto clínico.

Quadro 3.38 – Dicionário de dados do enum AllergyType

Valor	Tipo	Descrição
MEDICATION	int	Alergias relacionadas a medicamentos.
FOOD	int	Alergias a alimentos.
ENVIRONMENTAL	int	Alergias ambientais (ex.: pólen, poeira).
INSECT_STING	int	Alergias a picadas de insetos.
ANIMAL	int	Alergias a animais (ex.: pêlos).
LATEX	int	Alergia a látex.
OTHER	int	Categoria geral para outros tipos de alergias.

Fonte: Elaborado pelos autores.

O *Quadro 3.39* apresenta o dicionário de dados da entidade *NursingCarePlan*, utilizada para definir planos de cuidados de enfermagem, incluindo frequência e intervalo exato entre as ações planejadas. Essa entidade é essencial para a organização e execução sistematizada da assistência.

Quadro 3.39 – Dicionário de dados da entidade NursingCarePlan

Atributo	Tipo	Descrição
Id	long	Código único que identifica um plano de cuidados de enfermagem.
ServiceFrequency	int	Frequência de prestação do cuidado (ex.: a cada X horas).
Interval	TimeSpan	Intervalo exato entre as prestações de cuidado.

Fonte: Elaborado pelos autores.

O *Quadro 3.40* apresenta o dicionário de dados da entidade *NursingService*, que representa cada serviço individual de enfermagem disponível no sistema. Cada registro define ações assistenciais específicas realizadas pela equipe.

Quadro 3.40 – Dicionário de dados da entidade NursingService

Atributo	Tipo	Descrição
Id	int	Código único que identifica um serviço de enfermagem.

Fonte: Elaborado pelos autores.

O *Quadro 3.41* apresenta o dicionário de dados da entidade *Prescription*, utilizada para registrar prescrições médicas relacionadas aos residentes. Esta entidade funciona como documento central para organização e execução dos cuidados prescritos.

Quadro 3.41 – Dicionário de dados da entidade Prescription

Atributo	Tipo	Descrição
Id	int	Código único que identifica uma prescrição médica.

Fonte: Elaborado pelos autores.

O *Quadro 3.42* apresenta o dicionário de dados da entidade *Schedule*, responsável por armazenar as tarefas agendadas na rotina de cuidados. Inclui o momento exato de execução, garantindo organização e cumprimento das atividades previstas.

Quadro 3.42 – Dicionário de dados da entidade Schedule

Atributo	Tipo	Descrição
Id	long	Código único que identifica uma tarefa agendada.
DateTimeExecution	DateTime	Data e hora exata da execução prevista.

Fonte: Elaborado pelos autores.

O *Quadro 3.43* apresenta o dicionário de dados da entidade *CID10*, que armazena os códigos da Classificação Internacional de Doenças (CID-10), utilizados para fins epidemiológicos, administrativos e clínicos.

Quadro 3.43 – Dicionário de dados da entidade CID10

Atributo	Tipo	Descrição
Id	int	Código único que identifica um código da Classificação CID-10.

Fonte: Elaborado pelos autores.

O *Quadro 3.44* apresenta o dicionário de dados da entidade *Company*, que armazena informações cadastrais e institucionais da empresa responsável pela operação do sistema. Seus atributos contemplam dados de identificação, endereço, contato e logomarca.

Quadro 3.44 – Dicionário de dados da entidade Company

Atributo	Tipo	Descrição
Id	int	Código único da empresa que opera o sistema.
CompanyName	String	Nome legal da empresa (Razão Social).
TradeName	String	Nome comercial da empresa (Nome Fantasia).
Cnpj	String	Número do CNPJ da empresa.
Email	String	E-mail de contato principal.
Street	String	Logradouro do endereço da empresa.
Number	String	Número do imóvel.
District	String	Bairro do endereço.
AddressComplement	String	Complemento do endereço.
City	String	Cidade do endereço.
State	String	Sigla do estado (UF).
PostalCode	String	CEP da empresa.
CompanyLogo	Text	Arquivo contendo a imagem do logotipo institucional.

Fonte: Elaborado pelos autores.

O Quadro 3.45 apresenta o dicionário de dados da entidade *TechnicalResponsibility*, responsável por registrar informações sobre o profissional detentor da responsabilidade técnica da instituição, incluindo dados pessoais, registros profissionais e período de vigência.

Quadro 3.45 – Dicionário de dados da entidade TechnicalResponsibility

Atributo	Tipo	Descrição
Id	int	Código único do registro de responsabilidade técnica.
ResponsibleName	String	Nome do profissional responsável técnico.
ProfessionalRegistration	String	Registro profissional (ex.: CRM, CO-REN).
ServicesResponsability	String	Descrição dos serviços sob responsabilidade técnica.
StartDate	DateTime	Data de início da responsabilidade técnica.
EndDate	DateTime	Data de término da responsabilidade técnica.

Fonte: Elaborado pelos autores.

O Quadro 3.46 apresenta o dicionário de dados da entidade *SystemParameters*, utilizada para armazenamento de parâmetros configuráveis que controlam o comportamento

do sistema. Cada registro pode conter valores em diferentes tipos, conforme a necessidade de configuração.

Quadro 3.46 – Dicionário de dados da entidade SystemParameters

Atributo	Tipo	Descrição
Id	int	Código único do parâmetro de configuração.
ParameterName	String	Nome chave do parâmetro (ex.: TIME-OUT_SESSAO).
ParameterDescription	String	Descrição da função do parâmetro.
ParameterFlagType	String	Indica qual campo de valor deve ser utilizado.
ParameterFlagBoolean	boolean	Valor do parâmetro quando do tipo booleano.
ParameterFlagDate	DateTime	Valor do parâmetro quando do tipo data/hora.
ParameterFlagInt	int	Valor do parâmetro quando do tipo inteiro.
ParameterFlagDecimal	decimal	Valor do parâmetro quando do tipo decimal.
ParameterFlagString	String	Valor do parâmetro quando do tipo texto.

Fonte: Elaborado pelos autores.

O Quadro 3.47 apresenta o dicionário de dados da entidade *SystemModule*, que define os módulos funcionais disponíveis no sistema, bem como seu status de acessibilidade.

Quadro 3.47 – Dicionário de dados da entidade SystemModule

Atributo	Tipo	Descrição
Id	int	Código único do módulo do sistema.
Name	String	Nome do módulo (ex.: Estoque, Financeiro).
IsModuloAcessible	boolean	Indica se o módulo está acessível.

Fonte: Elaborado pelos autores.

O Quadro 3.48 apresenta o dicionário de dados da entidade *SystemModuloScreen*, responsável por definir as telas pertencentes a cada módulo, bem como sua disponibilidade de acesso.

Quadro 3.48 – Dicionário de dados da entidade SystemModuloScreen

Atributo	Tipo	Descrição
Id	int	Código único da tela do módulo.
Name	String	Nome da tela.
IsScreenAccessible	boolean	Indica se a tela está acessível.

Fonte: Elaborado pelos autores.

O Quadro 3.49 apresenta o dicionário de dados da entidade *UserTypeDefaultAccess*, responsável por registrar as permissões padrão atribuídas a cada tipo de usuário, definindo ações permitidas como leitura, criação, atualização ou remoção.

Quadro 3.49 – Dicionário de dados da entidade UserTypeDefaultAccess

Atributo	Tipo	Descrição
Id	int	Código único da regra de acesso padrão.
UserType	UserType	Tipo de usuário ao qual a regra se aplica.
ReadOnly	boolean	Permite apenas leitura.
WriteCreate	boolean	Permite criação de registros.
WriteUpdate	boolean	Permite atualização de registros.
WriteRemove	boolean	Permite remoção de registros.

Fonte: Elaborado pelos autores.

O Quadro 3.50 apresenta o enum *StatusEmployee*, empregado para representar a situação funcional de um colaborador dentro da instituição, abrangendo estados como ativo, demitido ou afastado.

Quadro 3.50 – Dicionário de dados do enum StatusEmployee

Valor	Tipo	Descrição
ACTIVE	int	Funcionário ativo.
FIRED	int	Funcionário desligado/demitido.
PAID_LEAVE	int	Funcionário em licença remunerada.
ANNUAL_LEAVE	int	Funcionário em férias.

Fonte: Elaborado pelos autores.

3.3 Definição de Atores

Conforme apresentado na Figura 3.2, os atores do sistema representam os diferentes perfis de usuário e suas interações diretas com as funcionalidades disponibilizadas

pela plataforma. Cada ator possui atribuições específicas, definidas de acordo com suas responsabilidades operacionais dentro da instituição. Essa modelagem permite identificar claramente os papéis, garantindo que as permissões e acessos sejam coerentes com as necessidades de cada tipo de usuário.

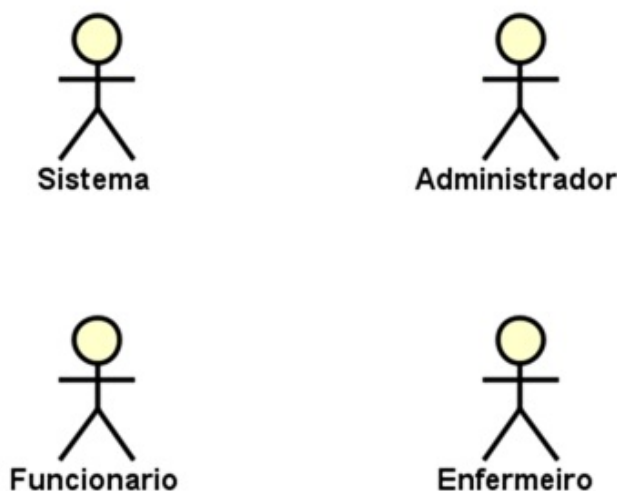
O ator *Administrador* (Figura 3.2) desempenha o papel central na supervisão e manutenção global do sistema. Ele é responsável pela gestão completa dos funcionários, dos idosos assistidos e dos demais administradores cadastrados. Suas interações incluem operações fundamentais como *Cadastrar*, *Listar*, *Atualizar* e *Excluir* os elementos sob sua responsabilidade. Esse perfil de usuário possui o nível de acesso mais elevado, permitindo a configuração de dados essenciais, o gerenciamento de permissões e o controle das rotinas administrativas da instituição. Dessa forma, o Administrador assegura o bom funcionamento do sistema e o alinhamento das operações às diretrizes organizacionais.

O ator *Enfermeiro* (Figura 3.2) representa o profissional de saúde diretamente envolvido no cuidado aos idosos. Suas funções incluem a realização de atendimentos, o acompanhamento clínico, a administração de medicamentos e vacinas, e o registro de informações assistenciais relevantes. No sistema, o Enfermeiro possui acesso às funcionalidades relacionadas ao prontuário eletrônico, como *Abrir Prontuário*, *Acessar SOAP*, *Acessar Acompanhamento* e *Acessar Histórico de Atendimento*. Além disso, possui permissão para consultar informações do estoque por meio da funcionalidade *Acessar Estoque*, garantindo que tenha visibilidade sobre os insumos necessários para a assistência. Esse ator desempenha papel crucial na manutenção da continuidade do cuidado e na atualização das informações de saúde dos idosos.

O ator *Funcionário* (Figura 3.2) está diretamente associado às operações diárias de gerenciamento do estoque da instituição. Ele é responsável pela execução de atividades essenciais, incluindo a entrada e saída de produtos, a realocação de itens, o controle de quantidades e o cadastro de novos produtos no inventário. O Funcionário atua sob orientação e supervisão do Administrador, seguindo protocolos operacionais para garantir precisão e rastreabilidade das movimentações realizadas. Seu papel é fundamental para a organização logística da instituição, sustentando o fluxo contínuo de materiais e contribuindo para o funcionamento eficiente das rotinas assistenciais.

Por fim, o ator *Sistema* (Figura 3.2) representa a própria plataforma digital responsável por processar informações, aplicar regras de negócio, manter registros estruturados e oferecer suporte às interações dos demais atores. Embora não seja um usuário humano, o Sistema desempenha um papel central ao automatizar tarefas, garantir consistência dos dados e disponibilizar funcionalidades essenciais para Administradores, Funcionários e Enfermeiros. Ele é a base operacional que integra processos, organiza informações e assegura que as atividades sejam realizadas de forma eficiente e segura.

Figura 3.2 – Diagrama de Atores



Fonte: Elaborado pelos autores (2025).

3.4 Casos de Uso

De acordo com **Guedes2018**, os casos de uso têm como propósito identificar os usuários de um sistema e os requisitos que orientam seu desenvolvimento, definindo “serviços, tarefas ou funcionalidades identificados como necessários ao software e que podem ser utilizados de alguma maneira pelos atores que interagem com o sistema”. Nesse contexto, a modelagem de casos de uso constitui uma etapa essencial no processo de engenharia de software, pois permite compreender, de maneira estruturada, o comportamento do sistema *SeniorCareManager* e as interações estabelecidas entre os diferentes perfis de usuários.

Com base nos requisitos levantados, foram elaborados quadros que apresentam, de forma detalhada, as operações acessíveis a cada ator do sistema — Administrador, Funcionário e Enfermeiro — de acordo com suas respectivas atribuições institucionais. Essa representação possibilita uma visão clara e organizada dos fluxos de trabalho e das responsabilidades de cada perfil, assegurando que o sistema atenda simultaneamente às demandas administrativas e assistenciais do Lar São Vicente de Paulo e da APAE de Jales, promovendo maior eficiência, rastreabilidade e segurança das informações.

3.4.1 Casos de Uso do Administrador

O ator Administrador desempenha a função de maior privilégio dentro do sistema *SeniorCareManager*, sendo responsável pela gestão integral e pela supervisão das atividades realizadas pelos demais usuários. As tarefas apresentadas no Quadro 3.51 são exclusivas desse ator e abrangem desde o cadastro e a manutenção de informações administrativas, como usuários, produtos e lotes, até o controle de entradas e saídas de materiais e a

configuração de parâmetros do sistema. Além de executar suas próprias operações, o Administrador possui permissão para acessar, acompanhar e intervir nas ações dos demais perfis — como Funcionários e Enfermeiros — assegurando a integridade dos dados, a conformidade dos processos e a eficiência das rotinas institucionais. Dessa forma, sua atuação é essencial para garantir a governança, a rastreabilidade e a confiabilidade do sistema como um todo.

Quadro 3.51 – Casos de Uso — Administrador

Nº	Descrição do Caso de Uso	Entrada	Caso de Uso	Resposta
01	Administrador cadastra funcionário	Dados do funcionário	Cadastrar Funcionário	Mensagem: “Funcionário cadastrado com sucesso.”
02	Administrador atualiza funcionário	Dados do funcionário	Atualizar Funcionário	Mensagem: “Funcionário atualizado com sucesso.”
03	Administrador lista funcionários	—	Listar Funcionários	Retorna lista de funcionários.
04	Administrador carrega funcionário	—	Carregar Funcionário	Retorna informações do funcionário.
05	Administrador exclui funcionário	—	Excluir Funcionário	Mensagem: “Funcionário excluído com êxito.”
06	Administrador cadastra produto	Dados do produto	Cadastrar Produto	Mensagem: “Produto cadastrado com sucesso.”
07	Administrador lista produtos	—	Listar Produtos	Retorna lista de produtos.
08	Administrador atualiza produto	Dados do produto	Atualizar Produto	Mensagem: “Produto atualizado com sucesso.”
09	Administrador exclui produto	—	Excluir Produto	Mensagem: “Produto excluído com êxito.”
10	Administrador carrega produto	—	Carregar Produto	Retorna informações do produto.
11	Administrador cadastra tipo de produto	Dados do tipo de produto	Cadastrar Tipo de Produto	Mensagem: “Tipo de produto cadastrado com sucesso.”
12	Administrador lista tipos de produto	—	Listar Tipos de Produto	Retorna lista de tipos de produto.
13	Administrador atualiza tipo de produto	Dados do tipo de produto	Atualizar Tipo de Produto	Mensagem: “Tipo de produto atualizado com sucesso.”

Continua na próxima página

Nº	Descrição do Caso de Uso	Entrada	Caso de Uso	Resposta
14	Administrador exclui tipo de produto	—	Excluir Tipo de Produto	Mensagem: “Tipo de produto excluído com êxito.”
15	Administrador carrega tipo de produto	—	Carregar Tipo de Produto	Retorna informações do tipo de produto.
16	Administrador cadastra lote	Dados do lote	Cadastrar Lote	Mensagem: “Lote cadastrado com sucesso.”
17	Administrador lista lotes	—	Listar Lotes	Retorna lista de lotes.
18	Administrador atualiza lote	Dados do lote	Atualizar Lote	Mensagem: “Lote atualizado com sucesso.”
19	Administrador exclui lote	—	Excluir Lote	Mensagem: “Lote excluído com êxito.”
20	Administrador carrega lote	—	Carregar Lote	Retorna informações do lote.
21	Administrador realiza saída	Dados da saída	Realizar Saída	Mensagem: “Saída realizada com sucesso.”
22	Administrador realiza entrada	Dados da entrada	Realizar Entrada	Mensagem: “Entrada realizada com sucesso.”
23	Administrador realiza login	E-mail e senha	Login	Redireciona à página inicial (home).

Fonte: elaborado pelo autor.

3.4.2 Casos de Uso do Funcionário

O ator Funcionário desempenha um papel operacional essencial no sistema *SeniorCare-Manager*, sendo responsável pela execução das atividades cotidianas relacionadas à gestão de produtos, lotes e controle de estoque. As ações atribuídas a esse perfil estão descritas de forma detalhada no Quadro 3.52, que apresenta os casos de uso aos quais o funcionário tem acesso. Esses casos englobam o cadastro, atualização, exclusão e consulta de informações pertinentes ao fluxo de materiais, além da realização de entradas e saídas de produtos. Dessa forma, o sistema assegura que o funcionário disponha de um ambiente controlado e eficiente para o desempenho de suas funções, mantendo a rastreabilidade das operações e a integridade dos dados registrados.

Quadro 3.52 – Casos de Uso — Funcionário

Nº	Descrição do Caso de Uso	Entrada	Caso de Uso	Resposta
01	Funcionário realiza login	E-mail e senha	Login	Retorna tela inicial.
02	Funcionário cadastra produto	Dados do produto	Cadastrar Produto	Mensagem: “Produto cadastrado com sucesso.”
03	Funcionário cadastra tipo de produto	Dados do tipo de produto	Cadastrar Tipo de Produto	Mensagem: “Tipo de produto cadastrado com sucesso.”
04	Funcionário cadastra lote	Dados do lote	Cadastrar Lote	Mensagem: “Lote cadastrado com sucesso.”
05	Funcionário lista produtos	—	Listar Produtos	Retorna lista de produtos.
06	Funcionário lista tipos de produto	—	Listar Tipos de Produto	Retorna lista de tipos de produto.
07	Funcionário lista lotes	—	Listar Lotes	Retorna lista de lotes.
08	Funcionário atualiza produto	Dados do produto	Atualizar Produto	Mensagem: “Produto atualizado com sucesso.”
09	Funcionário atualiza tipo de produto	Dados do tipo de produto	Atualizar Tipo de Produto	Mensagem: “Tipo de produto atualizado com sucesso.”
10	Funcionário atualiza lote	Dados do lote	Atualizar Lote	Mensagem: “Lote atualizado com sucesso.”
11	Funcionário exclui produto	—	Excluir Produto	Mensagem: “Produto excluído com êxito.”
12	Funcionário exclui tipo de produto	—	Excluir Tipo de Produto	Mensagem: “Tipo de produto excluído com êxito.”
13	Funcionário exclui lote	—	Excluir Lote	Mensagem: “Lote excluído com êxito.”
14	Funcionário carrega produto	ID do produto	Carregar Produto	Retorna informações do produto.
15	Funcionário carrega tipo de produto	ID do tipo de produto	Carregar Tipo de Produto	Retorna informações do tipo de produto.
16	Funcionário carrega lote	ID do lote	Carregar Lote	Retorna informações do lote.

Continua na próxima página

Nº	Descrição do Caso de Uso	Entrada	Caso de Uso	Resposta
17	Funcionário realiza entrada	Dados da entrada	Realizar Entrada	Mensagem: “Entrada realizada com sucesso.”
18	Funcionário realiza saída	Dados da saída	Realizar Saída	Mensagem: “Saída realizada com sucesso.”

Fonte: elaborado pelo autor.

3.4.3 Casos de Uso do Enfermeiro

O ator Enfermeiro possui atribuições voltadas ao acompanhamento assistencial e ao gerenciamento das informações clínicas dos residentes no sistema *SeniorCareManager*. Conforme demonstrado no Quadro 3.53, os casos de uso associados a esse perfil abrangem o cadastramento, atualização e exclusão de dados referentes a idosos, carteiras de vacinação, vacinas aplicadas e agendamentos. Além disso, o enfermeiro tem acesso a funcionalidades que permitem o controle do histórico vacinal e o monitoramento da saúde dos residentes, favorecendo uma gestão mais segura e organizada da assistência prestada. Dessa forma, as operações disponíveis ao enfermeiro contribuem diretamente para a qualidade do cuidado e para a integração das informações entre os diferentes setores da instituição.

Quadro 3.53 – Casos de Uso — Enfermeiro

Nº	Descrição do Caso de Uso	Entrada	Caso de Uso	Resposta
01	Pessoa cadastra carteira de vacinação	Dados da carteira de vacinação	Cadastrar Carteira de Vacinação	Mensagem 01
02	Enfermeiro cadastra dados da carteira de vacinação	Dados da carteira de vacinação	Cadastrar Dados da Carteira de Vacinação	Mensagem 02
03	Enfermeiro lista idosos	—	Listar Idosos	Mensagem 03
04	Enfermeiro cadastra idoso	Dados do idoso	Cadastrar Idoso	Mensagem 04
05	Enfermeiro edita idoso	ID e dados do idoso	Editar Idoso	Mensagem 05
06	Enfermeiro exclui idoso	ID do idoso	Excluir Idoso	Mensagem 06

Continua na próxima página

Nº	Descrição do Caso de Uso	Entrada	Caso de Uso	Resposta
07	Enfermeiro lista agendamentos de vacina	—	Listar Agendamento de Vacina	Mensagem 07
08	Enfermeiro cadastra agendamento de vacina	Dados do agendamento	Cadastrar Agendamento de Vacina	Mensagem 08
09	Enfermeiro edita dados do agendamento	ID e dados do agendamento	Editar Dados do Agendamento	Mensagem 09
10	Enfermeiro exclui idosos que não desejam vacinação	ID do idoso	Excluir Idoso Não Vacinado	Mensagem 10
11	Administrador cadastra lote de vacina	Dados do lote de vacina	Cadastrar Lote de Vacina	Mensagem 11
12	Administrador lista lotes de vacina	—	Listar Lotes de Vacina	Mensagem 12
13	Administrador exclui lote de vacina	ID do lote de vacina	Excluir Lote de Vacina	Mensagem 13
14	Administrador edita lote de vacina	ID e dados do lote	Editar Lote de Vacina	Mensagem 14
15	Administrador cadastra vacina aplicada	Dados da vacina aplicada	Cadastrar Vacina Aplicada	Mensagem 15
16	Administrador exclui vacina aplicada	ID da vacina aplicada	Excluir Vacina Aplicada	Mensagem 16
17	Administrador edita vacina aplicada	ID e dados da vacina aplicada	Editar Vacina Aplicada	Mensagem 17
18	Administrador lista vacinas aplicadas	—	Listar Vacinas Aplicadas	Mensagem 18
19	Administrador lista carteiras de vacinação	—	Listar Carteiras de Vacinação	Mensagem 19
20	Administrador lista dados da carteira de vacinação	ID da carteira	Listar Dados da Carteira de Vacinação	Mensagem 20

Continua na próxima página

Nº	Descrição do Caso de Uso	Entrada	Caso de Uso	Resposta
21	Administrador edita dados da carteira de vacinação	ID e dados da carteira	Editar Dados da Carteira de Vacinação	Mensagem 21
22	Administrador exclui dados da carteira de vacinação	ID da carteira	Excluir Dados da Carteira de Vacinação	Mensagem 22
23	Administrador exclui carteira de vacinação	ID da carteira	Excluir Carteira de Vacinação	Mensagem 23

Fonte: elaborado pelo autor.

3.5 Diagrama de Casos de Uso

Guedes2018 descreve o *diagrama de casos de uso* como uma representação do sistema a partir da perspectiva do usuário, cujo objetivo é apresentar, de forma geral, as funcionalidades que o sistema deve oferecer ao usuário. Além disso, Guedes (2018) destaca que esse diagrama é uma ferramenta essencial para identificar e compreender os requisitos do sistema, auxiliando na especificação, visualização e documentação das características, funções e serviços que o sistema deve proporcionar.

O diagrama de caso de uso geral fornece uma visão abrangente das diversas interações que cada usuário pode realizar no sistema. Essa representação é especialmente valiosa para obter uma compreensão global das funcionalidades disponíveis para cada ator envolvido. Nas Figuras 3.3, 3.4 e 3.5, são apresentados os casos de uso gerais, destacando as ações que cada ator pode executar no sistema. Essa abordagem oferece uma visão consolidada das capacidades do sistema, facilitando a compreensão das funcionalidades disponíveis para cada usuário.

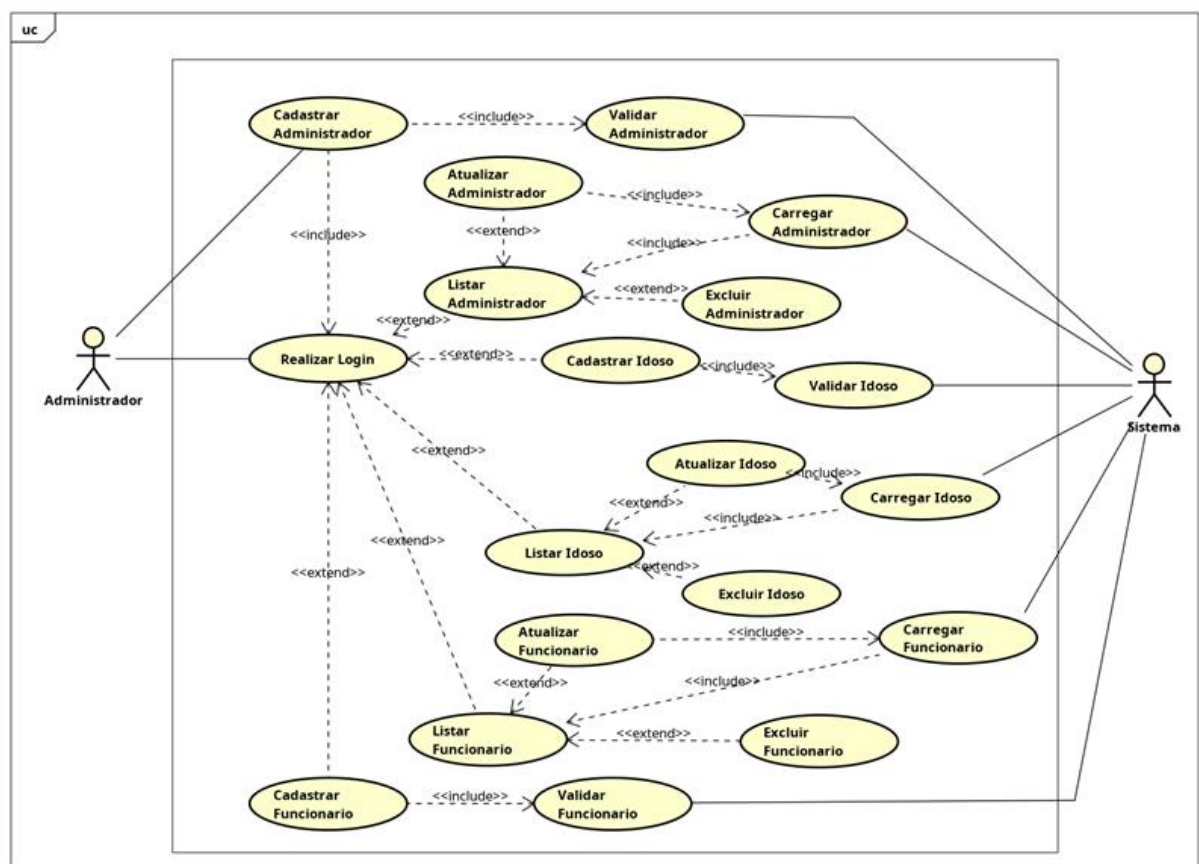
O diagrama de caso de uso geral apresentado na Figura 3.3 descreve todas as interações possíveis entre o ator Administrador e o Sistema, evidenciando as funcionalidades de gestão relacionadas aos usuários do sistema e aos idosos cadastrados. Esse diagrama oferece uma visão abrangente das operações administrativas, permitindo compreender como o ator atua na manutenção das informações essenciais para o funcionamento adequado do ambiente institucional.

Na visão do administrador, o processo tem início com o caso de uso *Realizar Login*, que habilita o acesso às funcionalidades seguintes. Após a autenticação, o administrador pode executar atividades referentes à gestão de outros administradores, funcionários e idosos. Entre essas funcionalidades destacam-se: *Cadastrar Administrador*, *Atualizar Administrador*, *Listar Administrador*, *Excluir Administrador* e *Validar Administrador*. Essas ações representam o controle hierárquico do sistema, permitindo que o administrador tenha domínio sobre os perfis de acesso e suas permissões.

O diagrama também evidencia as operações de gerenciamento de idosos, tais como *Cadastrar Idoso*, *Atualizar Idoso*, *Listar Idoso*, *Excluir Idoso*, *Validar Idoso* e *Carregar Idoso*. Todas essas funcionalidades são essenciais para manter atualizadas as informações clínicas e cadastrais dos residentes, garantindo consistência e rastreabilidade dos dados.

Além disso, o administrador possui acesso à gestão de funcionários por meio dos casos de uso *Cadastrar Funcionário*, *Atualizar Funcionário*, *Listar Funcionário*, *Excluir Funcionário*, *Validar Funcionário* e *Carregar Funcionário*. Cada um desses casos está relacionado ao fluxo de manutenção dos colaboradores que operam o sistema, permitindo que o administrador assegure controle organizacional e operacional.

Figura 3.3 – Diagrama de Caso de Uso Geral - Administrador



Fonte: Elaborada pelos autores.

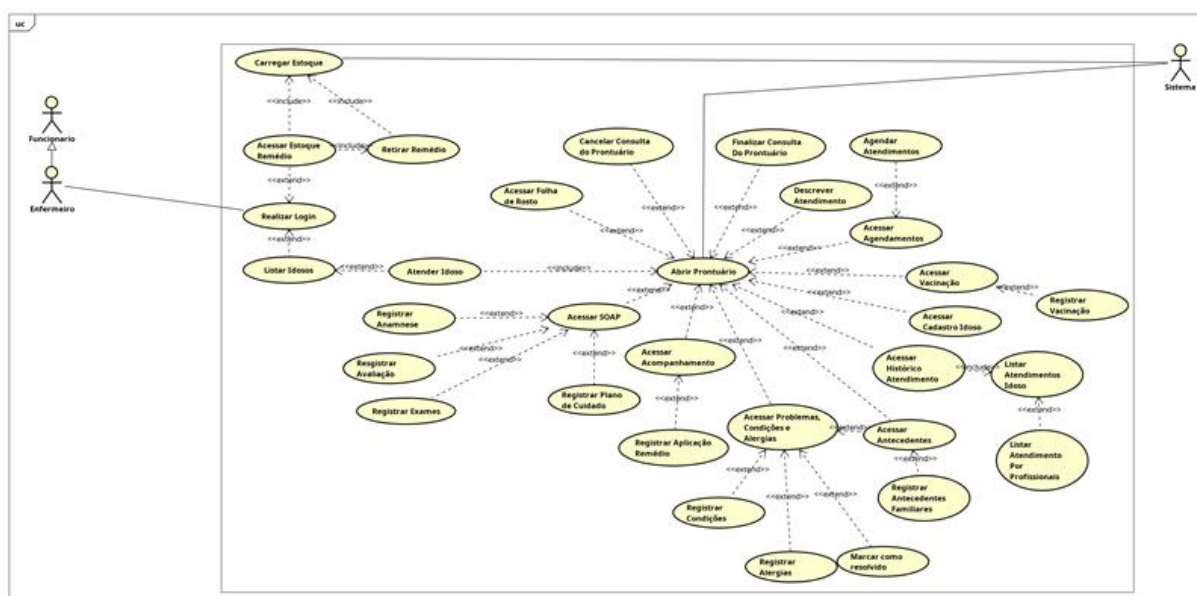
As relações *include* e *extend* presentes no diagrama demonstram a modularidade e a reutilização de comportamentos, reforçando a lógica de que algumas operações dependem de ações anteriores — como a necessidade de carregar dados antes de validá-los — ou podem ser especializadas a partir de um fluxo principal. Dessa forma, o diagrama sintetiza a complexidade e a amplitude das funções administrativas, oferecendo uma visão estruturada do papel central do Administrador no gerenciamento do sistema.

Na Figura 3.4, apresenta-se o diagrama de casos de uso geral na perspectiva do ator

Enfermeiro. Conforme exposto por **Guedes2018**, o diagrama de casos de uso tem como objetivo representar as funcionalidades do sistema a partir do ponto de vista do usuário, destacando as interações possíveis entre o ator e o sistema. Nesse contexto, o *Enfermeiro* é o profissional que utiliza o sistema para registrar e acompanhar as informações clínicas dos idosos assistidos.

Inicialmente, o *Enfermeiro* realiza o caso de uso *Realizar Login*, que lhe concede acesso às funcionalidades internas do sistema. A partir disso, ele pode executar o caso de uso *Listar Idosos*, que permite localizar o idoso desejado e, em seguida, acionar o caso de uso central *Abrir Prontuário*. Esse caso de uso funciona como eixo principal do diagrama, pois concentra a maior parte das interações clínicas realizadas pelo profissional de enfermagem.

Figura 3.4 – Diagrama de Caso de Uso Geral - Enfermeiro



Fonte: Elaborada pelos autores.

Uma vez com o prontuário aberto, o *Enfermeiro* pode acessar diferentes seções e registros, tais como *Acessar SOAP*, *Acessar Acompanhamento*, *Acessar Cadastro Idoso*, *Acessar Histórico de Atendimento* e *Acessar Problemas, Condições e Alergias*. Cada uma dessas funcionalidades dá suporte a ações específicas do cuidado, permitindo ao ator consultar e atualizar informações relevantes sobre o estado de saúde do idoso.

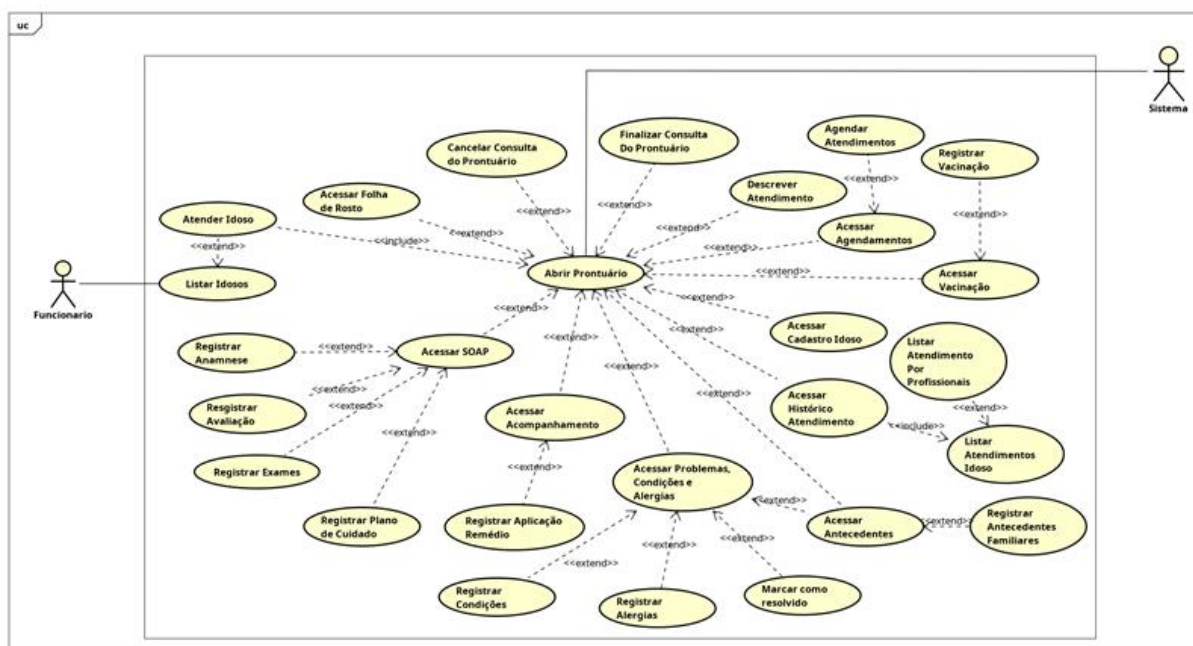
O diagrama também evidencia casos de uso voltados ao registro de informações assistenciais, como *Registrar Anamnese*, *Registrar Avaliação*, *Registrar Exames*, *Registrar Plano de Cuidado*, *Registrar Aplicação de Remédio*, *Registrar Condições*, *Registrar Alergias* e *Registrar Antecedentes Familiares*. Tais casos de uso estão associados ao processo de documentação sistemática da assistência, garantindo rastreabilidade e continuidade do cuidado. Além disso, o *Enfermeiro* pode marcar problemas como resolvidos, bem como acessar e registrar informações relacionadas a atendimentos, agendamentos e vacinação, assegurando uma visão longitudinal do acompanhamento clínico.

Dessa forma, o diagrama de caso de uso geral na visão do *Enfermeiro* apresenta um conjunto abrangente de funcionalidades orientadas à prática assistencial, reforçando o papel central desse profissional na utilização do sistema para registro, monitoramento e tomada de decisão em relação à saúde dos idosos.

O diagrama de caso de uso apresentado na Figura 3.5 ilustra, de forma ampliada, todas as interações possíveis entre o ator *Funcionário* e o Sistema no contexto do gerenciamento do prontuário eletrônico do idoso. A figura representa um panorama geral das funcionalidades acessíveis ao funcionário, evidenciando tanto os fluxos diretamente iniciados por ele quanto aqueles acionados como extensões ou inclusões decorrentes de outras operações.

Inicialmente, o funcionário pode listar idosos e, a partir dessa ação, proceder ao atendimento do idoso. O atendimento dá origem à operação de abrir prontuário, considerada central no diagrama, pois dela se ramificam diversas funcionalidades essenciais ao registro e acompanhamento clínico. Entre essas funcionalidades, destacam-se: Acessar SOAP, Registrar Anamnese, Registrar Avaliação, Registrar Exames, Registrar Plano de Cuidado, Registrar Aplicação de Remédio, Registrar Condições e Registrar Alergias. Todas essas operações estendem o acesso ao módulo SOAP, reforçando a estrutura lógica do prontuário clínico.

Figura 3.5 – Diagrama de Caso de Uso Geral - Profissional



Fonte: Elaborada pelos autores.

Além disso, o funcionário também interage com funções de gestão do atendimento, como Acessar Acompanhamento, Acessar Problemas, Condições e Alergias, Acessar Histórico de Atendimento, Acessar Antecedentes, Acessar Antecedentes Familiares, Acessar Cadastro do Idoso e Acessar Agendamentos. O diagrama também contempla ações administrativas relacionadas ao fluxo de atendimento, como Cancelar Consulta do Prontuário, Finalizar Consulta do Prontuário

e Marcar como Resolvido.

A modelagem demonstra ainda a interação indireta entre o funcionário e o sistema na emissão de notificações, descrição de atendimentos e registro de vacinação, ações que surgem como extensões específicas do processo principal. Ao organizar visualmente esses relacionamentos, o diagrama fornece uma visão clara e sistematizada das responsabilidades do ator e do comportamento esperado do sistema, facilitando tanto o entendimento quanto a documentação dos requisitos funcionais associados ao módulo de prontuário eletrônico.

3.6 Diagrama de Casos de Uso - Individuais

De acordo com **Guedes2018**, o diagrama de caso de uso é uma representação gráfica das interações entre um sistema e seus usuários. Seu principal objetivo é descrever, de maneira simples e intuitiva, as funcionalidades que o sistema oferece, facilitando o entendimento tanto para os desenvolvedores quanto para os usuários. Além disso, esse tipo de diagrama proporciona uma visão geral do funcionamento do sistema, destacando as interações com os diferentes tipos de usuários que podem acessá-lo.

Nos tópicos a seguir, serão apresentados alguns dos casos de uso do sistema, contemplando funcionalidades específicas de um CRUD do SOAP do prontuário do atendido pela instituição.

O método SOAP é uma forma estruturada de registro utilizada em prontuários clínicos para organizar as informações do paciente de maneira clara, objetiva e padronizada. A sigla SOAP corresponde aos termos **S**ubjetivo, **O**bjetivo, **A**valiação e **P**lano. Na seção Subjetiva, são descritas as queixas, percepções e sentimentos relatados pelo paciente; na Objetiva, registram-se sinais clínicos observáveis, dados mensuráveis e resultados de exames físicos ou complementares. A Avaliação reúne a interpretação profissional dessas informações, incluindo hipóteses diagnósticas e justificativas clínicas; por fim, o Plano define as condutas a serem adotadas, como tratamentos, prescrições, exames adicionais ou orientações. Dessa forma, o SOAP contribui para a organização do cuidado, favorecendo a continuidade assistencial e a comunicação entre os profissionais de saúde.

Esses casos de uso foram elaborados com o objetivo de proporcionar uma compreensão mais aprofundada sobre o funcionamento do sistema, ilustrando as interações entre os usuários e o sistema em diferentes áreas de gerenciamento.

A descrição de cada caso de uso visa facilitar a visualização do fluxo de atividades e das tarefas que podem ser executadas no sistema, além de evidenciar a lógica envolvida em cada operação.

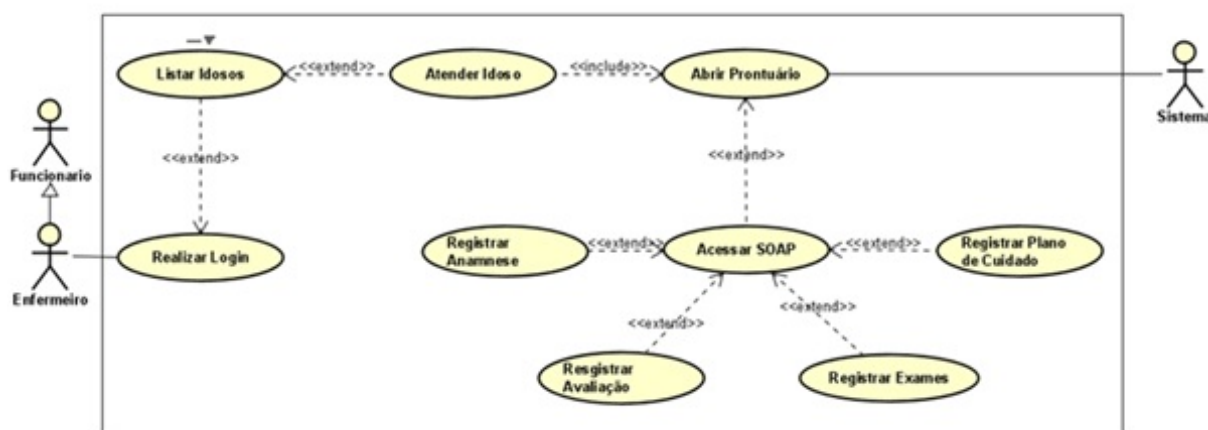
3.6.1 Caso de Uso Individual - Cadastro de SOAP

Os casos de uso apresentados nesta seção foram elaborados a partir do documento-base fornecido pelo usuário, descrevendo de forma detalhada o fluxo de interação entre o ator

Funcionário e o **Sistema** no contexto do prontuário eletrônico do idoso, abrangendo registros clínicos, avaliações e procedimentos.

A Figura 3.6 apresenta o diagrama de caso de uso que representa as principais interações entre os atores **Funcionário**, **Enfermeiro** e o **Sistema** no contexto das atividades relacionadas ao prontuário eletrônico do idoso. O diagrama evidencia o fluxo geral de ações necessárias para o atendimento, incluindo listagem e seleção do idoso, abertura do prontuário e acesso ao modelo SOAP, bem como os casos de uso estendidos envolvendo o registro de anamnese, avaliação, exames e plano de cuidado. Essa representação permite visualizar de forma clara e organizada a estrutura funcional do módulo de prontuário, destacando como cada ação se relaciona e se integra ao processo assistencial.

Figura 3.6 – Diagrama de Caso: Prontuário SOAP



Fonte: Elaborada pelos autores.

Cada caso de uso foi estruturado seguindo um padrão que inclui resumo, pré-condições, pós-condições, cenário principal e possíveis variações alternativas ou de exceção. No caso específico relacionado ao acesso ao modelo SOAP, destaca-se que sua finalidade é permitir ao funcionário visualizar informações clínicas organizadas nos componentes Subjetivo, Objetivo, Avaliação e Plano, fundamentais para assegurar a continuidade e a qualidade do atendimento.

No Quadro 3.54 tem-se o fluxo para o funcionário acessar o SOAP de um idoso assistido pela entidade.

Quadro 3.54 – UC: Acessar SOAP

Ator Principal	Funcionário
Atores Secundários	Sistema
Resumo	Descreve as etapas para o funcionário acessar as informações do prontuário SOAP de um idoso.
Pré-condições	Atender um idoso, abrir o prontuário e estar logado no sistema.
Pós-condições	–
Cenário Principal	Ações do Ator: 1. Clica em “Acessar SOAP”. Ações do Sistema: 2. Exibe os campos do SOAP (subjetivo, objetivo, avaliação e plano).
Restrições e Validações	O idoso deve estar cadastrado.
Cenário Alternativo	Não há.
Cenário de Exceção	Não há.

Fonte: Elaborada pelos autores.

No Quadro 3.55 trata-se do registro da anamnese, etapa fundamental na coleta de informações subjetivas sobre o estado de saúde do idoso.

Quadro 3.55 – UC: Registrar Anamnese

Ator Principal	Funcionário
Atores Secundários	Sistema
Resumo	Descreve as etapas para preenchimento da anamnese de um idoso.
Pré-condições	Atender um idoso, abrir o prontuário e estar logado.
Pós-condições	–
Cenário Principal	1. Funcionário acessa o SOAP. 2. Sistema exibe os campos SOAP. 3. Funcionário seleciona “Registrar Anamnese”. 4. Sistema mostra o formulário. 5. Funcionário preenche as informações. 7. Sistema valida e salva os dados.
Restrições e Validações	Idoso deve estar cadastrado.
Cenário Alternativo	Dados Inválidos: 1. Sistema avisa sobre dados inválidos. 2. Pedido é recusado.
Cenário de Exceção	Não há.

Fonte: Elaborada pelos autores.

Para o caso de uso Registrar Avaliação (Quadro 3.56) descreve-se a etapa de avaliação

clínica, contemplando análise subjetiva e objetiva, com inserção de códigos CIAP2 e CID-10.

Quadro 3.56 – UC: Registrar Avaliação

Ator Principal	Funcionário
Atores Secundários	Sistema
Resumo	Descreve as etapas de avaliação clínica baseada nos blocos Subjetivo e Objetivo.
Pré-condições	Atendimento ativo, prontuário aberto e login efetuado.
Pós-condições	–
Cenário Principal	1. Acessar SOAP. 2. Sistema exibe campos. 3. Funcionário informa CIAP2, CID-10 e observações. 4. Sistema valida. 5. Sistema salva.
Restrições e Validações	1. Idoso deve estar cadastrado. 2. Informar pelo menos um código (CIAP2 ou CID-10).
Cenário Alternativo	Não há.
Cenário de Exceção	Não há.

Fonte: Elaborada pelos autores.

No caso de uso Registrar Exames (Quadro 3.57) demonstra-se o registro de exames contempla campos estruturados para aferições clínicas rotineiras, como antropometria, sinais vitais e glicemia.

Quadro 3.57 – UC: Registrar Exames

Ator Principal	Funcionário
Atores Secundários	Sistema
Resumo	Descreve o processo de registro de exames realizados ou solicitados para o idoso, incluindo informações clínicas e anexação de resultados.
Pré-condições	Prontuário aberto, atendimento ativo e usuário logado.
Pós-condições	O exame é registrado e armazenado no prontuário eletrônico.
Cenário Principal	1. Acessar SOAP. 2. Sistema exibe os campos disponíveis. 3. Funcionário seleciona “Registrar Exames”. 4. Sistema apresenta o formulário de registro de exames. 5. Funcionário informa tipo de exame, data, observações e anexa arquivo (opcional). 6. Sistema valida as informações. 7. Sistema salva o registro no prontuário.
Restrições e Validações	1. O idoso deve estar cadastrado. 2. Informações obrigatórias devem ser preenchidas (tipo e data do exame).
Cenário Alternativo – Dados Inválidos	1. Sistema identifica dados inconsistentes ou campos obrigatórios vazios. 2. Sistema informa o erro ao usuário. 3. Funcionário revisa e corrige os dados.
Cenário de Exceção	Falha ao anexar o arquivo ou indisponibilidade temporária do sistema.

Fonte: Elaborada pelos autores.

No caso de uso Plano de Cuidado (Quadro 3.58) tem-se o eixo central do acompanhamento de saúde, reunindo prescrições, orientações, encaminhamentos e lembretes clínicos.

Quadro 3.58 – UC: Registrar Plano de Cuidado

Ator Principal	Funcionário
Atores Secundários	Sistema
Resumo	Descreve o processo de criação e atualização do plano de cuidado do idoso, incluindo metas, prescrições e condutas assistenciais.
Pré-condições	Prontuário aberto, atendimento ativo e usuário autenticado.
Pós-condições	O plano de cuidado é registrado e disponibilizado para consulta.
Cenário Principal	1. Acessar SOAP. 2. Sistema exibe os campos disponíveis. 3. Funcionário seleciona “Registrar Plano de Cuidado”. 4. Sistema apresenta o formulário de plano de cuidado. 5. Funcionário preenche diagnósticos, intervenções e metas assistenciais. 6. Sistema valida as informações. 7. Sistema salva o plano de cuidado no prontuário eletrônico.
Restrições e Validações	1. O idoso deve estar cadastrado. 2. O plano deve conter ao menos uma intervenção ou diagnóstico.
Cenário Alternativo – Dados Insuficientes	1. Sistema identifica ausência de dados essenciais. 2. Sistema notifica o usuário sobre o preenchimento obrigatório. 3. Funcionário ajusta o formulário.
Cenário de Exceção	Não há.

Fonte: Elaborada pelos autores.

3.7 Diagrama de Sequência

Os diagramas de sequência desempenham um papel essencial na modelagem de sistemas, pois representam de maneira clara a ordem cronológica das interações entre objetos, serviços, camadas e atores envolvidos na execução de um determinado comportamento.

Segundo **Sommerville2018ES10**, esse tipo de diagrama destaca o fluxo temporal das mensagens, revelando como os componentes colaboram para realizar uma operação. Da mesma forma, **Pressman2011ES7** reforça que diagramas de sequência são ferramentas fundamentais para compreender o comportamento dinâmico do software, permitindo visualizar a troca de mensagens e o encadeamento lógico das ações.

No contexto do sistema analisado, os diagramas de sequência foram desenvolvidos para representar o funcionamento completo das operações de manutenção de dados sobre *resident relatives* (parentes ou associados do residente). As representações incluem as operações de

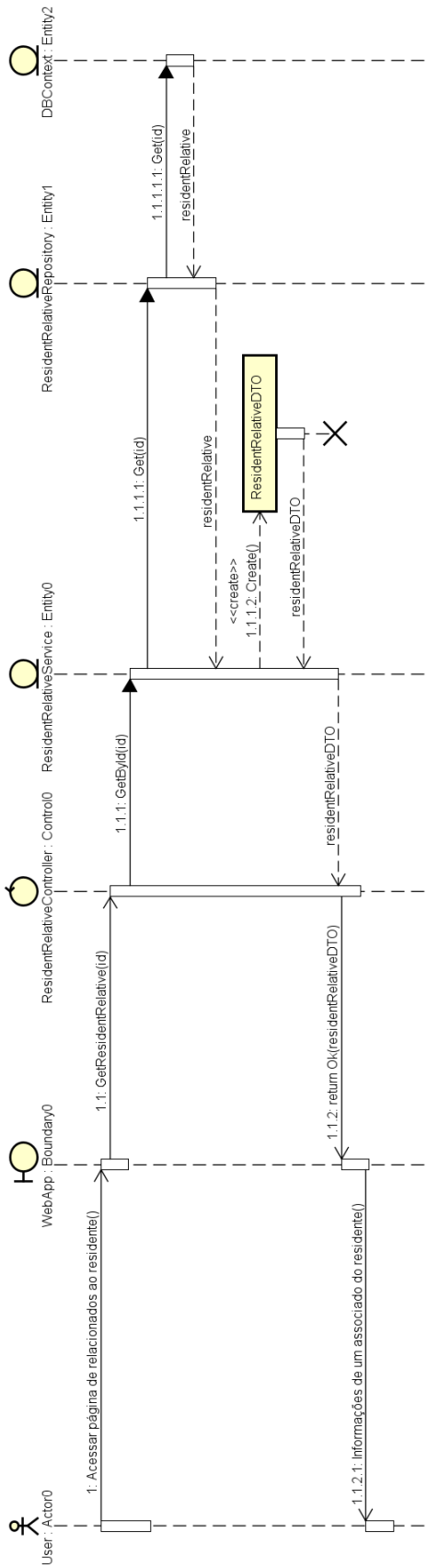
listar, **cadastrar**, **editar** e **remover**, conforme detalhadas no documento base fornecido pelo usuário:contentReference[oaicite:0]index=0. Cada diagrama descreve a colaboração entre a WebApp, os controladores, os serviços de domínio, os repositórios e o banco de dados, evidenciando a utilização do padrão DTO (*Data Transfer Object*), padrão amplamente recomendado por **Guedes2018** para garantir organização, modularidade e independência de camadas.

A seguir, apresenta-se a descrição detalhada de cada diagrama, acompanhada das Figuras 3.7, 3.8, 3.9 e 3.10, que ilustram, respectivamente, os processos de listagem, cadastro, edição e remoção de parentes de residentes.

3.7.1 Diagrama de Sequência: Listar Parente de Residente

O diagrama de sequência de listagem (Figura 3.7) descreve o fluxo necessário para recuperar as informações de um parente associado ao residente. O processo inicia-se com o usuário acessando a página correspondente na WebApp, que, por sua vez, envia uma requisição HTTP ao *controller*. Este repassa o processamento para a camada de serviço, que consulta o repositório e o banco de dados. Após recuperar a entidade, ocorre a criação de um DTO, que retorna até o usuário final por meio de uma resposta HTTP (200 OK). Esse fluxo reflete a arquitetura em camadas descrita por **Sommerville2018ES10**, reforçando a separação entre lógica de negócio, acesso a dados e apresentação.

Figura 3.7 – Diagrama de sequência para listagem de parente de residente.

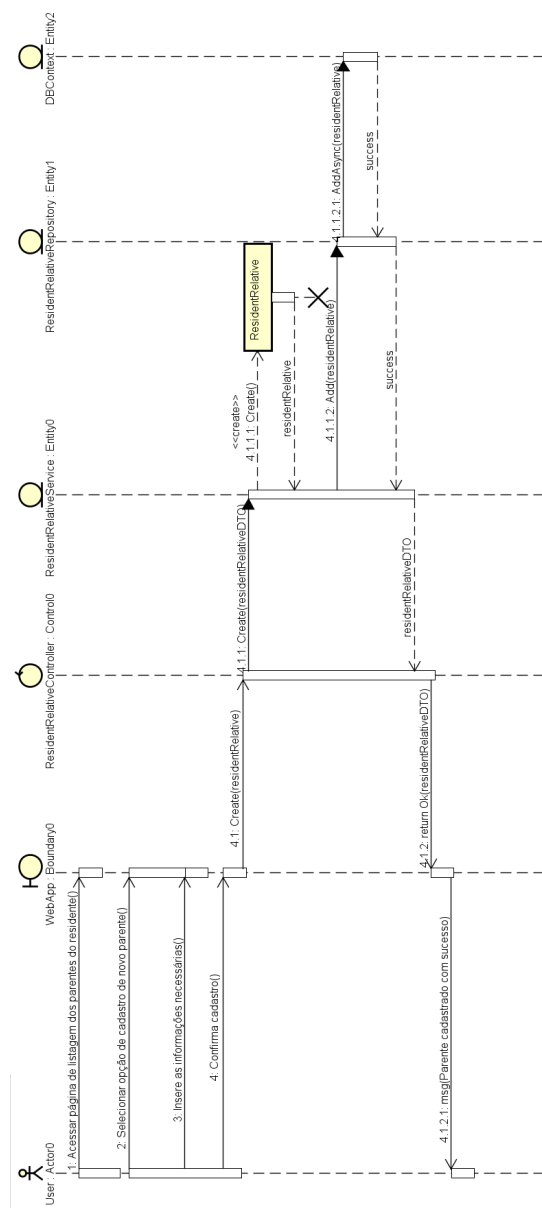


Fonte: Elaborada pelos autores.

3.7.2 Diagrama de Sequência: Cadastrar Parente de Residente

O cadastro de um novo parente de residente é representado na Figura 3.8. O fluxo inicia-se com a submissão de um formulário na WebApp, contendo os dados inseridos pelo usuário. A aplicação encaminha essas informações ao *controller*, que cria um DTO e o repassa para a camada de serviço. A entidade correspondente é instanciada, validada e persistida no banco de dados por meio do repositório. Após a inserção, os dados são retornados como um DTO atualizado, incluindo o novo identificador. Essa dinâmica representa fielmente a abordagem orientada a objetos recomendada por **Guedes2018**, na qual cada camada desempenha um papel específico e bem definido.

Figura 3.8 – Diagrama de sequência para cadastro de parente de residente.

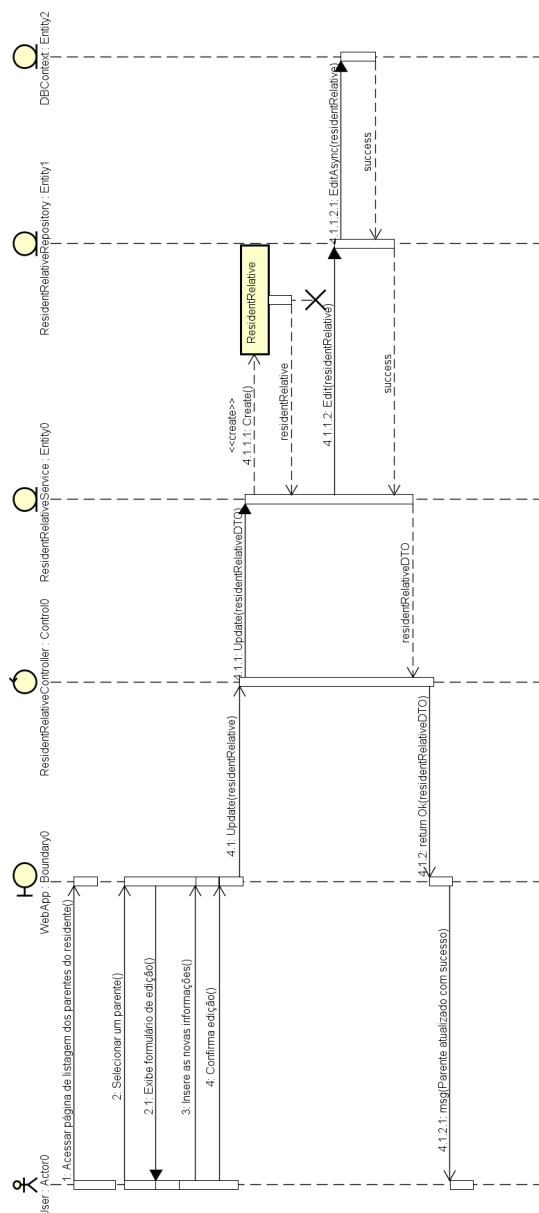


Fonte: Elaborada pelos autores.

3.7.3 Diagrama de Sequência: Editar Parente de Residente

A edição de dados de um parente de residente é apresentada na Figura 3.9. O processo inicia-se com o acesso à página de listagem, seguido da seleção do parente a ser alterado. O sistema exibe o formulário de edição com os dados atuais e, após a submissão das novas informações, o *controller* encaminha a requisição à camada de serviço, que atualiza a entidade no repositório e persiste as alterações no banco de dados. Ao final, um DTO atualizado é retornado à interface, permitindo ao usuário visualizar a confirmação do sucesso da operação. Assim como nos demais diagramas, observa-se o uso consistente do padrão DTO, o que, segundo **Pressman2011ES7**, contribui para a flexibilidade e manutenção do sistema.

Figura 3.9 – Diagrama de sequência para edição de parente de residente.

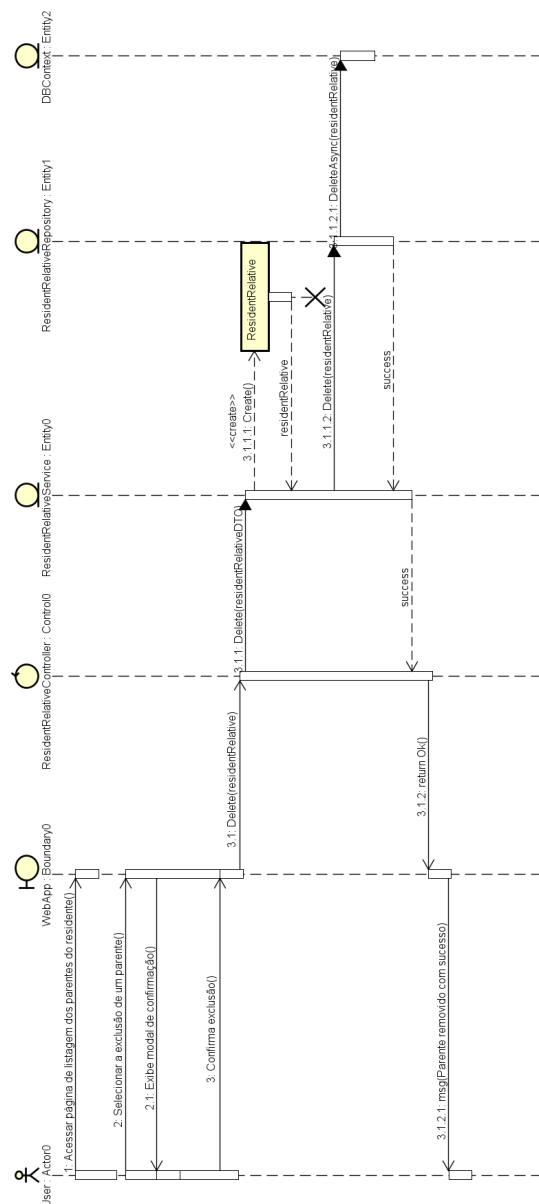


Fonte: Elaborada pelos autores.

3.7.4 Diagrama de Sequência: Remover Parente de Residente

A operação de remoção, ilustrada na Figura 3.10, inicia-se com uma etapa de confirmação por parte do usuário, correspondente a uma boa prática de segurança e prevenção de erros operacionais. Após a confirmação, a WebApp envia um comando de exclusão ao *controller*, que aciona a operação de deleção na camada de serviço e, em seguida, no repositório. O banco de dados executa a exclusão e retorna o sucesso da operação. Por fim, o usuário recebe uma mensagem de confirmação visual na interface. Segundo **Sommerville2018ES10**, a modelagem dessa interação detalhada é fundamental para garantir que o sistema atenda aos requisitos funcionais de forma previsível e organizada.

Figura 3.10 – Diagrama de sequência para remoção de parente de residente.



Fonte: Elaborada pelos autores.

Em conjunto, os diagramas analisados permitem uma visão clara do ciclo de vida completo

das operações sobre parentes de residentes, evidenciando o grau de acoplamento entre as camadas, o fluxo de dados e a ordem das mensagens. Essa perspectiva dinâmica complementa a visão estrutural oferecida pelos diagramas de classes, garantindo uma documentação completa, alinhada aos princípios de engenharia de software estabelecidos pela literatura.

3.8 Diagrama de Atividade

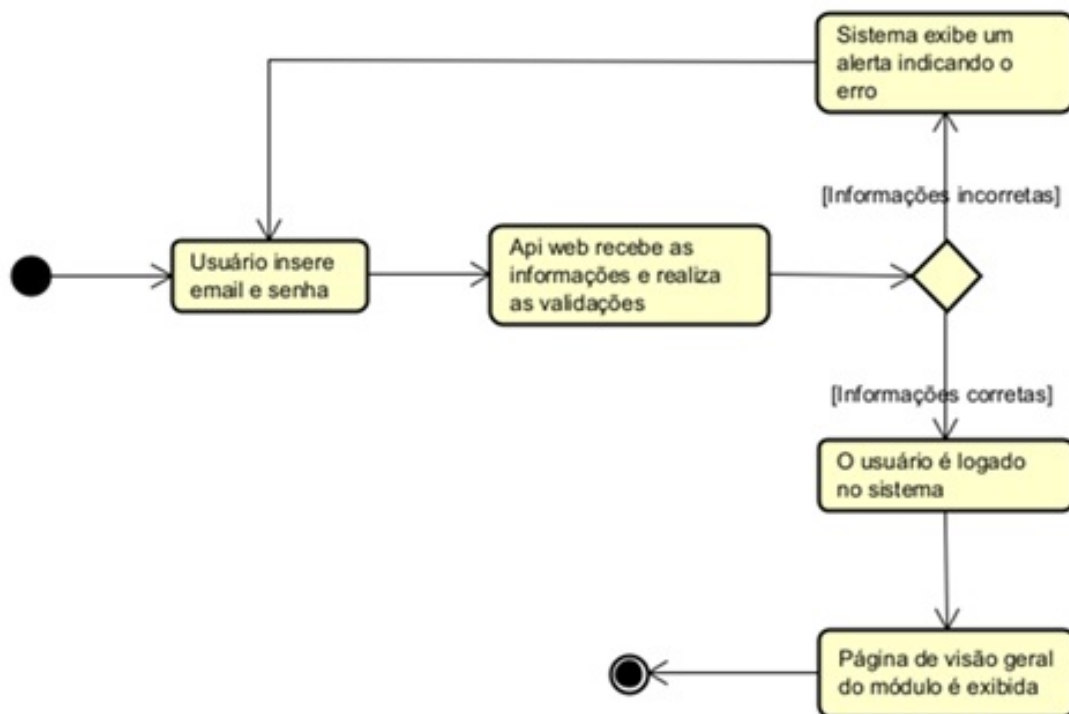
O diagrama de atividade, embora tenha sido inicialmente tratado como um caso especial do diagrama de máquina de estados, é atualmente reconhecido como um diagrama independente dentro da UML. Segundo **Guedes2018**, trata-se do diagrama com “mais ênfase em nível de algoritmo da UML e provavelmente um dos mais detalhistas”.

Esse tipo de diagrama é amplamente utilizado na modelagem de *atividades*, entendidas como métodos, algoritmos ou processos completos. Nesse sentido, **Guedes2018** esclarece que:

Uma atividade é composta por um conjunto de ações, ou seja, os passos necessários para que a atividade seja concluída.

A Figura 3.11 apresenta o diagrama de atividade referente ao fluxo de autenticação do usuário no sistema, ilustrando de forma clara e sequencial as ações realizadas desde o momento em que o usuário insere suas credenciais até a validação final e redirecionamento.

Figura 3.11 – Diagrama de atividade para o processo de autenticação do usuário.



Fonte: Elaborada pelos autores.

O diagrama (3.11) inicia-se com o nó inicial (*black node*), indicando o começo do fluxo. Em seguida, o usuário insere seu e-mail e senha, que são enviados à API web responsável por receber e validar essas informações.

Após a etapa de validação, o fluxo se divide em dois caminhos possíveis:

- **Informações incorretas:** o sistema exibe uma mensagem de erro informando que os dados inseridos são inválidos. O fluxo retorna à etapa de inserção das credenciais.
- **Informações corretas:** o usuário é autenticado com sucesso e direcionado à página de visão geral do módulo principal do sistema.

Por fim, o diagrama é encerrado pelo nó final, representando o término do processo de autenticação. Esse tipo de modelagem é essencial para evidenciar a lógica de controle do sistema, além de facilitar a comunicação entre desenvolvedores, analistas e demais stakeholders envolvidos no projeto.

4 Definição da Interface com o Usuário

O design de UX vai muito além da estética: trata-se de compreender contextos, tarefas e objetivos dos usuários e conectá-los a metas do negócio para produzir experiências úteis, utilizáveis e desejáveis. Isso envolve investigar o problema, definir requisitos baseados em evidências, estruturar informação e fluxos, prototipar soluções, validar com pessoas reais e iterar continuamente com foco em resultado—acessibilidade, eficiência, satisfação e impacto no negócio.

Unger2012 destacam que o(a) designer de UX atua como elo entre áreas multidisciplinares (produto, engenharia, conteúdo, dados e partes interessadas), orquestrando do descobrimento à avaliação: pesquisa com usuários, modelagem de personas e jornadas, construção de *wireframes*/protótipos, testes de usabilidade e comunicação de decisões. O foco permanece na experiência do usuário, mas sempre alinhada a objetivos claros e mensuráveis do produto.

4.1 Descrição de Cenário

A descrição de cenário é uma técnica fundamental no processo de design de interação, utilizada para representar situações reais de uso de um sistema ou produto digital. De acordo com **Rogers2013**, os cenários contribuem para a compreensão do contexto de uso, dos objetivos dos usuários e dos desafios que podem surgir durante a interação. Essas descrições assumem a forma de narrativas simples e acessíveis, que ilustram como um usuário específico realiza tarefas em determinado ambiente. Essa abordagem permite aos designers anteciparem as necessidades e comportamentos, contribuindo para que o produto final esteja alinhado às expectativas e características do público-alvo.

Para ilustrar o comportamento do usuário, foram desenvolvidos dois cenários. Esses cenários descrevem os objetivos dos usuários e as possíveis formas de interação com o sistema, funcionando como narrativas que representam situações reais de uso. Por meio dessas descrições, é possível antecipar como os usuários poderão se relacionar com a interface, contribuindo para um projeto mais alinhado às suas necessidades e expectativas.

Para descrever o processo de cadastro de uma nova religião, foi elaborado o primeiro cenário (Quadro 4.1), que simula como o funcionário pode interagir com o sistema, apresentando uma descrição das etapas envolvidas na inserção das informações de religião.

Quadro 4.1 – Cenário — Registro de Religião por um Funcionário do Lar dos Velhinhos

No contexto de um lar de idosos, uma funcionária administrativa acessa o sistema institucional para cadastrar uma nova religião de um idoso recém-chegado. A funcionária, autenticada no sistema, acessa a página de religiões e clica em “Nova Religião”. Em seguida, preenche os dados da nova religião, como o nome, e ao concluir o processo clica em “Salvar”. Essa ação garante que a crença do idoso esteja registrada no sistema, permitindo que suas práticas espirituais sejam respeitadas e consideradas na rotina da instituição.

Fonte: Elaborado pelos autores.

O cenário de consulta médica (Quadro 4.2) apresenta a situação em que um médico utiliza o sistema durante uma consulta clínica com um residente, descrevendo como o sistema permite o acesso ao prontuário digital do paciente, incluindo informações sobre alergias previamente registradas. Com base nesses dados, o médico ajusta a prescrição de medicamentos, assegurando um cuidado seguro, personalizado e compatível com as condições de saúde do residente.

Quadro 4.2 – Cenário — Registro e Consulta de Alergias por Médico em Exame

Durante um exame clínico de um residente, o médico, autenticado no sistema, procura o residente pelo nome e acessa o prontuário digital para consultar as alergias previamente registradas. Ao identificar que o residente possui alergia a frutos do mar, o profissional registra cuidadosamente essa informação, preenchendo os campos: tipo da alergia, descrição, data de detecção e, se aplicável, data de liberação, garantindo que toda a instituição esteja ciente da restrição. Em seguida, com base nesses dados, ajusta a prescrição de medicamentos, assegurando um cuidado seguro, individualizado e compatível com as condições de saúde do paciente.

Fonte: Elaborado pelos autores.

4.2 Descrição de Personas

No desenvolvimento centrado no usuário, *personas* funcionam como arquétipos que sintetizam padrões de comportamento, objetivos, dores e expectativas do público-alvo. Elas tornam explícito “quem usa o sistema, por quê e em quais condições”, orientando priorização de requisitos, decisões de navegação e o vocabulário da interface (**Unger2012**). Conforme a boa prática, não são perfis fictícios arbitrários, mas representações baseadas em evidências (observação, entrevistas e análise de processos), e devem ser revisadas à medida que novas descobertas surgirem (**Unger2012**).

No contexto de um lar de idosos, mapeamos os principais atores que interagem com o sistema institucional e consolidamos duas *personas* operacionais. O processo incluiu: (i) levantamento dos fluxos de trabalho e pontos de contato; (ii) coleta de dados sobre tarefas frequentes, frustrações e critérios de sucesso; (iii) síntese em arquétipos com objetivos claros; e (iv) validação com as áreas envolvidas. Para cada *persona* registramos dados demográficos sintéticos, metas, responsabilidades, tarefas prioritárias e implicações de design (por exemplo, necessidades de acesso

rápido, alertas, permissões e rastreabilidade), de modo a alinhar protótipos e funcionalidades às demandas reais (Unger2012).

Figura 4.1 – Persona 1 — Enfermeira-chefe (Carla Regina de Souza)

Persona 1



Carla Regina de Souza
Cargo: Enfermeira Chefe

Idade: 38 anos
Cidade: Jales
Estado: São Paulo
Estado Civil: Casada

Descrição: Carla é uma enfermeira experiente, trabalhando no Lar dos Velhinhos há mais de 10 anos. Ela é responsável por coordenar a equipe de enfermagem, garantir o bem-estar dos residentes e monitorar seus prontuários de saúde. Sua principal preocupação é a segurança e o cuidado individualizado de cada idoso.

Necessidades e Responsabilidades:

- Acesso rápido ao prontuário digital dos residentes para checar informações de saúde.
- Capacidade de registrar novos medicamentos e tratamentos.
- Responsável por manter a equipe informada sobre alergias e condições de saúde específicas.
- Colabora com os médicos para ajustar planos de cuidado e medicação.
- Utiliza o sistema para agendar consultas e exames de rotina.

Fonte: Elaborada pelos autores.

A Figura 4.1 representa “Carla Regina de Souza”, enfermeira-chefe com experiência na coordenação da equipe e no acompanhamento clínico dos residentes. Suas necessidades traduzem requisitos concretos do sistema: acesso ágil ao prontuário digital, registro rápido e seguro de medicamentos e tratamentos, visibilidade de alergias e condições críticas, agenda de consultas e comunicação com a equipe multiprofissional. Implicações de design incluem atalhos para tarefas recorrentes, alertas não intrusivos (por exemplo, alergias), histórico auditável e formulários otimizados para reduzir retrabalho durante o plantão (Unger2012).

Figura 4.2 – Persona 2 — Recepcionista (Fernando Costa)

Persona 2



Fernando Costa
Cargo: Recepcionista

Idade: 28 anos
Cidade: Jales
Estado: São Paulo
Estado Civil: Solteiro

Descrição: Fernando é o primeiro ponto de contato no Lar dos Velhinhos. Ele é responsável por gerenciar a entrada e saída de visitantes, atender chamadas telefônicas e auxiliar nos registros de novos residentes. Sua principal meta é garantir que a comunicação seja eficiente e que as informações de contato estejam sempre atualizadas.

Necessidades e Responsabilidades:

- Acesso rápido ao sistema para cadastrar visitantes.
- Capacidade de consultar informações de contato de familiares.
- Responsável por agendar visitas e eventos no calendário da instituição.
- Colabora com a equipe de enfermagem e administração para comunicar recados urgentes.
- Utiliza o sistema para registrar entrada e saída de correspondências e encomendas.

Fonte: Elaborada pelos autores.

A Figura 4.2 apresenta “Fernando Costa”, recepcionista responsável pela entrada de visitantes, comunicação com familiares e organização de agendas. Suas tarefas demandam cadastro e consulta ágeis, calendário institucional claro, registros de entrada/saída e mecanismos de comunicação confiáveis. Daí decorrem requisitos como busca por nome/CPF, formulários enxutos, confirmação de identidade e logs de eventos, além de mensagens de feedback que evitem ambiguidades no atendimento (Unger2012).

Em conjunto, as *personas* estruturam critérios de aceitação e histórias de usuário (“como Carla/Fernando, quero ... para ...”) e servem de referência para avaliar protótipos e decisões de priorização ao longo de todo o ciclo do projeto (Unger2012).

4.3 Esboços de tela (wireframes)

Wireframes são esboços de baixa fidelidade usados nas etapas iniciais de UX para tomada de decisão sobre **arquitetura da informação**, **hierarquia de conteúdo** e **fluxos de navegação** antes de investir em layout final e código. Ao abstrair cores, tipografia e imagens, eles mantêm o foco no que é essencial: quais tarefas o usuário precisa executar, em que ordem, com quais componentes (campos, botões, listas, filtros) e que *feedback* a interface deve oferecer (estados vazio/erro/sucesso).

Normalmente apresentados em tons de cinza e com *placeholders*, os wireframes servem

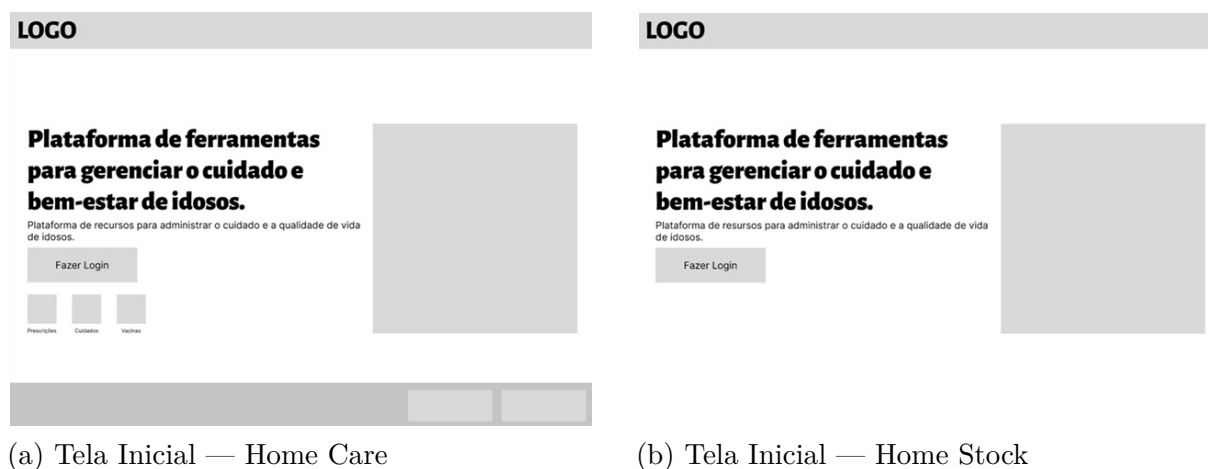
como artefato de alinhamento entre equipes de produto, design e tecnologia. Eles viabilizam discussões rápidas sobre rótulos, prioridades e regras de negócio, permitem testes exploratórios com usuários e ajudam a estimar esforço técnico (*ex.*: dependências de dados, validações, paginação, permissões). Também facilitam a definição de padrões—grade, espaçamentos, tipos de campo, posição de ações—que mais tarde serão consolidados no *design system*, assegurando consistência entre telas e pontos de quebra responsivos.

Na Figura 4.5b, o wireframe da tela de cadastro/gestão exemplifica esse padrão funcional: trilha de navegação (*breadcrumbs*) para contexto, busca por termo, ação primária “+ Adicionar” e uma listagem tabular com seleção por linha e ações de *editar/excluir*. Esse arranjo evidencia o fluxo típico de manutenção (localizar → incluir/alterar → confirmar), reduz a carga cognitiva e estabelece um modelo reutilizável para as demais telas de registro do sistema, promovendo previsibilidade e menor curva de aprendizado.

As Figuras 4.3(a) e 4.3(b) mostram *wireframes* de baixa fidelidade para as telas iniciais dos módulos *Care* e *Stock*. Nesses esboços priorizam-se mensagem, hierarquia e *call to action* (CTA), sem preocupação com o acabamento visual, a fim de validar rapidamente a clareza da proposta e o encaminhamento ao login.

No (a) **Home Care**, o layout em duas colunas destaca, à esquerda, um **título de alto impacto**, **textinho de apoio** e o CTA **Fazer Login**; abaixo, uma faixa de **atalhos** (prescrições, cuidados e vacinas) antecipa rotas frequentes do módulo. À direita, reserva-se área para *hero* ilustrativo. Já o (b) **Home Stock** mantém a mesma estrutura de *hero* (título, apoio e CTA), porém *sem* os atalhos inferiores, simplificando a entrada e reduzindo a carga cognitiva para o contexto de estoque. Em ambos, espera-se responsividade, contraste suficiente, ordem de leitura previsível e texto alternativo adequado para a imagem.

Figura 4.3 – Wireframes — (a) Tela Inicial para Módulo de Cuidado (Home Care) e (b) Tela Inicial para Módulo de Estoque (Home Stock)



Fonte: Elaborada pelos autores.

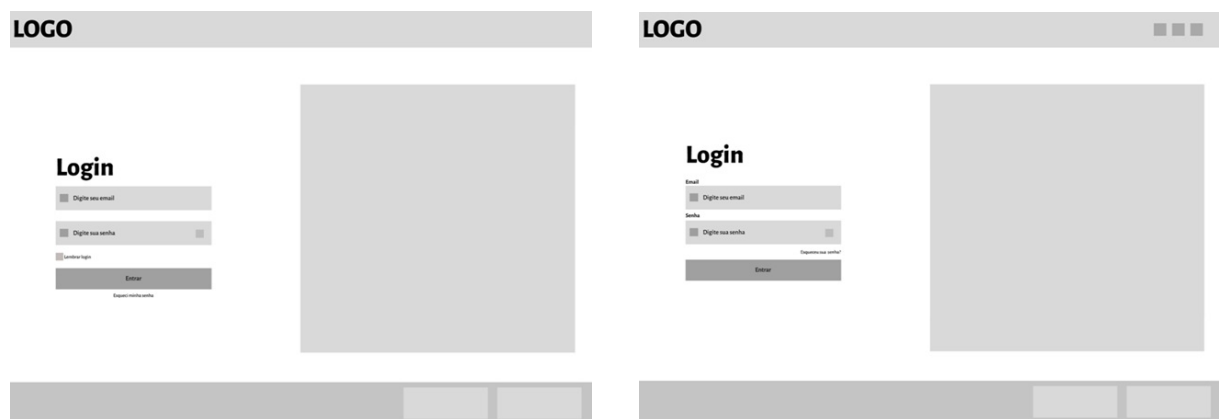
As Figuras 4.4(a) e 4.4(b) apresentam *wireframes* de baixa fidelidade para o fluxo de autenticação dos módulos *Care* e *Stock*. Por se tratarem de esboços, privilegiam-se hierarquia e

fluxo—não o acabamento visual—para discutir rótulos, ordem dos campos, posicionamento de ações e mensagens antes da implementação.

Em ambos os casos o formulário **Login** aparece à esquerda com os campos *Email* e *Senha* (placeholders/ícones) e a ação primária **Entrar**; à direita há uma área reservada para imagem institucional. No (a) **Care**, incluem-se o *checkbox* *Lembrar login* (persistência controlada de credenciais) e o link *Esqueci minha senha* sob o botão, reforçando conveniência no acesso domiciliar. No (b) **Stock**, a omissão do *Lembrar login* prioriza segurança em ambientes de estoque, mantendo o link de recuperação próximo ao campo de senha para acesso rápido.

Comportamentos esperados: validação de formato do e-mail, ocultação de caracteres da senha, estados desabilitado/ativo do botão **Entrar**, mensagens de erro junto aos campos e suporte a navegação por teclado e leitores de tela.

Figura 4.4 – Wireframes — (a) Tela de Login para o Módulo de Cuidado (Care) e (b) Tela de Login para o Módulo de Estoque (Stock)



(a) Tela de Login — Care

(b) Tela de Login — Stock

Fonte: Elaborada pelos autores.

As Figuras 4.5(a) e 4.5(b) apresentam *wireframes* de baixa fidelidade que documentam os padrões de cadastro do projeto. Esses esboços priorizam fluxo e hierarquia de informação (e não o acabamento visual), permitindo discutir campos obrigatórios, ações primárias e navegação antes da implementação.

O **wireframe (a)** ilustra o *cadastro de produto*: trilha de navegação (*Visão Geral* → *Cadastros* → *Produto* → *Cadastro Produto*), campo de busca global e formulário em duas colunas com metadados essenciais (nome genérico, descrição, tipo, grupo, unidade de medida), opções operacionais (controle de validade, estoque atual/mínimo) e a ação primária **Cadastrar produto**. Já o **wireframe (b)** representa a *gestão de relações*: busca por nome, botão **+ Adicionar**, e listagem tabular com seleção por linha e ações de *editar/excluir*. Em conjunto, as telas evidenciam o padrão do sistema para criação e manutenção de registros.

Figura 4.5 – Wireframes — (a) Tela de Cadastro de Instituição e (b) Tela de Gestão de Religião

The image contains two wireframe screenshots of a web application. Screenshot (a) is titled 'Cadastro de Instituição' and shows a form for adding a product. It includes a search bar at the top, a sidebar with a 'Produto' menu, and a main form area with fields for 'Nome genérico', 'Descrição', 'Tipo', 'Origem', 'Unidade de medida', 'Estoque atual', and 'Estoque mínimo'. Screenshot (b) is titled 'Gestão de Religião' and shows a table with columns for 'Nome da Religião', 'Descrição', 'Origem', and 'Unidade de medida'. It also includes a search bar and a 'Adicionar Produto' button.

(a) Cadastro de Instituição

(b) Gestão de Religião

Fonte: Elaborada pelos autores.

4.4 Protótipos de tela

Protótipos de tela são representações visuais (estáticas ou interativas) que simulam a interface e o comportamento de um sistema, permitindo explorar ideias, validar hipóteses de uso e antecipar ajustes antes do investimento em desenvolvimento. **Rosa2025** classifica-os quanto à fidelidade em baixa, média e alta: os de *baixa* apoiam a ideação rápida e o alinhamento conceitual; os de *média* permitem refinar fluxos, rótulos e hierarquia de informação; e os de *alta* aproximam-se do produto final, servindo à validação de microinterações, estados e conteúdo. Já **Garrett2011** enquadra a prototipagem nos planos de *estrutura* e *superfície* da experiência do usuário, não apenas como representação estética, mas como instrumento estratégico para avaliar a disposição de elementos, os fluxos de navegação e a usabilidade de ponta a ponta.

Como prática, a prototipagem deve ser iterativa: hipóteses são materializadas em telas, testadas com usuários e partes interessadas, e os aprendizados retornam ao modelo em ciclos sucessivos de melhoria. Esse processo reduz riscos, acelera decisões de design, favorece o alinhamento entre equipes e produz artefatos que comunicam requisitos de forma mais clara do que especificações exclusivamente textuais (**Garrett2011**; **Rosa2025**).

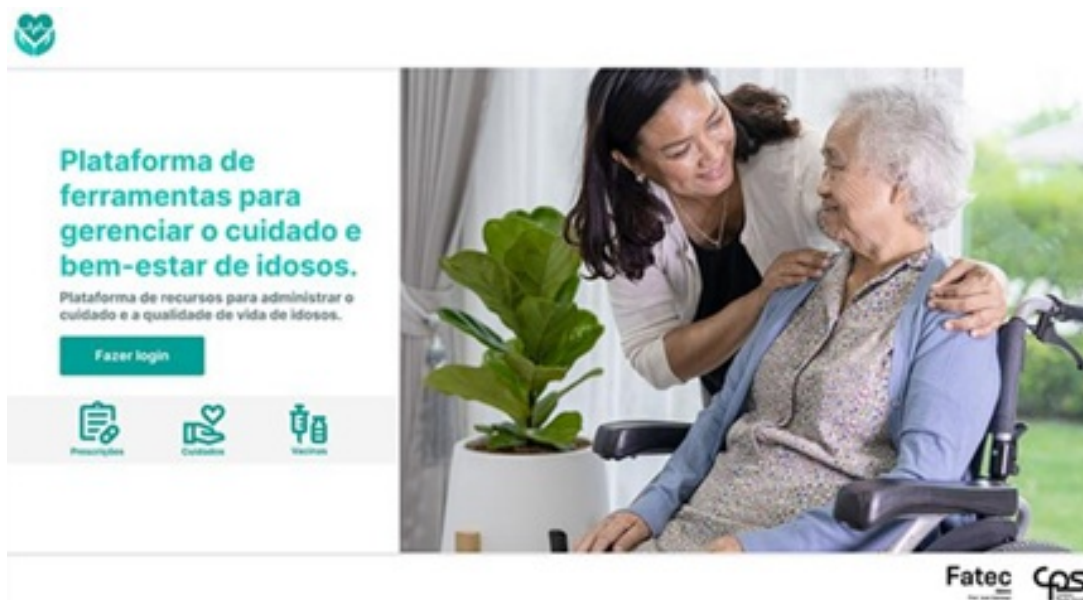
No projeto *SeniorCareManager*, utilizou-se o Figma para produzir protótipos de *alta fidelidade* com interações navegáveis, estados de componentes e variações de telas responsivas. O uso de bibliotecas de componentes e estilos consolidou padrões visuais e de interação, facilitando testes de usabilidade e o *handoff* para desenvolvimento, com insumos precisos sobre hierarquia, fluxos críticos e microcomportamentos (**Rosa2025**; **Garrett2011**).

A Figura 4.6 apresenta a tela de abertura (*hero*) do *SeniorCareManager*. O layout em duas colunas prioriza a comunicação da proposta de valor e a ação principal: à esquerda, um **título de alto destaque** e um **textinho de apoio** introduzem o objetivo de gerenciar cuidado e

bem-estar de idosos, seguido do botão primário **Fazer login**, que conduz ao fluxo de autenticação. Abaixo, uma faixa de **atalhos com ícones** oferece acesso rápido às áreas mais usadas do sistema (por exemplo, prontuários, cuidados, ocorrências e vacinas), reforçando a orientação a tarefas. À direita, uma **imagem contextual** transmite a ideia de acolhimento e cuidado.

A composição privilegia hierarquia tipográfica, espaçamento generoso e uma única chamada para ação, reduzindo a carga cognitiva na primeira interação. A imagem ilustrativa pode ser adaptada ao tema (alto contraste ou não) e, quando marcada como decorativa, utiliza texto alternativo vazio para não interferir na navegação por leitores de tela. O resultado é uma porta de entrada simples, responsiva e acessível, que direciona o usuário ao próximo passo do uso do sistema.

Figura 4.6 – SeniorCareManager — Tela de Abertura

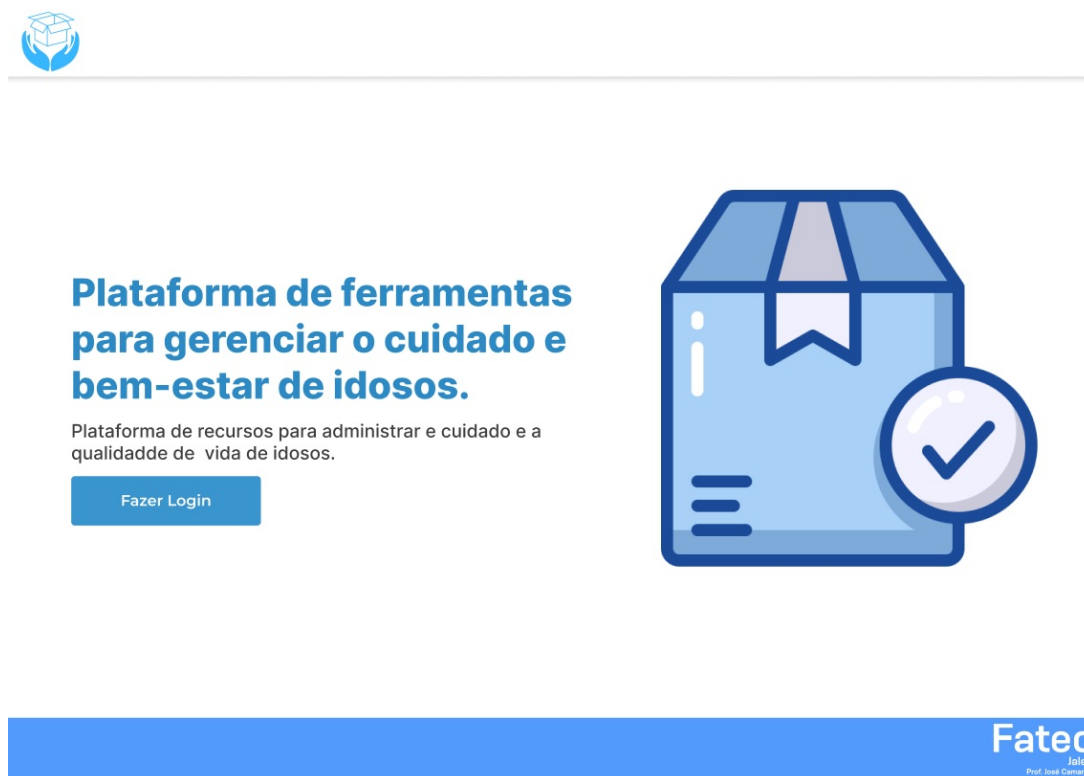


Fonte: Elaborado pelos autores.

A Figura 4.7 apresenta a tela de abertura (*landing/hero*) do *SeniorStockManager*. O layout em duas colunas prioriza a mensagem central e a ação principal: à esquerda, um **título de alto contraste** comunica a proposta de valor da plataforma, seguido de um **textinho de apoio** e do **botão primário “Fazer Login”**, que conduz ao fluxo de autenticação. À direita, uma **ilustração temática** relacionada a estoque e conferência reforça o contexto do módulo.

A composição privilegia hierarquia tipográfica, espaçamento generoso e foco em uma única chamada para ação, reduzindo a carga cognitiva e facilitando a decisão do usuário. A ilustração é carregada dinamicamente e se adapta ao tema selecionado, preservando legibilidade e contraste; quando marcada como decorativa, utiliza texto alternativo vazio para não interferir na navegação por leitores de tela. O resultado é uma tela de boas-vindas simples, orientada a objetivos e alinhada a boas práticas de usabilidade e acessibilidade.

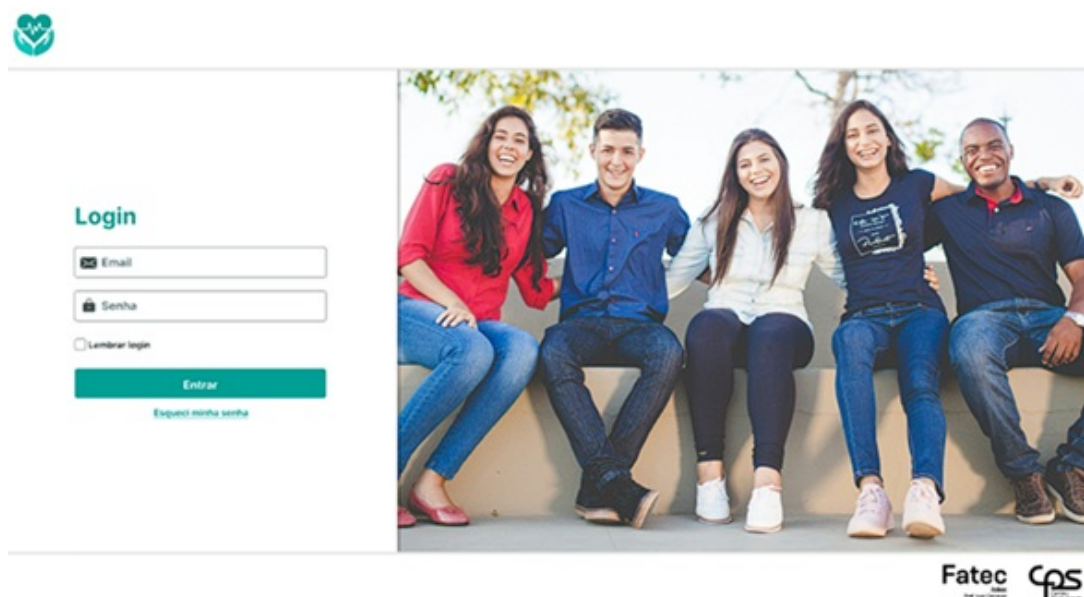
Figura 4.7 – SeniorStockManager — Tela de Abertura



Fonte: Elaborado pelos autores.

A Figura 4.8 apresenta a tela de autenticação do *SeniorCareManager*. O layout em duas colunas destaca, à esquerda, o formulário **Login** com campos *Email* e *Senha* (com *placeholders* e ícones), a opção *Lembrar login*, o botão primário **Entrar** e o link *Esqueci minha senha*. À direita, uma imagem temática reforça a proposta de cuidado e acolhimento do módulo. A organização privilegia hierarquia visual e foco na tarefa: rotulagem acima dos campos, áreas de ação bem delimitadas e espaçamento regular. Comportamentos esperados incluem validação do formato de e-mail, ocultação dos caracteres de senha, mensagens de erro junto aos campos, persistência controlada das credenciais quando o *checkbox* é ativado, além de suporte a navegação por teclado e leitura por tecnologias assistivas.

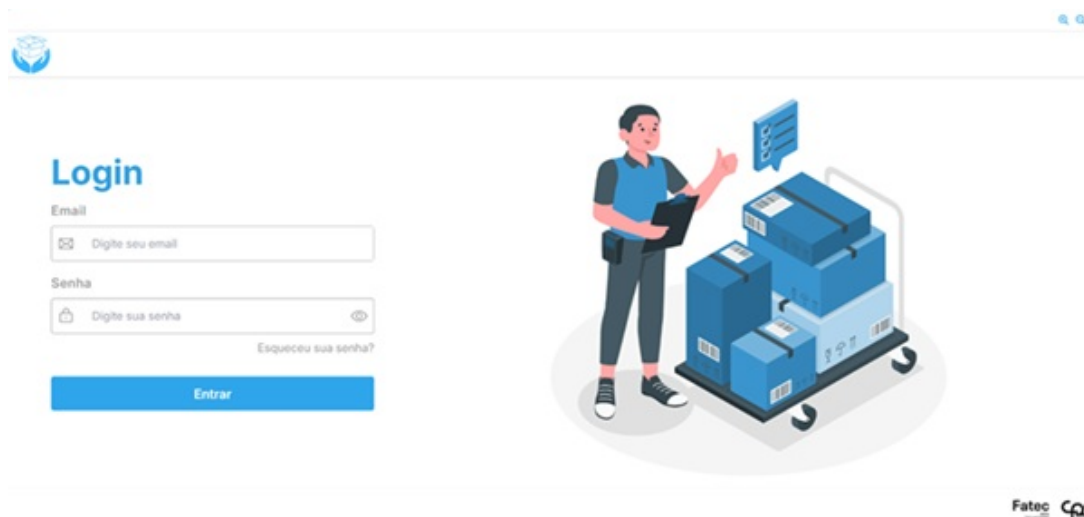
Figura 4.8 – SeniorCareManager — Tela de Login



Fonte: Elaborado pelos autores.

A Figura 4.9 apresenta a tela de autenticação do *SeniorStockManager*. À esquerda, o formulário **Login** dispõe dos campos *Email* e *Senha* com *placeholders* (“Digite seu email”, “Digite sua senha”), ícones de apoio nos *inputs*, link *Esqueceu sua senha?* e o botão primário **Entrar**. À direita, uma ilustração temática de caixas/estoque reforça o contexto do módulo. O layout em duas colunas é responsivo e privilegia hierarquia visual (título destacado, rótulos acima dos campos e áreas de ação bem delimitadas), promovendo leitura rápida e clara. O fluxo principal consiste em informar as credenciais e acionar **Entrar**; regras usuais incluem validação de formato para e-mail, ocultação de caracteres no campo de senha e exibição de mensagens de erro junto aos campos quando necessário, além do atalho para recuperação de credenciais via link.

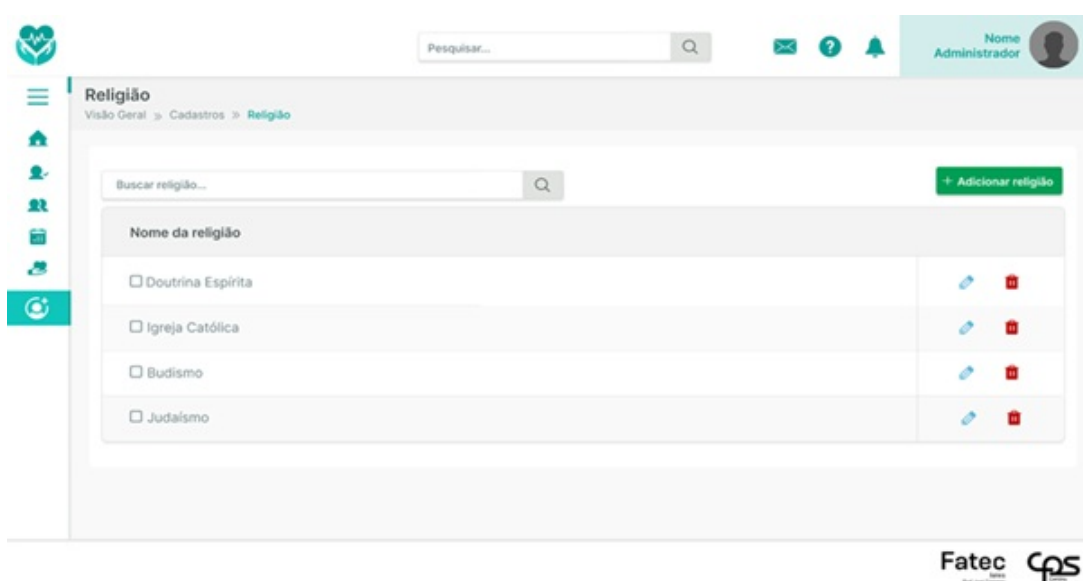
Figura 4.9 – SeniorStockManager — Tela de Login



Fonte: Elaborado pelos autores.

A Figura 4.10 apresenta a tela de *gestão de religiões* do *SeniorCareManager*. A interface organiza o fluxo de manutenção em três áreas claras: (i) **contexto e navegação**, com trilha “Visão Geral → Cadastros → Religião” e campo de busca por nome; (ii) **ação primária** “+ Adicionar religião” para inclusão de novos registros; e (iii) **listagem tabular** “Nome da religião”, com seleção por linha e ações contextuais de *editar* (ícone de lápis) e *excluir* (ícone de lixeira). Esse arranjo permite localizar rapidamente um registro, criar novos itens e manter o cadastro atualizado por meio de operações pontuais sobre cada entrada, preservando a consistência das informações institucionais.

Figura 4.10 – SeniorCareManager — Gestão de Religiões (listagem, busca e manutenção)



Fonte: Elaborado pelos autores.

A Figura 4.11 apresenta a tela de manutenção de produtos do *SeniorStockManager*. A área superior fornece **contexto e navegação**, com a trilha “Visão Geral → Cadastros → Produto → Cadastro Produto” e um campo de *busca* global. O formulário **Produto** é organizado em duas colunas e reúne os principais metadados do item: *Nome genérico* e *Descrição*; listas suspensas para *Tipo* e *Grupo*, acompanhadas de atalhos “Tipo não encontrado” e “Grupo não encontrado” para inclusão rápida de classificações ausentes; *Unidade de medida* (com link “Unidade de medida não encontrada”); opção *Possui controle de validade* (habilita o acompanhamento por data de vencimento); *Preço unitário*; *Estoque atual* e *Estoque mínimo*, com o seletor *Possui estoque mínimo* para ativar a regra de reposição. O fluxo principal consiste em informar os atributos do produto, selecionar suas classificações e unidade, definir regras de validade e de estoque e, por fim, confirmar a inclusão por meio do botão **Cadastrar produto**. Essa organização favorece a consistência cadastral e a manutenção contínua do catálogo, reduzindo retrabalho e erros operacionais.

Figura 4.11 – SeniorStockManager — Tela de Manutenção de Produtos

The screenshot displays the 'Produto' (Product) maintenance interface in the SeniorStockManager application. The layout includes a top header with a search bar and a user profile section labeled 'Nome Administrador'. A left sidebar contains navigation icons for home, shopping cart, and other functions. The main content area is titled 'Produto' and shows a breadcrumb trail: 'Visão Geral >> Cadastros >> Produto >> Cadastro Produto'. The form itself is divided into several sections: 'Nome genérico' and 'Descrição' (text inputs), 'Tipo' and 'Grupo' (dropdown menus), 'Unidade de medida' (dropdown menu), 'Preço unitário' (text input), 'Estoque atual' and 'Estoque mínimo' (text inputs), and checkboxes for 'Possui controle de validade' and 'Possui estoque mínimo'. Below the 'Unidade de medida' dropdown, there is a message 'Unidade de medida não encontrada'. Similarly, below the 'Grupo' dropdown, there is a message 'Grupo não encontrado'. At the bottom right of the form, there is a blue button labeled 'Cadastrar produto'. The footer of the page features the logos for 'Fatec' and 'CPS'.

Fonte: Elaborado pelos autores.

4.5 Acessibilidade e Inclusão de Pessoas com Deficiência

A inclusão de pessoas com deficiência (PCD) constitui um tema de grande relevância para a sociedade contemporânea. De acordo com a **OMS2011**, aproximadamente 1 bilhão de pessoas em todo o mundo vive com algum tipo de deficiência, o que equivale a cerca de uma em cada sete pessoas. A Convenção Internacional sobre os Direitos das Pessoas com Deficiência, assinada em Nova Iorque em 2007 e ratificada no Brasil pelo Decreto nº 6.949, de 25 de agosto de 2009 (**Brasil2009**), define como pessoas com deficiência aquelas que apresentam impedimentos de longa duração, sejam eles de ordem física, mental, intelectual ou sensorial, os quais, em interação com barreiras diversas, podem restringir sua plena participação na sociedade.

Para combater o preconceito e assegurar a igualdade de oportunidades, torna-se indispensável a adoção de medidas que envolvem a adaptação de espaços físicos, a adequação da linguagem, o uso de tecnologias inclusivas e a promoção da inserção no mercado de trabalho. Tais ações são fundamentais para garantir que as PCD possam viver de forma independente e participar plenamente da vida social.

A promoção da acessibilidade implica assegurar condições de acesso igualitário ao meio físico, ao transporte, à informação e à comunicação, incluindo os sistemas e tecnologias digitais, bem como a outros serviços e instalações disponíveis ao público. Apesar dos direitos assegurados por documentos internacionais, como a Declaração Universal dos Direitos Humanos, e por

recomendações da própria OMS, ainda persistem barreiras significativas que comprometem a efetivação desses direitos. O engajamento coletivo da sociedade é, portanto, essencial para a eliminação de obstáculos e formas de discriminação, configurando-se como um dever social e ético em prol da construção de um mundo mais justo e inclusivo.

No âmbito digital, a acessibilidade refere-se à eliminação de barreiras na web, garantindo que páginas e portais sejam projetados de forma que todas as pessoas possam perceber, compreender, navegar e interagir de maneira eficaz com os conteúdos e serviços oferecidos. O objetivo é democratizar o acesso e assegurar que todos, independentemente de suas condições físico-motoras, perceptivas, culturais ou sociais, consigam usufruir plenamente dos recursos digitais.

As barreiras digitais representam um desafio significativo. Segundo dados do **IBGE2010**, cerca de 45 milhões de brasileiros possuem algum tipo de deficiência, representando 23,9% da população nacional. Esse cenário evidencia a urgência de um ambiente digital inclusivo. A implementação de práticas de acessibilidade digital não apenas democratiza o acesso, mas também favorece a inclusão social, possibilitando que pessoas com deficiência utilizem a internet para estudar, trabalhar, realizar compras e compartilhar experiências, sem a necessidade de deslocamento físico.

Além disso, um site acessível tende a apresentar maior compatibilidade com diferentes dispositivos e aplicativos, além de ser melhor indexado por mecanismos de busca. Assim, os benefícios da acessibilidade digital estendem-se não apenas às pessoas com deficiência, mas também a idosos e a usuários com menor familiaridade com ferramentas digitais ou que dependem de dispositivos móveis. Dessa forma, a acessibilidade digital configura-se como um princípio de justiça social, cuja finalidade é garantir que todos possam participar plenamente da sociedade moderna.

4.5.1 Práticas de Acessibilidade Implementadas

Após um estudo detalhado das práticas de acessibilidade recomendadas para sistemas web, foram aplicadas diretrizes que asseguram uma experiência inclusiva para todos os usuários. Essas práticas estão fundamentadas nas recomendações internacionais das *Web Content Accessibility Guidelines* (WCAG 2.1) (**WCAG2018**), bem como nas normativas brasileiras, representadas pelo Decreto nº 5.296/2004 (**Brasil2004**) e pela Lei Brasileira de Inclusão da Pessoa com Deficiência (Lei nº 13.146/2015) (**Brasil2015**). Tais medidas visam aprimorar a navegação, a usabilidade e a compreensão do conteúdo, garantindo equidade de acesso a pessoas com diferentes tipos de deficiência.

Entre as práticas adotadas, destaca-se a inclusão de textos alternativos em todas as imagens do sistema, permitindo que leitores de tela transmitam o conteúdo visual a usuários com deficiência visual. Esse recurso assegura que os elementos gráficos tenham significado mesmo quando não podem ser visualizados diretamente.

Outro recurso essencial foi a implementação de teclas de atalho que possibilitam a navegação sem a utilização do mouse, atendendo especialmente às necessidades de pessoas com

limitações motoras. A Tabela 4.1 apresenta as principais combinações de teclas definidas no sistema, bem como suas funções.

Esses atalhos desempenham papel fundamental na acessibilidade do sistema, pois possibilitam a reversão rápida de ações incorretas, o acesso imediato a funções críticas como salvar ou imprimir documentos, a alternância entre diferentes seções e a navegação fluida entre elementos interativos. Além disso, recursos como o fechamento ágil de janelas modais e o acesso ao topo da página contribuem para reduzir barreiras e garantir maior autonomia ao usuário.

A acessibilidade também foi fortalecida por meio da organização semântica do conteúdo, estruturada de forma lógica para facilitar a interpretação por leitores de tela. Esse recurso permite que títulos, subtítulos e parágrafos sejam identificados corretamente, garantindo que a hierarquia informacional seja preservada.

Tabela 4.1 – Teclas de Atalho Definidas no Sistema

Tecla	Função
Alt + Z	Desfazer a última ação
Alt + Y	Refazer a ação desfeita
Alt + T	Selecionar todo o conteúdo
Alt + G	Abrir a caixa de busca de texto
Alt + D	Salvar o documento ou projeto atual
Alt + W	Alternar entre janelas abertas
Ctrl + →	Navegar entre elementos interativos
Ctrl + ←	Retornar ao elemento anterior
Alt + X	Fechar janelas modais ou menus
Alt + P	Imprimir o documento atual
Alt + N	Abrir nova janela ou documento
F2	Abrir a ajuda do sistema
Alt + C	Fechar a janela ativa
Alt + U	Ir ao topo da página

Elaborado pelo autor.

A implementação de práticas de acessibilidade no sistema tem como objetivo assegurar que pessoas com diferentes tipos de deficiência possam utilizá-lo de forma plena e inclusiva. As diretrizes adotadas estão em conformidade com as recomendações das *Web Content Accessibility Guidelines* (WCAG 2.1) (**WCAG2018**), com o Decreto nº 5.296/2004 (**Brasil2004**) e com a Lei Brasileira de Inclusão da Pessoa com Deficiência (Lei nº 13.146/2015) (**Brasil2015**). A seguir, são descritos os principais recursos desenvolvidos.

Outro aspecto fundamental implementado no sistema foi o contraste adequado entre texto e fundo, assegurando maior legibilidade, sobretudo para pessoas com baixa visão ou daltonismo.

Para isso, foi disponibilizada uma opção de **alto contraste**, ativada por meio de um ícone localizado no canto superior direito de todas as telas, que aplica uma paleta neutra composta por preto, branco e cinza. Essa solução amplia a clareza e a simplicidade visual, reduz a sobrecarga cognitiva dos usuários e garante consistência da experiência tanto na versão para *desktop* quanto para dispositivos móveis.

A Figura 4.12 apresenta a interface de login em sua versão para *desktop*, permitindo a comparação entre o modo com alto contraste ativado (Figura 4.12(a)) e a versão sem o recurso (Figura 4.12(b)). Observa-se que o alto contraste proporciona maior destaque aos elementos da interface, favorecendo a leitura e interação dos usuários.

Figura 4.12 – Alto Contraste — (a) Tela de login com alto contraste ativado e (b) Tela de login sem alto contraste



(a) Tela de login com o recurso de alto contraste ativado.



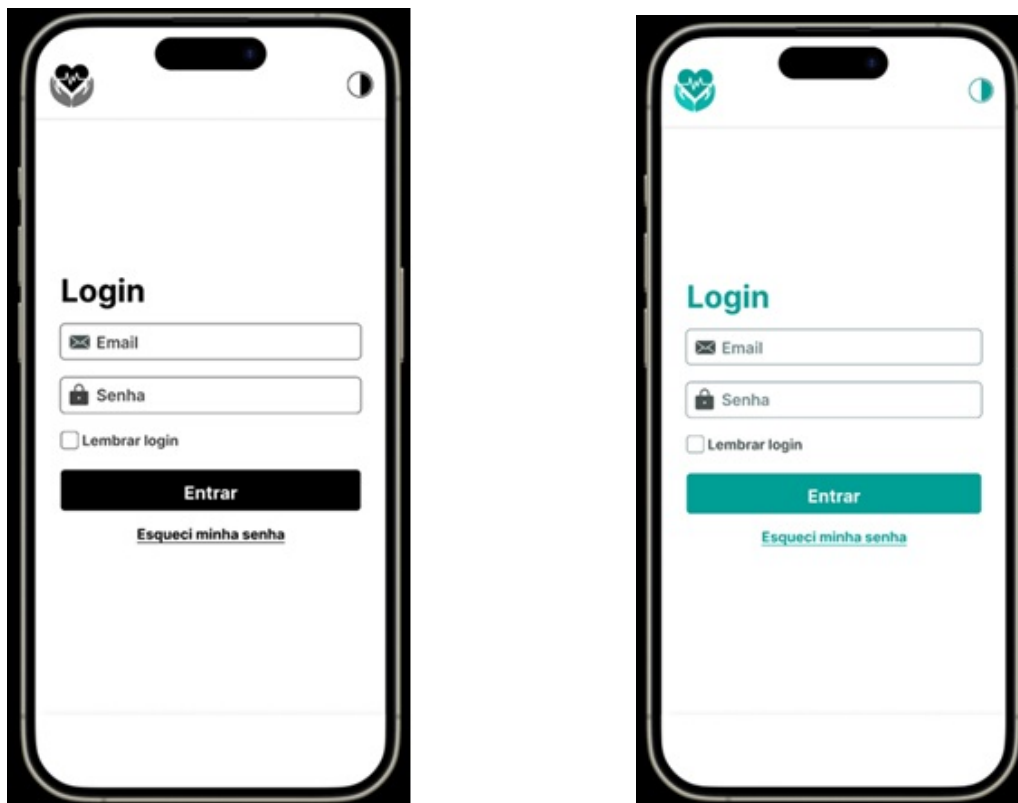
(b) Tela de login com o recurso de alto contraste desativado.

Fonte: Elaborada pelos autores.

De forma complementar, a figura 4.13 ilustra a aplicação do mesmo recurso em dispositivos móveis, destacando a consistência da solução. Na Figura 4.13(a) está representada a tela de

login com alto contraste ativado, enquanto a Figura 4.13(b) mostra a versão sem contraste. Essa adaptação garante que a acessibilidade seja mantida em diferentes plataformas, assegurando a inclusão digital de forma equitativa.

Figura 4.13 – Alto Contraste — Dispositivos Móveis — (a) Tela de login com alto contraste ativado e (b) Tela de login sem alto contraste



(a) Tela de login com o recurso de alto contraste ativado.

(b) Tela de login com o recurso de alto contraste desativado.

Fonte: Elaborada pelos autores.

Outro recurso implementado foi o **texto redimensionável**, que possibilita ao usuário ajustar dinamicamente o tamanho das fontes sem comprometer a disposição ou a legibilidade do conteúdo exibido. Essa funcionalidade é especialmente relevante para pessoas com dificuldades visuais, pois oferece maior conforto na leitura e reduz o esforço cognitivo durante a navegação. Além disso, o recurso beneficia a utilização em diferentes dispositivos, como tablets e smartphones, assegurando que o conteúdo mantenha sua clareza e organização independentemente do tamanho da tela.

A Figura 4.14 ilustra a aplicação prática desse recurso, apresentando a tela principal do sistema (Figura 4.14(a)) e o menu lateral (Figura 4.14(b)) com o texto redimensionado. Observa-se que, mesmo com o aumento do tamanho da fonte, a interface preserva sua estrutura e mantém a usabilidade, garantindo uma experiência inclusiva e acessível para todos os usuários.

Figura 4.14 – Fonte Redimensionada - (a) Tela principal (b) Menu Lateral com fonte redimensinada.



(a) Tela principal com a fonte redimensionada.



(b) Tela principal com o menu lateral com fonte redimensionada.

Fonte: Elaborada pelos autores.

Além das práticas já descritas, outros recursos foram identificados como essenciais para assegurar a acessibilidade em sistemas informatizados. Entre eles, destaca-se a **compatibilidade com leitores de tela**, que garante que todos os elementos interativos, como botões, menus e campos de formulário, sejam corretamente interpretados e compreendidos por pessoas com deficiência visual. Essa funcionalidade possibilita que tecnologias assistivas transmitam de forma precisa o conteúdo, assegurando maior autonomia ao usuário.

Outro aspecto relevante é o **foco visível**, implementado em botões, links e outros elementos interativos, de modo que sejam destacados visualmente quando acessados por meio da navegação via teclado. Essa medida favorece principalmente pessoas com dificuldades motoras ou usuários que não utilizam o mouse, permitindo identificar com clareza o ponto de interação na interface.

Para promover uma experiência mais inclusiva e menos invasiva, o sistema evita a reprodução automática de mídias, adotando a prática de **não utilizar autoplay**. Dessa forma, o usuário mantém total controle sobre o início e o término de áudios e vídeos, o que contribui para a concentração, reduz distrações e assegura a compatibilidade com ferramentas de apoio tecnológico.

A **facilidade de fechamento** também foi considerada, permitindo que janelas modais e pop-ups sejam encerradas de maneira simples, tanto por cliques visuais quanto pela navegação via teclado. Esse recurso impede que o usuário fique preso em elementos da interface, preservando a fluidez da navegação.

Outro recurso de destaque é o **suporte a múltiplas línguas**, com a correta definição da linguagem principal das páginas. Essa prática facilita a atuação de tradutores automáticos e leitores de tela, garantindo interpretação adequada do conteúdo, com leitura e pronúncia precisas.

Para orientar a navegação dentro do sistema, foi implementado o recurso de **breadcrumbs** (migalhas de pão), que exibe ao usuário o caminho percorrido até a página atual. Isso favorece o entendimento da hierarquia do site e facilita a localização de conteúdos.

Além disso, todos os conteúdos multimídia foram planejados de forma acessível. Os **vídeos possuem legendas** e os conteúdos em áudio são acompanhados de transcrições, assegurando que pessoas com deficiência auditiva também tenham acesso completo às informações apresentadas.

A **formatação clara e consistente** do layout foi priorizada, com uso de espaçamento adequado, organização lógica e padronização visual. Essa estrutura beneficia todos os usuários, em especial aqueles com deficiência cognitiva, que necessitam de maior clareza para compreender informações.

Outro ponto importante é o **feedback acessível**, em que mensagens de erro e avisos do sistema são apresentados de forma simples, direta e acompanhados de explicações visuais e textuais. Esse recurso garante que o usuário saiba identificar falhas e entenda como corrigi-las de forma eficiente.

Por fim, assegurou-se o **desempenho em dispositivos móveis**, mantendo as boas práticas de acessibilidade tanto em celulares quanto em tablets. Isso permite que a experiência inclusiva seja preservada em diferentes plataformas, garantindo a universalidade do acesso às funcionalidades do sistema.

Em síntese, a combinação de alto contraste, texto redimensionável, compatibilidade com leitores de tela, foco visível, suporte a múltiplos idiomas, navegação assistida e conteúdos multimídia acessíveis reflete o compromisso com a inclusão digital, assegurando que todos os usuários possam interagir com o sistema de maneira equitativa e eficaz.

5 Banco de Dados

Os dados constituem um ativo central em sistemas de informação: deles derivam relatórios, análises e funcionalidades críticas do negócio. Nessa perspectiva, um banco de dados pode ser definido como um conjunto organizado de dados relacionados que representam aspectos do mundo real, armazenados com um propósito específico e de forma a permitir recuperação e atualização consistentes (Machado2020).

Dentre os paradigmas de armazenamento, o modelo relacional permanece hegemônico no processamento transacional e analítico de grande parte das aplicações. Nesse modelo, a informação é estruturada em tabelas (relações) formadas por tuplas (linhas) e atributos (colunas), sob um esquema que explicita domínios, chaves e restrições de integridade (Silberschatz2012). Em um processo típico de projeto, parte-se de um modelo conceitual (E-R), mapeiam-se entidades e relacionamentos para o modelo lógico relacional e, por fim, define-se o esquema físico com chaves primárias e estrangeiras, índices e restrições. A normalização reduz redundâncias e anomalias de atualização, enquanto a criação criteriosa de índices e *views* apoia desempenho e legibilidade das consultas (Silberschatz2012).

No contexto deste projeto, adotou-se o PostgreSQL como SGBD relacional para gerenciar e processar o fluxo de dados de forma eficiente. Trata-se de uma plataforma objeto-relacional de código aberto, apta a implementar chaves, restrições, *triggers*, funções e consultas complexas, alinhando-se às necessidades de integridade e desempenho esperadas em sistemas corporativos (Silberschatz2012).

Operações de acesso e atualização são agrupadas em transações, unidades lógicas de trabalho que devem preservar as propriedades *ACID* (Silberschatz2012): (i) **atomicidade**, garantindo que todas as ações de uma transação sejam aplicadas integralmente ou, em caso de falha, nenhuma o seja; (ii) **consistência**, assegurando que o banco de dados transite apenas entre estados válidos; (iii) **isolamento**, evitando que execuções concorrentes interfiram entre si e produzam leituras sujas, não repetíveis ou fenômenos de fantasma; e (iv) **durabilidade**, pela qual os efeitos de uma transação confirmada permanecem mesmo diante de falhas do sistema. Em conjunto, essas propriedades permitem conciliar concorrência, integridade e desempenho, sustentando um comportamento previsível da aplicação (Silberschatz2012).

5.1 Modelo Entidade-Relacionamento

O Modelo Entidade-Relacionamento (E-R) é um artefato conceitual voltado a capturar, de forma independente de implementação, os aspectos de um domínio de dados. Por meio dele identificam-se *entidades* e seus *atributos* (simples, compostos, multivalorados ou derivados), bem como *relacionamentos* (binários ou n-ários) com suas *cardinalidades* (1:1, 1:N, N:M) e *participações* (total ou parcial). O modelo também evidencia chaves (candidatas e primária),

entidades fracas e especializações/generalizações, oferecendo uma visão clara e compartilhável da estrutura informacional antes de decisões físicas (**Silberschatz2012**).

A passagem do E-R para o esquema relacional (*mapeamento objeto-relacional*, no sentido de mapeamento conceitual→relacional) segue regras sistemáticas: entidades fortes tornam-se tabelas; entidades fracas geram tabelas que incorporam a chave do “dono”; relacionamentos 1:N tendem a ser implementados por chave estrangeira no lado N; relacionamentos N:M resultam em tabelas associativas; relacionamentos 1:1 podem ser implementados com chave estrangeira (com unicidade); atributos multivalorados e compostos demandam decomposição apropriada; e atributos derivados, em geral, não são persistidos. O resultado é um esquema relacional com chaves primárias e estrangeiras, restrições de integridade e base para normalização (1FN/2FN/3FN), reduzindo redundâncias e anomalias de atualização (**Silberschatz2012**).

A Figura 5.1 apresenta o mapeamento conceitual→relacional do sistema desenvolvido neste trabalho, evidenciando tabelas, atributos e relacionamentos. Essa visão auxilia a compreender a organização do banco de dados, destacando como as restrições de integridade e as dependências entre entidades foram traduzidas para o modelo relacional (**Silberschatz2012**).

Figura 5.1 – Mapeamento Objeto-Relacional

Fonte: Elaborada pelos autores.

5.2 Script das Tabelas

Segundo **Machado2020**, um *script* de tabelas corresponde a um conjunto de instruções SQL voltadas à definição do *esquema* do banco de dados. Em termos clássicos, trata-se de comandos de *Data Definition Language (DDL)*—como **CREATE**, **ALTER** e **DROP**—que estabelecem estruturas e restrições, em contraste com a *Data Manipulation Language (DML)*, voltada ao manuseio dos dados em si. Na criação de tabelas, o comando **CREATE TABLE** especifica o nome da tabela, suas colunas e tipos de dados, bem como propriedades essenciais: nulabilidade (**NOT NULL**), valores padrão (**DEFAULT**), chaves primárias e únicas (**PRIMARY KEY**, **UNIQUE**), *checks* (**CHECK**) e chaves estrangeiras (**FOREIGN KEY**) com suas ações referenciais (por exemplo, **ON DELETE CASCADE/RESTRICT**) (**Machado2020**).

Boas práticas usuais incluem padronização de nomes, escolha adequada de tipos (evitando comprimentos excessivos ou tipos genéricos), definição explícita de restrições de integridade, comentários no esquema quando disponíveis e criação de índices auxiliares (**CREATE INDEX**) para colunas com alta seletividade. Essas decisões de projeto tornam o esquema mais claro e resiliente, e mitigam anomalias de atualização e gargalos de desempenho (**Machado2020**).

A seguir, nos Quadros 5.1 a 5.23 apresenta-se os scripts para geração do banco de dados utilizado no projeto.

Quadro 5.1 – Script SQL — Tabela address

```
1 CREATE TABLE address (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     street character varying(255) NOT NULL,  
4     number character varying(20) NOT NULL,  
5     complement character varying(255),  
6     neighborhood character varying(255) NOT NULL,  
7     zip_code character varying(20) NOT NULL,  
8     city_id integer NOT NULL,  
9     CONSTRAINT "PK_address" PRIMARY KEY (id)  
10 );
```

Fonte: Elaborado pelos autores.

Quadro 5.2 – Script SQL — Tabela allergy

```
1 CREATE TABLE allergy (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(100) NOT NULL,  
4     type integer NOT NULL,  
5     CONSTRAINT "PK_allergy" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.3 – Script SQL — Tabela carrier

```
1 CREATE TABLE carrier (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     document_number character varying(18) NOT NULL,  
5     email character varying(255),  
6     phone_number character varying(20),  
7     description character varying(255),  
8     CONSTRAINT "PK_carrier" PRIMARY KEY (id)  
9 );
```

Fonte: Elaborado pelos autores.

Quadro 5.4 – Script SQL — Tabela city

```
1 CREATE TABLE city (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     state_id integer NOT NULL,  
5     CONSTRAINT "PK_city" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.5 – Script SQL — Tabela health_insurance

```
1 CREATE TABLE health_insurance (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     register_number character varying(50),  
5     phone_number character varying(20),  
6     email character varying(255),  
7     description character varying(255),  
8     CONSTRAINT "PK_health_insurance" PRIMARY KEY (id)  
9 );
```

Fonte: Elaborado pelos autores.

Quadro 5.6 – Script SQL — Tabela health_insurance_plan

```
1 CREATE TABLE health_insurance_plan (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     code character varying(50),  
5     description character varying(255),  
6     health_insurance_id integer NOT NULL,  
7     CONSTRAINT "PK_health_insurance_plan" PRIMARY KEY (id)  
8 );
```

Fonte: Elaborado pelos autores.

Quadro 5.7 – Script SQL — Tabela login

```
1 CREATE TABLE login (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     user_id integer NOT NULL,  
4     login_date timestamp without time zone NOT NULL,  
5     ip_address character varying(45),  
6     user_agent character varying(255),  
7     CONSTRAINT "PK_login" PRIMARY KEY (id)  
8 );
```

Fonte: Elaborado pelos autores.

Quadro 5.8 – Script SQL — Tabela manufacturer

```
1 CREATE TABLE manufacturer (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     document_number character varying(18),  
5     phone_number character varying(20),  
6     email character varying(255),  
7     description character varying(255),  
8     CONSTRAINT "PK_manufacturer" PRIMARY KEY (id)  
9 );
```

Fonte: Elaborado pelos autores.

Quadro 5.9 – Script SQL — Tabela product

```
1 CREATE TABLE product (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     code character varying(50) NOT NULL,  
5     description character varying(255),  
6     product_group_id integer NOT NULL,  
7     product_type_id integer NOT NULL,  
8     manufacturer_id integer,  
9     unit_of_measure_id integer NOT NULL,  
10    CONSTRAINT "PK_product" PRIMARY KEY (id)  
11 );
```

Fonte: Elaborado pelos autores.

Quadro 5.10 – Script SQL — Tabela product_category

```
1 CREATE TABLE product_category (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     description character varying(255),  
5     CONSTRAINT "PK_product_category" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.11 – Script SQL — Tabela product_group

```
1 CREATE TABLE product_group (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     description character varying(255),  
5     product_category_id integer NOT NULL,  
6     CONSTRAINT "PK_product_group" PRIMARY KEY (id)  
7 );
```

Fonte: Elaborado pelos autores.

Quadro 5.12 – Script SQL — Tabela product_type

```
1 CREATE TABLE product_type (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     description character varying(255),  
5     CONSTRAINT "PK_product_type" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.13 – Script SQL — Tabela profile

```
1 CREATE TABLE profile (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     description character varying(255),  
5     CONSTRAINT "PK_profile" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.14 – Script SQL — Tabela profile_claim

```
1 CREATE TABLE profile_claim (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     profile_id integer NOT NULL,  
4     claim character varying(255) NOT NULL,  
5     CONSTRAINT "PK_profile_claim" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.15 – Script SQL — Tabela profile_resource

```
1 CREATE TABLE profile_resource (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     profile_id integer NOT NULL,  
4     resource_id integer NOT NULL,  
5     CONSTRAINT "PK_profile_resource" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.16 – Script SQL — Tabela resident

```
1 CREATE TABLE resident (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     date_of_birth date NOT NULL,  
5     document_number character varying(20),  
6     gender integer NOT NULL,  
7     marital_status integer,  
8     phone_number character varying(20),  
9     email character varying(255),  
10    address_id integer NOT NULL,  
11    CONSTRAINT "PK_resident" PRIMARY KEY (id)  
12 );
```

Fonte: Elaborado pelos autores.

Quadro 5.17 – Script SQL — Tabela resident_allergy

```
1 CREATE TABLE resident_allergy (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     resident_id integer NOT NULL,  
4     allergy_id integer NOT NULL,  
5     description character varying(255),  
6     CONSTRAINT "PK_resident_allergy" PRIMARY KEY (id)  
7 );
```

Fonte: Elaborado pelos autores.

Quadro 5.18 – Script SQL — Tabela resident_health_insurance

```
1 CREATE TABLE resident_health_insurance (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     resident_id integer NOT NULL,  
4     health_insurance_plan_id integer NOT NULL,  
5     card_number character varying(50),  
6     expiration_date date,  
7     CONSTRAINT "PK_resident_health_insurance" PRIMARY KEY (id)  
8 );
```

Fonte: Elaborado pelos autores.

Quadro 5.19 – Script SQL — Tabela resident_relative

```
1 CREATE TABLE resident_relative (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     resident_id integer NOT NULL,  
4     name character varying(255) NOT NULL,  
5     kinship_degree integer NOT NULL,  
6     phone_number character varying(20),  
7     email character varying(255),  
8     CONSTRAINT "PK_resident_relative" PRIMARY KEY (id)  
9 );
```

Fonte: Elaborado pelos autores.

Quadro 5.20 – Script SQL — Tabela resource

```
1 CREATE TABLE resource (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     description character varying(255),  
5     CONSTRAINT "PK_resource" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.21 – Script SQL — Tabela state

```
1 CREATE TABLE state (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     abbreviation character varying(2) NOT NULL,  
5     CONSTRAINT "PK_state" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.22 – Script SQL — Tabela unitofmeasure

```
1 CREATE TABLE unitofmeasure (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(50) NOT NULL,  
4     abbreviation character varying(10) NOT NULL,  
5     CONSTRAINT "PK_unitofmeasure" PRIMARY KEY (id)  
6 );
```

Fonte: Elaborado pelos autores.

Quadro 5.23 – Script SQL — Tabela user

```
1 CREATE TABLE "user" (  
2     id integer GENERATED BY DEFAULT AS IDENTITY,  
3     name character varying(255) NOT NULL,  
4     email character varying(255) NOT NULL,  
5     password_hash character varying(255) NOT NULL,  
6     is_active boolean NOT NULL,  
7     profile_id integer NOT NULL,  
8     CONSTRAINT "PK_user" PRIMARY KEY (id)  
9 );
```

Fonte: Elaborado pelos autores.

5.3 Mapeamento Objeto Relacional

De acordo com **Silberschatz2012**, o Mapeamento Objeto-Relacional (ORM) é uma técnica que facilita a interação entre sistemas orientados a objetos e bancos de dados relacionais. Em essência, o ORM provê uma camada de abstração que representa tabelas, colunas e relações por meio de classes, atributos e associações, reduzindo o chamado “descompasso” objeto-relacional e evitando a escrita manual de comandos SQL para operações de criação, leitura, atualização e exclusão (*CRUD*). Com isso, o desenvolvimento torna-se mais simples e coeso, favorecendo legibilidade, reuso e manutenção do código (**Silberschatz2012**).

No contexto deste projeto, adotou-se o *Entity Framework*, um *framework* ORM difundido para a linguagem C#. A abordagem permite mapear classes da aplicação para tabelas do banco com configurações mínimas (quando desejado), ao mesmo tempo em que respeita customizações do desenvolvedor (**Silberschatz2012**). O processo de mapeamento ocorre de forma transparente: a camada de persistência assume tarefas como gerar instruções SQL, materializar objetos a partir de tuplas e sincronizar alterações, enquanto o desenvolvedor trabalha com modelos de domínio (**Silberschatz2012**).

Além do mapeamento básico (classes → tabelas, atributos → colunas, associações → chaves estrangeiras), o uso de ORM favorece:

- a definição explícita de relacionamentos (*um-para-muitos*, *muitos-para-muitos*) com integridade referencial preservada (**Silberschatz2012**);
- o controle de transações e a aderência às propriedades *ACID*, assegurando consistência e durabilidade das operações (**Silberschatz2012**);
- a validação de dados e o rastreamento de alterações (*change tracking*) no ciclo de vida dos objetos, reduzindo *boilerplate* na camada de persistência (**Silberschatz2012**);
- a otimização de acesso por consultas expressivas e carregamento apropriado de associações, conforme a necessidade do caso de uso (**Silberschatz2012**).

Aplicado ao sistema em desenvolvimento, por exemplo, a entidade que representa *pontos turísticos* é mapeada diretamente para uma tabela relacional; as operações de persistência são delegadas ao *Entity Framework*, que trata a emissão de comandos, o gerenciamento de relacionamentos e as transações. O resultado é um ciclo de desenvolvimento mais ágil, com menor complexidade acoplada à camada de dados e com foco do time nas regras de negócio (**Silberschatz2012**).

Em suma, o uso de um ORM como o *Entity Framework* viabiliza uma separação clara de responsabilidades: a aplicação concentra-se nas funcionalidades de domínio, enquanto o *framework* atua sobre a persistência, mantendo as garantias do modelo relacional (**Silberschatz2012**).

6 Arquitetura de Software

Segundo **Sommerville2018ES10**, a arquitetura do software deve ser definida antes da etapa de implementação, pois isso assegura uma conexão clara entre os requisitos do sistema e a estruturação de seus componentes.

Uma arquitetura bem planejada possibilita o desenvolvimento de um sistema eficiente, promovendo uma divisão clara dos componentes, organização e comunicação eficaz entre eles. Além disso, facilita uma implementação direta e objetiva, atendendo às necessidades operacionais e garantindo manutenção simplificada, o que resulta na redução de custos. Esses processos têm como objetivo otimizar o ciclo de vida do sistema e evitar a sobrecarga dos desenvolvedores durante o processo de criação (**Martin2020CleanArchPT**).

6.1 Arquitetura de Desenvolvimento

Para a execução deste projeto, foram realizadas etapas como a coleta de dados, o diagnóstico da situação atual e o levantamento das informações essenciais ao processo de gestão de despesas públicas. Com base na análise dos dados coletados e no estudo das funcionalidades necessárias, procedeu-se ao levantamento dos requisitos para a modelagem e o desenvolvimento do software, seguindo os princípios metodológicos da engenharia de software delineados por **Pressman2011ES7**.

Para representar a análise de requisitos, utilizou-se a Linguagem de Modelagem Unificada (UML), uma linguagem de modelagem baseada no paradigma de orientação a objetos (**Guedes2018UML2**). Por meio do software Astah UML (**AstahUML**), foram desenvolvidos diversos diagramas UML que permitiram uma análise detalhada das funcionalidades e dos requisitos operacionais do sistema, facilitando a identificação precisa dos serviços e recursos que a plataforma pode oferecer aos usuários.

Na fase de desenvolvimento do *back-end* da aplicação, foi utilizado o Ambiente de Desenvolvimento Integrado (IDE) Visual Studio, reconhecido por suas funcionalidades avançadas de edição de código e recursos robustos para testes. A linguagem de programação selecionada foi o C#, conhecida por ser uma linguagem de uso geral, multiplataforma e de alto desempenho (**Wagner2024TourCSharp**). Para o armazenamento e a gestão dos dados, foi empregado o banco de dados PostgreSQL, um Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR) projetado para administrar o acesso eficiente e seguro às informações (**Milani2008PostgreSQL**).

A prototipação das interfaces do sistema foi realizada com a ferramenta Figma (**Figma2024**), e, para o desenvolvimento do *front-end*, foi escolhido o framework Flutter, uma estrutura de código aberto que permite a criação de sistemas multiplataforma a partir de uma única base de código (**Flutter2024**). O ambiente de desenvolvimento utilizado foi o

Visual Studio Code, com a linguagem de programação Dart (**DartSite2024**), garantindo maior eficiência e uniformidade no desenvolvimento da interface de usuário.

Para a gestão do projeto, foi adotada a metodologia Scrum (**Sutherland2019Scrum**), utilizando *sprints* para um desenvolvimento iterativo e incremental. Após cada entrega parcial, foram realizadas análises detalhadas dos módulos específicos, seguidas por uma avaliação global do sistema. As etapas do projeto foram organizadas e monitoradas rigorosamente na plataforma Azure DevOps (**Microsoft2024AzureDevOps**), uma ferramenta de controle de tarefas que oferece funcionalidades gratuitas para o gerenciamento de projetos, incluindo planejamento de *sprints*, controle de tarefas e relatórios de desempenho. Para o armazenamento e versionamento de código do projeto *front-end* e *back-end*, empregou-se a ferramenta GitHub (**GitHub2024**).

6.1.1 Back-End

O back-end do *Comtur* foi desenvolvido em C#, amplamente reconhecida como uma das linguagens centrais do ecossistema .NET. **Schildt2009** descreve a linguagem como um dos pilares da Microsoft para o desenvolvimento moderno, enquanto **Skeet2019** a caracteriza como uma linguagem orientada a objetos, contemporânea e apropriada à criação de aplicações robustas e de alto desempenho. Tais características sustentam sua adoção em um sistema complexo e orientado a dados como o *Comtur*.

A arquitetura do back-end foi estruturada em múltiplas camadas, cada qual com responsabilidades bem definidas. **Fowler2012** ressalta que a separação de responsabilidades melhora modularidade, manutenção e escalabilidade. No *Comtur*, a camada de API atua como ponto de entrada, recebendo requisições, validando dados e encaminhando comandos para as camadas apropriadas. Em linha com padrões consolidados, **FreemanRobson2020** discutem como a organização clara de responsabilidades (*p. ex.*, abordagens inspiradas em MVC e em padrões de projeto) promove eficiência tanto no processamento quanto na interação com o usuário.

A manipulação de dados é encapsulada por uma camada específica de acesso e persistência, que intermedeia lógica de negócios e armazenamento. **Evans2004** destaca o uso de repositórios como uma abstração que simplifica a lógica de domínio e facilita testes. Já na camada de regras de negócio, concentram-se operações centrais do sistema e políticas de domínio; **Martin2020CleanArchPT** defende a separação entre lógica de negócios e persistência para reduzir acoplamento e aumentar flexibilidade.

Outra camada fundamental gerencia a conexão com o banco de dados e o mapeamento entre entidades e tabelas. **Lerman2019** observa que o uso de *DbContexts* no Entity Framework simplifica configuração, mapeamento e operações de persistência, elevando a eficiência do desenvolvimento. Por fim, arquivos de inicialização da aplicação (**Program.cs**, **Startup.cs**) centralizam registro de serviços e injeção de dependências; conforme **Proise2021**, essa configuração inicial adequada é essencial para evitar problemas em tempo de execução e assegurar um pipeline de inicialização coeso.

Com tal organização em camadas, a arquitetura do projeto favorece manutenção evolutiva

e expansão, com componentes de responsabilidade nítida e baixo acoplamento—um arranjo coerente com recomendações clássicas da literatura de arquitetura de software.

6.1.2 Front-End

O *front-end* constitui a camada de apresentação da aplicação, mediando a comunicação com o servidor e tornando possível a interação efetiva do usuário com o sistema. Trata-se da parte visível e manipulável da solução: onde conteúdos são exibidos, ações são iniciadas e feedbacks são recebidos. Segundo **Meloni2018HTMLCSSJS4e**, essa camada é fundamental porque concentra a experiência direta do usuário, respondendo pela organização visual das informações, pela clareza dos controles e pela fluidez das interações.

Neste projeto, o *front-end* foi implementado com *React*, biblioteca JavaScript voltada à criação de interfaces por meio de componentes reutilizáveis que gerenciam seu próprio estado e podem ser combinados para compor telas complexas (**ReactLegacyPTBR2024**). Como camada de interface, ele agrega elementos de navegação e interação (menus, botões, imagens, formulários e *layout*) com o objetivo de entregar uma experiência coerente, intuitiva e eficiente ao usuário (**TOTVS2021**).

A adoção de *React* favorece princípios de engenharia de software no cliente, como *componentização*, *separação de responsabilidades* e *baixo acoplamento*: cada componente encapsula apresentação, estado e regras locais, podendo evoluir de modo relativamente independente dos demais (**ReactLegacyPTBR2024**). Além disso, o modelo de atualização baseado em representação declarativa da interface e gerenciamento explícito de estado contribui para um código mais previsível e manutenível, reduzindo a necessidade de manipulações imperativas dispersas (**ReactLegacyPTBR2024**).

Do ponto de vista de experiência de uso, o *front-end* integra *layout* responsivo, padrões de navegação consistentes e componentes de interface padronizados para diminuir a carga cognitiva e facilitar a descoberta de funcionalidades (**TOTVS2021**). Em termos de evolução do produto, a granularidade dos componentes permite incorporar novos recursos de forma incremental, com menor risco de regressões e menor impacto nas partes já estáveis da aplicação (**ReactLegacyPTBR2024**). A ampla comunidade e a documentação de *React* também aceleram o desenvolvimento e ampliam o repertório de soluções prontas para problemas recorrentes de interface (**ReactLegacyPTBR2024**).

Em suma, ao alinhar boas práticas de construção de interfaces com uma biblioteca que estimula modularidade e reutilização, o *front-end* entrega uma base sólida para experiências eficientes, consistentes e escaláveis—em consonância com as recomendações de organização visual e interação descritas por **Meloni2018HTMLCSSJS4e** e com os pressupostos de usabilidade aplicados ao contexto de aplicações web (**TOTVS2021**; **ReactLegacyPTBR2024**).

6.2 Segurança da Informação

A segurança da informação constitui um dos pilares centrais na área de tecnologia da informação, visto que sua principal finalidade é proteger dados e informações sensíveis contra acessos não autorizados, modificações indevidas, perdas ou divulgações inadequadas. Segundo **Fontes2008**, a segurança da informação envolve a implementação de políticas e práticas capazes de assegurar a confidencialidade, a integridade e a disponibilidade das informações, aspectos essenciais para que o ambiente computacional seja considerado confiável e seguro. Esse conjunto de medidas tornou-se indispensável em razão do aumento da conectividade e da crescente exposição dos sistemas a ameaças digitais, que podem resultar em sérios prejuízos financeiros, danos reputacionais e implicações legais.

Conforme **HoldsworthKosinski2024**, a área de segurança da informação, também conhecida como *Information Security* (InfoSec), fundamenta-se na tríade CIA (*Confidentiality, Integrity e Availability*). A confidencialidade garante que somente usuários autorizados tenham acesso a informações sensíveis; a integridade assegura que os dados permaneçam completos, precisos e livres de modificações não autorizadas; e a disponibilidade garante que os recursos estejam acessíveis sempre que necessários, sem interrupções indevidas. Além desses princípios, os autores destacam a *Garantia da Informação*, que corresponde à manutenção contínua dos elementos da tríade CIA, e a *Não Repudição*, que impede que usuários neguem a autoria de ações realizadas no sistema, visto que cada interação é autenticada e rastreável.

Nesse contexto, soluções eficazes de autenticação e autorização desempenham papel fundamental para viabilizar a proteção de sistemas distribuídos. Uma das tecnologias mais utilizadas atualmente é o *JSON Web Token* (JWT), um padrão aberto definido na RFC 7519 por **JonesBradleySakimura2015**. O JWT é descrito como um formato compacto e autossuficiente para a transmissão de informações em formato JSON entre diferentes partes de um sistema, garantindo integridade e autenticidade dos dados (**JWTIO2024**). Seu funcionamento ocorre, em geral, a partir do login do usuário: as credenciais são validadas, um *payload* é criado com informações relevantes, e o token é assinado digitalmente com uma chave secreta. A estrutura do JWT é composta por três partes: o *header*, que contém dados sobre o tipo de token e o algoritmo de assinatura; o *payload*, que armazena as informações a serem transmitidas; e a *signature*, que assegura a autenticidade do emissor (**JWTIO2024**).

A utilização do JWT apresenta vantagens significativas para sistemas distribuídos. De acordo com **Bradley2020**, o uso de tokens elimina a necessidade de armazenamento de sessões no servidor, o que reduz riscos de roubo de credenciais e contribui para a escalabilidade da aplicação. Cada requisição autenticada passa a incluir o token no cabeçalho *Authorization*, possibilitando que o servidor valide sua integridade, sua assinatura e o prazo de validade. A segurança é reforçada com a adoção de algoritmos robustos de assinatura e de funções de *hash* como o SHA-256, amplamente empregado para garantir que as informações não tenham sido alteradas de forma indevida. Segundo a **IBM2021**, o SHA-256 é fundamental na proteção de dados em trânsito e na verificação da integridade de tokens e mensagens.

Além da autenticação, outro aspecto crucial da segurança da informação é a auditoria.

Schneier2015 ressalta que as auditorias devem registrar detalhadamente ações realizadas no sistema, como acessos, modificações e tentativas de invasão, de modo a viabilizar a detecção de falhas, o atendimento a requisitos legais e a conformidade com regulamentações internacionais, como o GDPR. No sistema, por exemplo, a auditoria registra informações como valores anteriores e atuais, datas, horários e usuários responsáveis pelas ações, permitindo rastreabilidade completa e fortalecendo a transparência das operações (**PressmanMaxim2021**). Além disso, esses registros podem subsidiar análises avançadas, identificando padrões de uso e possíveis atividades suspeitas.

Outro ponto fundamental refere-se à segurança na comunicação entre cliente e servidor. A utilização de protocolos criptográficos, como o TLS (*Transport Layer Security*), em conjunto com o uso de HTTPS, é indispensável para evitar a interceptação de informações sensíveis. **RFC8446** destaca que, quando transmitido sobre conexões seguras, o JWT mantém a confidencialidade das mensagens, assegurando que o token não seja capturado por agentes maliciosos durante o tráfego de dados.

Dessa forma, observa-se que a segurança da informação envolve a combinação de múltiplas estratégias: a aplicação da tríade CIA, a autenticação e autorização via JWT, o uso de algoritmos de criptografia e funções de *hash*, bem como a auditoria detalhada de ações no sistema. Em conjunto, essas medidas proporcionam ambientes computacionais mais confiáveis, capazes de proteger informações críticas e de atender às exigências legais e normativas. Assim, a integração entre tecnologias e boas práticas torna-se imprescindível para o desenvolvimento de sistemas modernos, seguros e escaláveis (**Fontes2008; Anderson2021; HoldsworthKosinski2024; JonesBradleySakimura2015; JWTIO2024; Bradley2020; IBM2021; Schneier2015; PressmanMaxim2021; RFC8446**).

7 Controle de Participação no Projeto

A participação dos estudantes no projeto de extensão universitária foi registrada de forma sistemática, contemplando as tarefas desenvolvidas em cada *sprint*. Esse acompanhamento possibilitou monitorar o cumprimento das atividades propostas e evidenciar tanto o envolvimento do aluno quanto a carga horária dedicada ao projeto. A Tabela ?? apresenta a relação das tarefas executadas, acompanhadas da respectiva descrição, data de realização e número de horas trabalhadas, fornecendo uma visão clara da contribuição efetiva do discente para o desenvolvimento das ações.

Tabela 7.1 – Horas Trabalhadas no Projeto.

Sprint	Data	Descrição	Horas
2024			107
#1	30/09/2024	Task 02 - Modelar o design da tela de abertura do sistema, tela de login e tela administrativa principal.	15
#1	30/09/2024	Task 03 - Modelar design responsivo para telas de celulares.	12
#2	13/10/2024	Task 04 - Definir componentes de acessibilidade.	30
#3	26/10/2024	Task 15 – Criar projeto <i>front-end React</i> .	02
#4	09/11/2024	Task 16 – Desenvolver tela de entrada do sistema.	07
#4	09/11/2024	Task 20 – Definir padronização de configurações de acessibilidade e interface.	09
#5	23/11/2024	Task 25 – Desenvolver a tela de cadastro para religião.	32
2025			99999
#1		Task - Modelar... .	15

Elaborado pelo autor.

7.1 Relatório de Sprints

7.1.1 Sprint 01 - Task 02 e 03



Faculdade de Tecnologia Professor José Camargo

Sprint Documentation		
Sprint #1	Start Date: 16/09/2024	Final Date: 30/09/2024
Project Name:	SeniorCareManager-Frontend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Edson Pereira De Carvalho		
Lemuel Prates Sobrinho Ferreira		
Maria Clara Borges Cardoso		
Rodrigo Yoshida Lombazzi		
Vinicius Lúcio Marcolino Da Silva		

Sprint Backlog			
Task#	Description	Start Date	Final date
2	Sprint 01 - Modelar o design da tela de abertura do sistema, tela de login e tela administrativa principal	16/09/2024	23/09/2024
3	Sprint 01 - Modelar design responsivo para telas de celulares	23/09/2024	30/09/2024

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
2	Sprint 01 - Modelar o design da tela de abertura do sistema, tela de login e tela administrativa principal	Maria Clara	Concluído	15	15
3	Sprint 01 - Modelar design responsivo para telas de celulares	Maria Clara	Concluído	8	12

Sprint Description	
Durante a Sprint 1, foram criados os primeiros protótipos do sistema SeniorCareManager, incluindo a tela de abertura, login, painel administrativo, calendário e visão geral, com foco em identidade visual, usabilidade e design responsivo. A paleta de cores escolhida (verde, verde-água, branco e cinza) transmite tranquilidade. Elementos visuais como logotipo e ícones foram utilizados para reforçar a identidade. As funcionalidades essenciais foram priorizadas, enquanto recursos adicionais ficaram para as próximas sprints. O protótipo está disponível no Figma e servirá de base para as futuras implementações.	

Sprint Results

Task #2:

Paleta de Cores:

No geral, as cores utilizadas nas páginas foram verde, verde-água, branco e cinza. O verde e o verde-água representam saúde, harmonia e tranquilidade; por isso, foram utilizados para destacar botões, textos, ícones, logo e informações. Já o branco e o cinza, por serem neutros, foram utilizados como cores de fundo do site.

Abertura do Sistema

Na página de abertura do sistema, diversos elementos estão dispostos para introduzir o usuário à plataforma.

1. Elementos Visuais

- **Logotipo do Site:** O logotipo do sistema está destacado, reforçando a identidade visual da plataforma.
- **Imagem:** Uma imagem selecionada no Canva complementa o design da página, tornando-a mais atrativa.

2. Informações e Funcionalidades

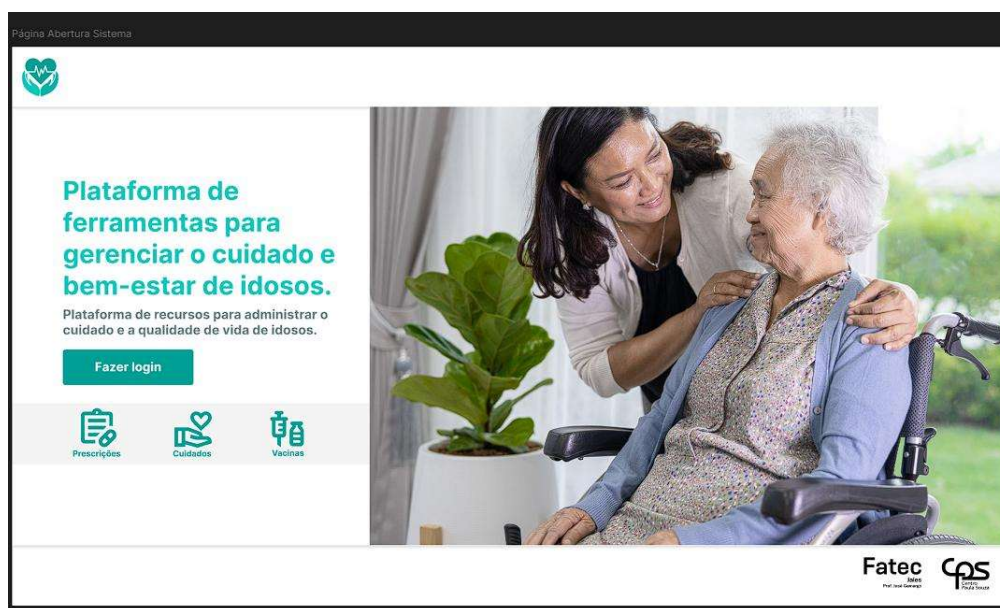
- **Atividades e Serviços:** São apresentadas algumas das atividades e serviços que podem ser realizados no sistema, ressaltando suas principais funcionalidades.
- **Descrição:** Uma breve descrição esclarece o funcionamento do sistema e sua utilidade, destacando os principais benefícios para o usuário.

3. Botão de Login

Um botão direciona o usuário para a página de login, onde poderá acessar o sistema com facilidade.

4. Rodapé

Na parte inferior da página, está o logotipo da empresa desenvolvedora do site, evidenciando a autoria e o suporte técnico por trás da plataforma.



Página de Login

A página de login é composta por diversos elementos que facilitam o acesso ao sistema. Na parte superior, está o logotipo do site, que destaca a identidade visual da plataforma.

1. Campos de Acesso

A área de login inclui campos para inserção de e-mail e senha, permitindo que o usuário se autentique no sistema. Abaixo desses campos, há uma opção para lembrar o login, possibilitando que o usuário permaneça conectado em acessos futuros.

2. Botões e Links

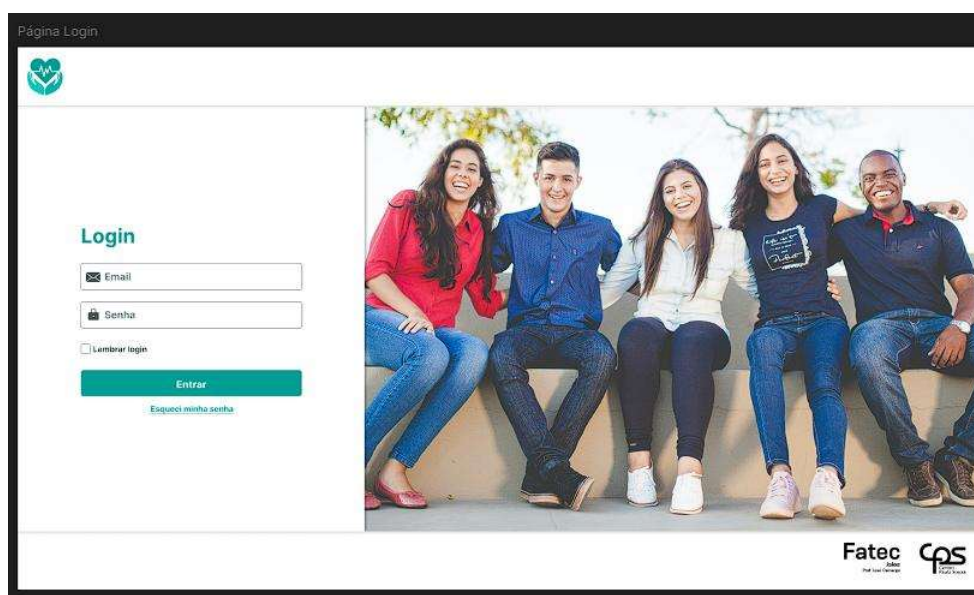
Além dos campos de acesso, há um botão para realizar o login. Caso o usuário esqueça a senha, um link redireciona para a página de recuperação de senha, garantindo um processo simples de recuperação.

3. Elementos Visuais

No lado direito da página, uma imagem obtida do Canva adiciona um elemento visual ao layout, proporcionando uma experiência mais atrativa e agradável para o usuário.

4. Rodapé

Na parte inferior da página, encontra-se o logotipo da entidade responsável pelo desenvolvimento do sistema, reforçando a credibilidade e a autoria da solução apresentada.



Página Administrativa Principal:

1. Visão do Administrador

A visão do administrador é composta por um menu com ícones que facilitam a navegação entre as funcionalidades principais do sistema. O primeiro ícone, representado por uma casa, direciona para a tela inicial. O segundo ícone exibe a lista de funcionários registrados. O terceiro ícone corresponde aos idosos cadastrados no lar. O quarto ícone é o calendário, onde estão registrados os compromissos e atividades agendadas. Por fim, o quinto ícone representa os cuidados prestados no lar.

2. Cabeçalho (Header)

O cabeçalho da página contém o logotipo do sistema e uma barra de pesquisa, que permite realizar buscas rápidas por ações ou informações específicas. Além disso, há três ícones adicionais: o primeiro, representado por uma carta, exibe as mensagens recebidas; o segundo, indicado por um ponto de interrogação, corresponde às dúvidas; e o terceiro, um sino, notifica sobre a conclusão de serviços realizados. Por fim, são exibidos o nome e a foto do administrador.

3. Menu

O menu de navegação está localizado na parte esquerda de todas as páginas administrativas. Ao clicar sobre a imagem do usuário, o menu se expande, exibindo as opções disponíveis para o usuário. As opções estão organizadas da seguinte forma:

- Tela Inicial (Ícone de Casa): O primeiro ícone, representado por uma casa, leva o usuário à página inicial do sistema.
- Funcionários: O segundo ícone exibe uma lista com os funcionários registrados, permitindo o gerenciamento eficiente de suas informações.
- Idosos: O terceiro ícone dá acesso aos idosos cadastrados no lar, oferecendo informações detalhadas e históricos.
- Calendário: O quarto ícone apresenta o calendário, onde estão registrados os compromissos e atividades agendadas no lar.
- Cuidados: O quinto ícone representa os cuidados prestados no lar, contendo informações sobre os serviços realizados para os idosos.

4. Visão Geral

A página de visão geral é uma das opções disponíveis no menu e desempenha um papel crucial no monitoramento das atividades diárias. Nela, são apresentadas informações rápidas e dados essenciais que requerem atenção constante dos administradores. Essa página foi projetada para facilitar a gestão e otimizar o acompanhamento das operações do lar.

Na página de visão geral, você encontrará:

Principais Dados:

- Total de Idosos: Exibe o número total de idosos atualmente cadastrados no lar.
- Medicamentos Mensais: Mostra o total de medicamentos distribuídas no mês.
- Relatórios Semanais: Apresenta a quantidade de relatórios semanais realizados.

Gráficos:

Os gráficos representam dados importantes que necessitam de atenção redobrada. Algumas ideias para esses gráficos são:

- Medicamentos: exibe os medicamentos mais utilizados e aplicados aos idosos.
- Consultas Externas: apresenta o número de idosos que estão em consultas médicas fora do lar.

Relatórios Recentes:

A seção de relatórios recentes apresenta os últimos relatórios gerados, contendo informações resumidas, como o nome dos idosos, os profissionais responsáveis, a data do atendimento e o horário em que ocorreu. Funciona como uma pré-visualização, permitindo que os administradores tenham uma visão rápida e clara dos dados antes de acessarem os relatórios completos. Além disso, essa seção oferece um botão de acesso rápido para facilitar a visualização dos relatórios detalhados.

Resumo:

A seção de resumo é uma área de visualização rápida, projetada para que os administradores tenham acesso a informações importantes de forma simples e eficiente. Nela, estão disponíveis dados principais que facilitam a gestão do lar, além de uma agenda do dia, que lista os compromissos programados, seus respectivos horários e endereços. Essa funcionalidade permite que os administradores planejem suas atividades com agilidade, garantindo uma melhor organização e acompanhamento das demandas diárias.

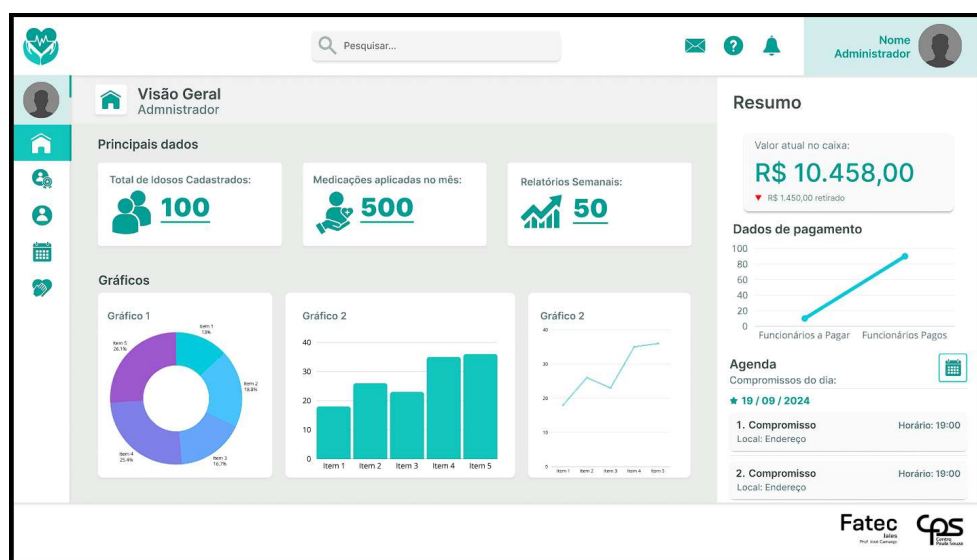
Calendário

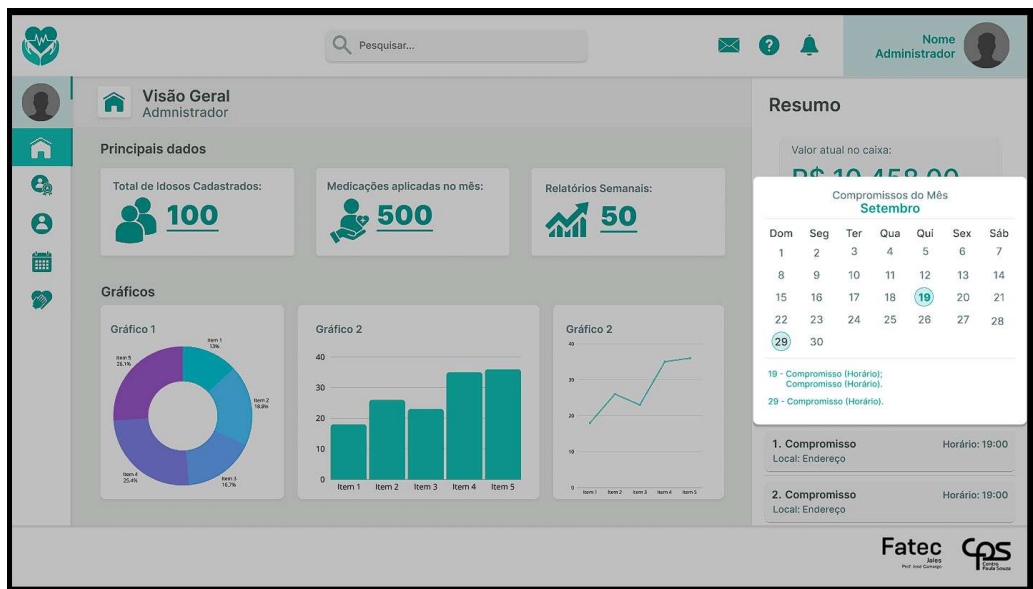
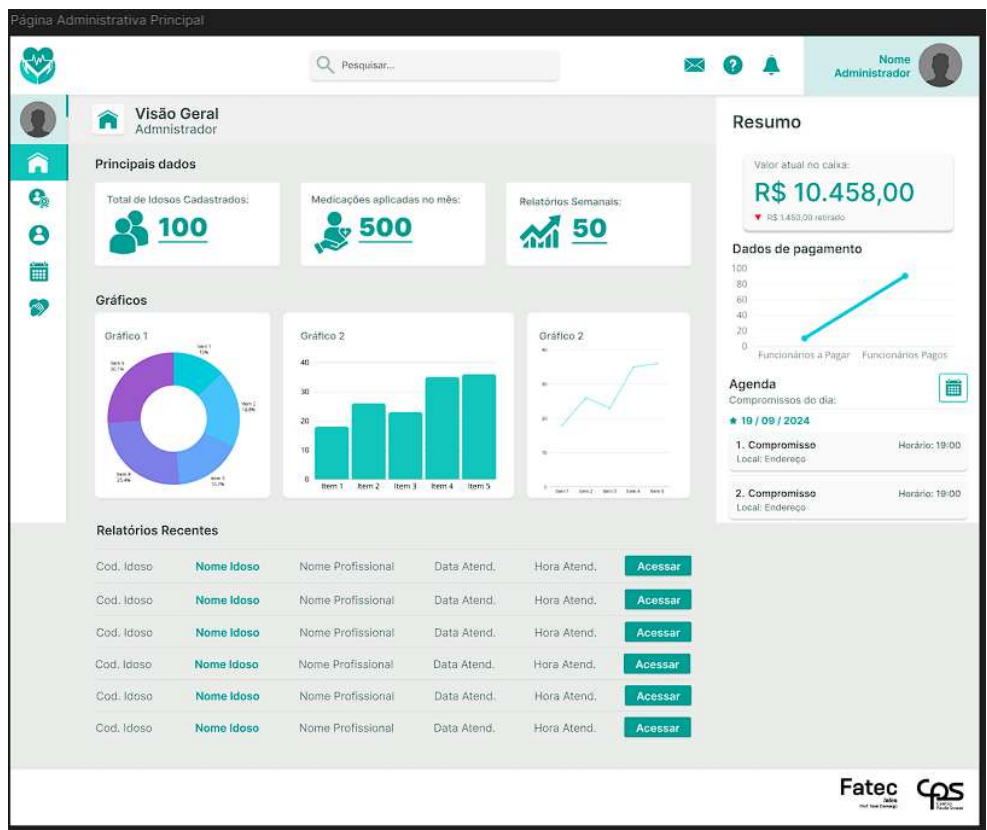
Ao clicar no ícone do calendário, um painel é exibido no lado direito da tela, proporcionando uma visualização rápida e intuitiva dos compromissos do mês.

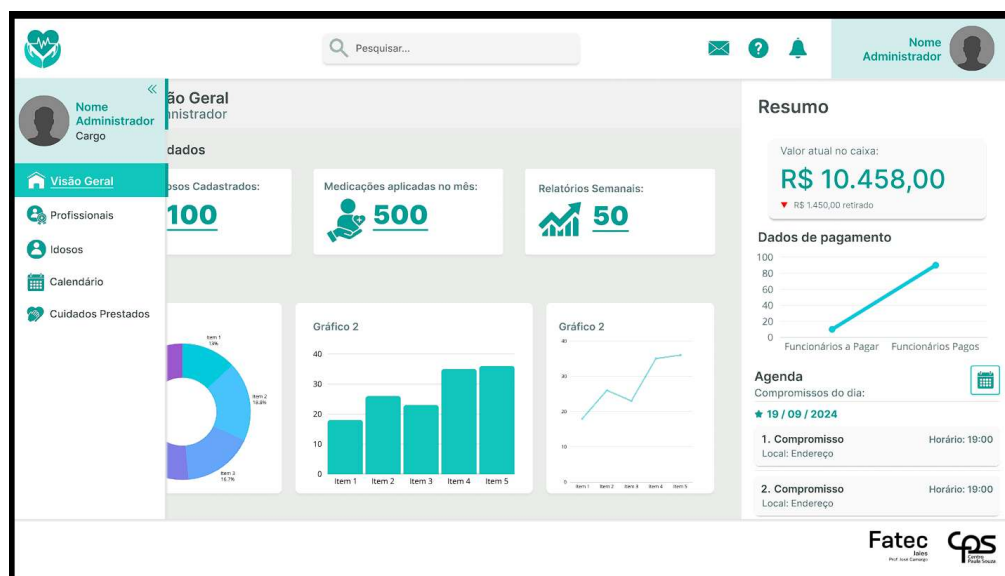
Funcionalidades:

- Visualização Compacta: Interagindo com o calendário, é possível consultar de forma compacta os compromissos e eventos agendados do mês, facilitando o acesso a datas importantes.
- Registro de Atividades: O calendário apresenta todos os compromissos e atividades programadas no lar no mês atual, permitindo um controle eficiente dos eventos diários, semanais e mensais.

A disposição e as funcionalidades do calendário garantem uma visualização clara e direta dos compromissos, contribuindo para uma gestão eficaz das atividades da instituição.







Foram concluídos os designs das seguintes telas do sistema: tela de abertura, tela de login, tela administrativa principal, calendário, além da implementação de um rodapé fixo e variações de componentes, como o menu lateral (sidebar). Essas telas foram desenvolvidas com foco na usabilidade e na identidade visual da plataforma, garantindo uma experiência intuitiva para o usuário.

No entanto, algumas funcionalidades importantes, como a adaptação da interface para dispositivos móveis, recursos de acessibilidade para daltônicos e interações mais complexas entre as páginas, ainda não foram implementadas. Essas características não foram priorizadas nesta sprint devido à limitação de tempo e ao foco nas funcionalidades essenciais para o lançamento inicial. A equipe reconhece a importância dessas melhorias e planeja abordá-las nas próximas iterações, visando oferecer uma experiência ainda mais completa e acessível a todos os usuários.

A intenção é garantir que, em futuras atualizações, o sistema não apenas mantenha sua eficiência, mas também se torne mais inclusivo e interativo, atendendo às necessidades de um público mais amplo.

Segue abaixo o link para o protótipo disponível no Figma:

<https://www.figma.com/design/uqFdrGZQXTRTliGP0vaWwd/Design-SeniorCareManager?node-id=0-1&t=wDDu974LSiiLILAs-1>

Task #3:

Nesta task, foram implementadas as telas responsivas do sistema, incluindo a tela inicial, a tela de login e a tela administrativa. O objetivo principal foi garantir que essas interfaces se adaptassem de forma eficiente para telas de celulares, proporcionando uma experiência de usuário fluida e agradável em diferentes tamanhos de tela.

Abertura do Sistema

Na página de abertura do sistema, diversos elementos estão dispostos para introduzir o usuário à plataforma.

5. Elementos Visuais

Logotipo do Site: O logotipo do sistema está destacado, reforçando a identidade visual da plataforma.

6. Informações e Funcionalidades

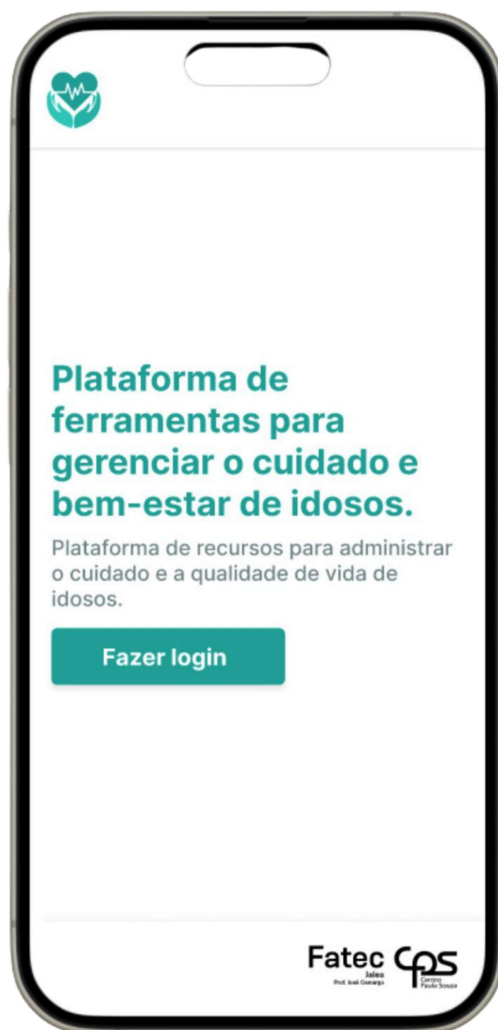
Descrição: Uma breve descrição esclarece o funcionamento do sistema e sua utilidade, destacando os principais benefícios para o usuário.

7. Botão de Login

Um botão direciona o usuário para a página de login, onde poderá acessar o sistema com facilidade.

8. Rodapé

Na parte inferior da página, está o logotipo da empresa desenvolvedora do site, evidenciando a autoria e o suporte técnico por trás da plataforma.



Página de Login

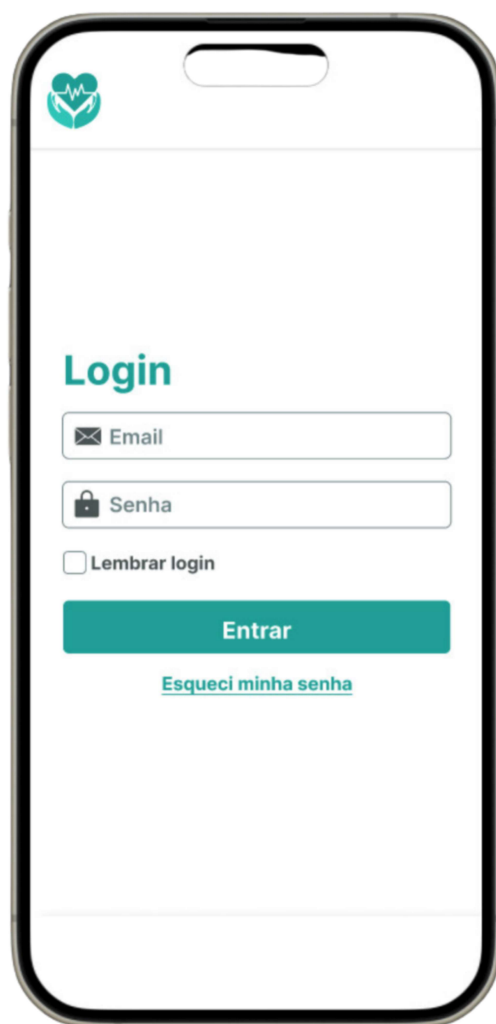
A página de login é composta por diversos elementos que facilitam o acesso ao sistema. Na parte superior, está o logotipo do site, que destaca a identidade visual da plataforma.

5. Campos de Acesso

A área de login inclui campos para inserção de e-mail e senha, permitindo que o usuário se autentique no sistema. Abaixo desses campos, há uma opção para lembrar o login, possibilitando que o usuário permaneça conectado em acessos futuros.

6. Botões e Links

Além dos campos de acesso, há um botão para realizar o login. Caso o usuário esqueça a senha, um link redireciona para a página de recuperação de senha, garantindo um processo simples de recuperação.



Página Administrativa Principal:

5. Visão do Administrador

A visão do administrador é composta por um menu com ícones que facilitam a navegação entre as funcionalidades principais do sistema. O primeiro ícone, representado por uma casa, direciona para a tela inicial. O segundo ícone exibe a lista de funcionários registrados. O terceiro ícone corresponde aos idosos cadastrados no lar. O quarto ícone é o calendário, onde estão registrados os compromissos e atividades agendadas. Por fim, o quinto ícone representa os cuidados prestados no lar.

6. Cabeçalho (Header)

O cabeçalho da página contém o logotipo do sistema.

7. Barra de Navegação

A barra de navegação está localizada na parte inferior da tela do aplicativo e oferece diversas funcionalidades. Um dos principais elementos é o ícone de lupa, que permite realizar buscas rápidas por ações ou informações específicas. Além disso, a barra inclui três ícones adicionais: o primeiro, representado por uma carta, exibe as mensagens recebidas; o segundo, simbolizado por um ponto de interrogação, serve para esclarecer dúvidas; e o terceiro, representado por um sino, notifica sobre a conclusão de serviços realizados.

8. Menu

Para abrir o menu de navegação, basta clicar no ícone de menu localizado no lado esquerdo de todas as páginas administrativas. Ao clicar no ícone, o menu se expande, exibindo as opções disponíveis para o usuário. As opções estão organizadas da seguinte maneira:

- **Tela Inicial (Ícone de Casa):** O primeiro ícone, representado por uma casa, leva o usuário à página inicial do sistema.
- **Funcionários:** O segundo ícone exibe uma lista com os funcionários registrados, permitindo o gerenciamento eficiente de suas informações.
- **Idosos:** O terceiro ícone dá acesso aos idosos cadastrados no lar, oferecendo informações detalhadas e históricos.
- **Calendário:** O quarto ícone apresenta o calendário, onde estão registrados os compromissos e atividades agendadas no lar.
- **Cuidados:** O quinto ícone representa os cuidados prestados no lar, contendo informações sobre os serviços realizados para os idosos.

9. Visão Geral

A página de visão geral é uma das opções disponíveis no menu e desempenha um papel crucial no monitoramento das atividades diárias. Nela, são apresentadas informações rápidas e dados essenciais que requerem atenção constante dos administradores. Essa página foi projetada para facilitar a gestão e otimizar o acompanhamento das operações do lar.

Na página de visão geral, você encontrará:

Principais Dados:

- **Total de Idosos:** Exibe o número total de idosos atualmente cadastrados no lar.
- **Medicações Mensais:** Mostra o total de medicações distribuídas no mês.
- **Relatórios Semanais:** Apresenta a quantidade de relatórios semanais realizados.

Gráficos:

Os gráficos representam dados importantes que necessitam de atenção redobrada. Algumas ideias para esses gráficos são:

- **Medicamentos:** exibe os medicamentos mais utilizados e aplicados aos idosos.
- **Consultas Externas:** apresenta o número de idosos que estão em consultas médicas fora do lar.

Relatórios Recentes:

A seção de relatórios recentes apresenta os últimos relatórios gerados, contendo informações resumidas, como o nome dos idosos, os profissionais responsáveis, a data do atendimento e o horário em que ocorreu. Funciona como uma pré-visualização, permitindo que os administradores tenham uma visão

rápida e clara dos dados antes de acessarem os relatórios completos. Além disso, essa seção oferece um botão de acesso rápido para facilitar a visualização dos relatórios detalhados.

Resumo:

A seção de resumo pode ser acessada através de um botão verde água, localizado no lado direito das páginas. Essa área foi projetada para proporcionar uma visualização rápida, permitindo que os administradores acessem informações cruciais de forma simples e eficiente. Nela, estão disponíveis dados essenciais que facilitam a gestão do lar, além de uma agenda do dia que lista os compromissos programados, incluindo horários e endereços. Essa funcionalidade capacita os administradores a planejar suas atividades com agilidade, garantindo uma organização eficaz e um acompanhamento preciso das demandas diárias.

Calendário

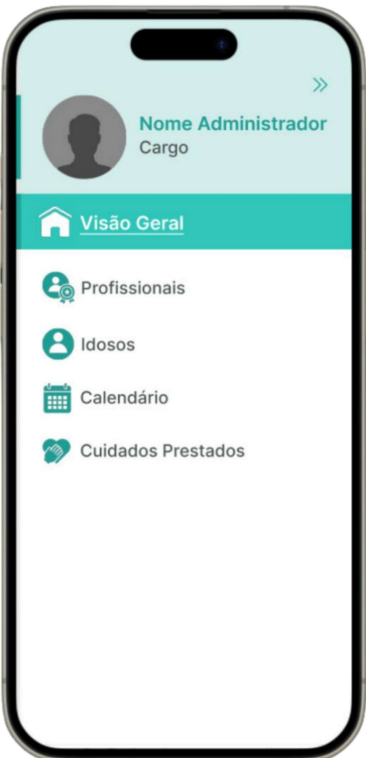
Ao clicar no ícone do calendário, um painel é exibido no lado direito da tela, proporcionando uma visualização rápida e intuitiva dos compromissos do mês.

Funcionalidades:

- **Visualização Compacta:** Interagindo com o calendário, é possível consultar de forma compacta os compromissos e eventos agendados do mês, facilitando o acesso a datas importantes.
- **Registro de Atividades:** O calendário apresenta todos os compromissos e atividades programadas no lar no mês atual, permitindo um controle eficiente dos eventos diários, semanais e mensais.

A disposição e as funcionalidades do calendário garantem uma visualização clara e direta dos compromissos, contribuindo para uma gestão eficaz das atividades da instituição.





Foram finalizados os designs responsivos das telas do sistema para dispositivos móveis, incluindo a tela de abertura, a tela de login, a tela administrativa principal e o calendário. Além disso, foi implementado um rodapé fixo e realizadas variações nos componentes, como o menu lateral (sidebar). Esses designs foram desenvolvidos com ênfase na usabilidade e na identidade visual da plataforma, assegurando uma experiência intuitiva e agradável para o usuário.

Segue abaixo o link para o protótipo disponível no Figma:

<https://www.figma.com/proto/U00Wjb4P7V2sa2wvqV0o8q/Untitled?node-id=6-87&node-type=canvas&t=0Hu2WrKOAu448hSx-0&scaling=scale-down&content-scaling=fixed&page-id=0%3A1>

Assinaturas

Documento assinado digitalmente
gov.br MARIA CLARA BORGES CARDOSO
Data: 26/05/2025 17:38:34-0300
Verifique em <https://validar.iti.gov.br>

Maria Clara Borges Cardoso

Orientador: Jefferson Antonio Ribeiro Passerini

7.1.2 Sprint 02 - Task 04



Faculdade de Tecnologia Professor José Camargo

Sprint Documentation		
Sprint #02	Start Date: 01/10/2024	Final Date: Final Date: 13/10/2024
Project Name:	SeniorCareManager-Frontend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Edson Pereira De Carvalho		
Lemuel Prates Sobrinho Ferreira		
Maria Clara Borges Cardoso		
Rodrigo Yoshida Lombezi		
Vinicius Lúcio Marcolino Da Silva		

Sprint Backlog			
Task#	Description	Start Date	Final date
4	Sprint 02 - Definir componentes de acessibilidade	01/10/2024	13/10/2024

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
4	Sprint 02 - Definir componentes de acessibilidade	Edson, Lemuel, Maria Clara, Rodrigo, Vinicius	Concluído	22	30

Sprint Description

Componentes de acessibilidade são elementos implementados no sistema para garantir que ele possa ser utilizado por todas as pessoas, incluindo aquelas com deficiências. Exemplos incluem: uso de textos alternativos (alt) em imagens, navegação por teclado, contraste adequado de cores e suporte a leitores de tela.

Sprint Results

Task #4:

A inclusão das pessoas com deficiência (PCD) é um tema de grande relevância na sociedade. Segundo a Organização Mundial da Saúde (OMS), aproximadamente 1 bilhão de pessoas em todo o mundo vive com algum tipo de deficiência, o que equivale a cerca de 1 em cada 7 indivíduos. De acordo com a Convenção Internacional sobre os Direitos das Pessoas com Deficiência, assinada em Nova York em 2007 e ratificada no Brasil pelo Decreto nº 6.949 de 25 de agosto de 2009, pessoas com deficiência são aquelas que enfrentam impedimentos de longa duração, sejam eles físicos, mentais, intelectuais ou sensoriais, os quais, em interação com barreiras diversas, podem obstruir sua plena participação na sociedade.

Para combater o preconceito e promover a igualdade de oportunidades, é fundamental que haja adaptações nos espaços físicos, adequação da linguagem, adoção de tecnologias inclusivas e promoção da inclusão no mercado de trabalho. Essas ações são essenciais para garantir que as PCDs possam viver de forma independente e participar plenamente da vida social.

A promoção da acessibilidade implica assegurar que as pessoas com deficiência tenham acesso igualitário ao meio físico, ao transporte, à informação e comunicação, incluindo os sistemas e tecnologias da informação, além de outros serviços e instalações disponíveis ao público. Apesar dos direitos garantidos

pela “Declaração Universal dos Direitos Humanos”, pela Organização das Nações Unidas e pela OMS, ainda existem numerosas barreiras à efetivação desses direitos. O engajamento da sociedade é crucial para a eliminação de obstáculos e formas de discriminação, sendo dever de todos buscar um mundo mais justo, onde a inclusão das PCDs ocorra sem demérito ou preconceito.

A acessibilidade digital refere-se à eliminação de barreiras na web, garantindo que sites e portais sejam projetados para que todas as pessoas possam perceber, entender, navegar e interagir de forma eficaz com as páginas. O objetivo é democratizar o acesso e assegurar que todos, independentemente de suas capacidades físico-motoras, perceptivas, culturais e sociais, possam compreender e controlar a navegação nos conteúdos e serviços do governo.

As barreiras digitais impactam significativamente as pessoas com deficiência. Um estudo do Censo IBGE de 2010 revelou que aproximadamente 45 milhões de brasileiros apresentam pelo menos uma deficiência, representando 23,9% da população. Este cenário destaca a necessidade urgente de um ambiente digital inclusivo. A implementação da acessibilidade digital não apenas democratiza o acesso, mas também promove a inclusão social e digital, permitindo que as pessoas com deficiência utilizem a internet para estudar, trabalhar, realizar compras e compartilhar experiências, tudo sem a necessidade de deslocamento.

Além disso, um site acessível é mais facilmente indexado por mecanismos de busca e se torna compatível com uma gama maior de aplicativos, facilitando a navegação. Essa acessibilidade beneficia não apenas as pessoas com deficiência, mas também idosos e outros usuários que podem ter dificuldades em utilizar a internet, como aqueles que não possuem habilidades digitais avançadas ou que utilizam dispositivos móveis. Portanto, a promoção da acessibilidade digital é uma questão de justiça social que visa garantir que todos tenham a oportunidade de participar plenamente da sociedade moderna.

Práticas de Acessibilidade Implementadas

Após um estudo detalhado das práticas de acessibilidade recomendadas para sistemas web, foram aplicadas diretrizes que garantem uma experiência inclusiva para todos os usuários. Essas práticas seguem as recomendações das *Web Content Accessibility Guidelines* (WCAG) e incluem medidas que melhoram a navegação e usabilidade para pessoas com diferentes tipos de deficiência. A seguir, cada uma dessas práticas será explicada em detalhes.

1. **Textos Alternativos para Imagens:** As imagens no sistema terão descrições textuais, permitindo que leitores de tela descrevam o conteúdo visual para pessoas com deficiência visual, facilitando o entendimento do propósito de cada imagem.
2. **Teclas de Atalho:** Serão implementados atalhos de teclado para permitir que os usuários naveguem rapidamente entre seções do sistema sem a necessidade de usar o mouse. Isso facilita a navegação para pessoas com dificuldades motoras. As teclas de atalho definidas para o sistema, juntamente com suas funções específicas, são as seguintes:

Tabela 1 - Teclas de Atalho

Teclas	Função
Alt + Z	Desfazer a última ação.
Alt + Y	Refazer a ação desfeita.
Alt + T	Selecionar tudo.
Alt + G	Abrir a caixa de busca para encontrar texto.
Alt + D	Salvar o documento ou projeto atual.
Alt + W	Alternar entre janelas abertas no sistema.
Ctrl + → (Seta Direita)	Navegar entre elementos interativos (botões, links, campos de formulário).
Ctrl + ← (Seta Esquerda)	Navegar para o elemento anterior.
Alt + X	Fechar janelas modais ou sair de menus.
Alt + P	Imprimir o documento atual.
Alt + N	Abrir uma nova janela ou documento.

F2	Abrir a ajuda do aplicativo ou sistema.
Alt + C	Fechar a janela ativa.
Alt + U	Ir diretamente ao topo da página em qualquer página do portal.

Suas principais importâncias incluem:

- **Alt + Z (Desfazer a última ação):** Permite que o usuário desfça uma ação recente, como a alteração acidental de dados críticos dos pacientes ou ajustes nos registros de atendimento. Isso é essencial para garantir que erros possam ser corrigidos rapidamente.
- **Alt + Y (Refazer a ação desfeita):** Permite refazer uma ação desfeita, garantindo que, se uma correção for aplicada por engano, ela possa ser revertida com facilidade. Isso é importante para evitar perda de dados em processos sensíveis de cuidado e monitoramento.
- **Alt + T (Selecionar tudo):** Facilita a seleção de grandes volumes de texto ou dados em formulários e registros. Isso é útil ao lidar com relatórios detalhados ou ao atualizar várias informações dos pacientes de uma só vez.
- **Alt + G (Abrir a caixa de busca para encontrar texto):** A busca rápida por nomes, prontuários ou informações específicas dentro do sistema é fundamental para profissionais de saúde que precisam acessar dados com agilidade em situações de cuidado emergencial ou monitoramento contínuo.
- **Alt + D (Salvar o documento ou projeto atual):** A função de salvar rapidamente é vital para garantir que alterações importantes no histórico de saúde, prescrições ou agendamentos sejam registradas sem risco de perda de informações, essencial em sistemas de monitoramento de saúde.
- **Alt + W (Alternar entre janelas abertas no sistema):** Permite que os cuidadores alternem rapidamente entre diferentes seções do sistema, como prontuários médicos, agendas de medicação ou relatórios de saúde, facilitando a multitarefa e o acesso rápido a dados em tempo real.
- **Ctrl + → (Navegar entre elementos interativos):** Ajuda na navegação fluida entre campos e botões, agilizando o preenchimento de formulários ou o acesso a diferentes funcionalidades. Isso é importante para os profissionais que precisam registrar informações com rapidez e eficiência.
- **Ctrl + ← (Navegar para o elemento anterior):** Facilita o retorno a elementos anteriores na interface, permitindo a correção de dados ou a verificação de informações sem a necessidade de uso do mouse, o que é útil em ambientes que exigem foco no atendimento.
- **Alt + X (Fechar janelas modais ou sair de menus):** Fecha rapidamente pop-ups ou janelas modais, permitindo que o usuário volte rapidamente ao fluxo principal do sistema. Isso é importante em situações onde o acesso a informações imediatas é crucial, como emergências ou alertas.
- **Alt + P (Imprimir o documento atual):** Permite a impressão rápida de relatórios médicos, prescrições ou planos de cuidado, facilitando a comunicação entre diferentes setores e garantindo que os cuidadores tenham documentos físicos para referência.
- **Alt + N (Abrir uma nova janela ou documento):** Facilita a abertura de novos registros de pacientes ou documentos sem fechar a sessão atual. Isso é útil para lidar com múltiplos residentes ao mesmo tempo, especialmente em ambientes de cuidado coletivo.
- **F2 (Abrir a ajuda do aplicativo ou sistema):** Acesso rápido a ajuda e suporte técnico dentro do sistema, o que é fundamental para os profissionais que possam precisar de orientação sobre como usar funções específicas ou resolver problemas durante o uso.
- **Alt + C (Fechar a janela ativa):** Fecha a janela atual sem a necessidade de uso do mouse, agilizando o trabalho e permitindo que o profissional de saúde ou cuidador se concentre em outras tarefas sem interrupções no fluxo de trabalho.

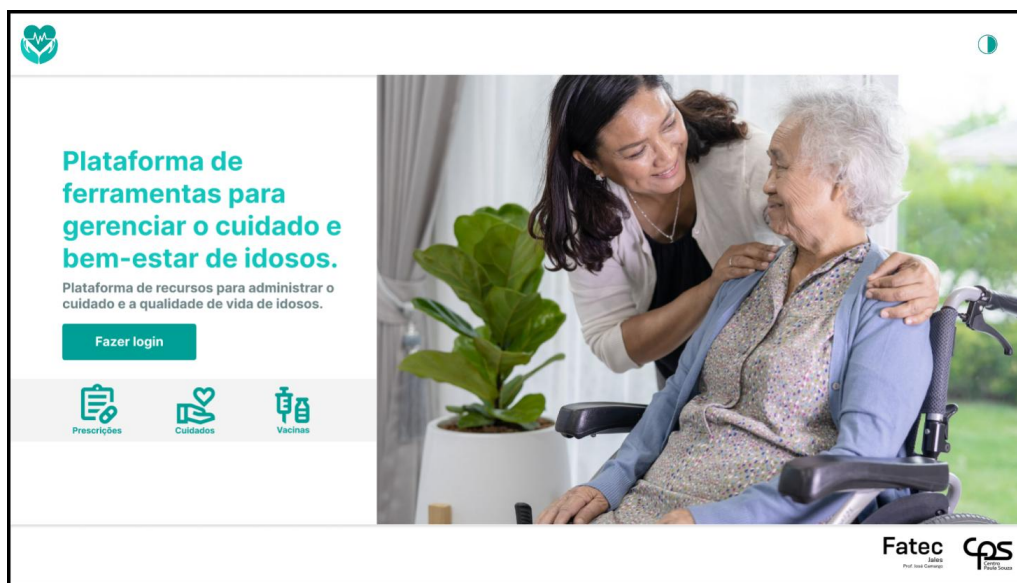
- **Alt + U (Ir diretamente ao topo da página):** Leva o usuário ao topo da página, facilitando a navegação em relatórios longos ou seções extensas do sistema, garantindo que as informações principais estejam sempre acessíveis rapidamente.
3. **Navegação Semântica:** A estrutura do conteúdo será organizada de forma lógica, utilizando elementos adequados para descrever cada parte da página. Isso melhora a navegação para quem usa leitores de tela, ajudando-os a entender a hierarquia do conteúdo.
 4. **Contraste de Cores:** Será assegurado um contraste adequado entre texto e fundo para facilitar a leitura, especialmente para pessoas com baixa visão ou daltonismo. Além disso, haverá a opção de alto contraste, permitindo que o sistema ajuste automaticamente as cores para melhorar ainda mais a visibilidade do conteúdo. Isso garantirá que o conteúdo seja visível e legível por todos.

Utilizar as cores preto, branco e cinza para criar alto contraste é uma estratégia eficaz em design de interfaces, especialmente para atender pessoas com baixa visão ou daltonismo. O alto contraste entre preto e branco facilita a leitura e a identificação de elementos, enquanto o cinza serve como uma cor intermediária que ajuda a equilibrar o layout e reduzir a sobrecarga visual. Essa combinação proporciona clareza e simplicidade, melhorando a navegação e permitindo que os usuários se familiarizem rapidamente com o sistema. Além disso, a paleta neutra é facilmente adaptável, garantindo que o alto contraste possa ser aplicado de forma flexível para atender às necessidades individuais dos usuários, assim como mostrado abaixo.

Figura 1 - Tela Inicial Desktop (Alto Contraste)



Figura 2 - Tela Inicial Desktop



Para ativar o alto contraste, basta clicar no ícone posicionado no canto superior direito de todas as telas do sistema. Essa opção, facilmente acessível, permite que os usuários ajustem rapidamente as configurações de contraste para melhorar a legibilidade do conteúdo. Ao selecionar essa funcionalidade, o sistema aplica automaticamente uma paleta de cores com alto contraste, garantindo que textos e elementos visuais sejam claramente visíveis.

Figura 3 - Tela Login Desktop (Alto Contraste)



Figura 4 - Tela Login Desktop

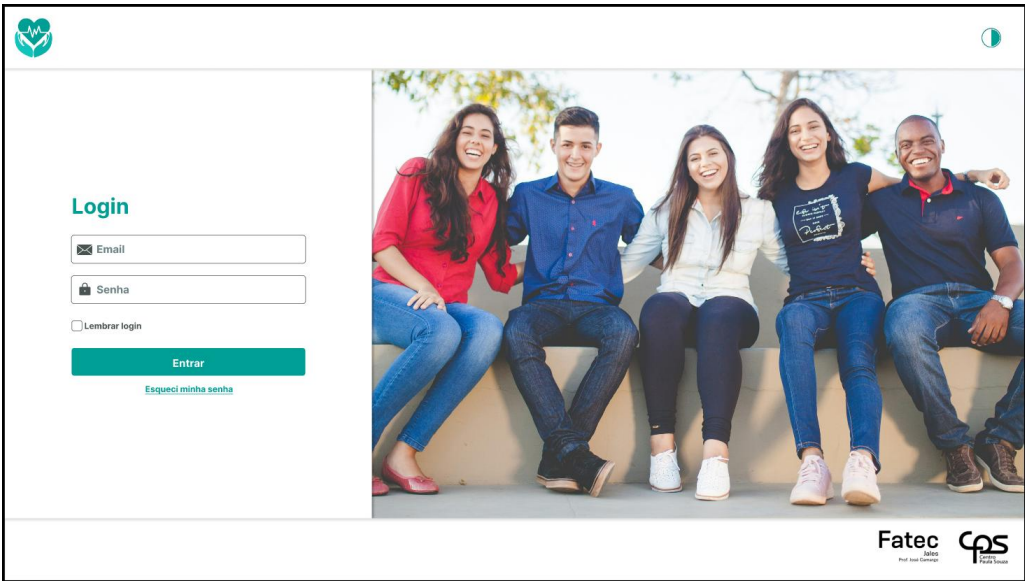


Figura 5 - Tela Principal Administrativa Desktop (Alto Contraste)

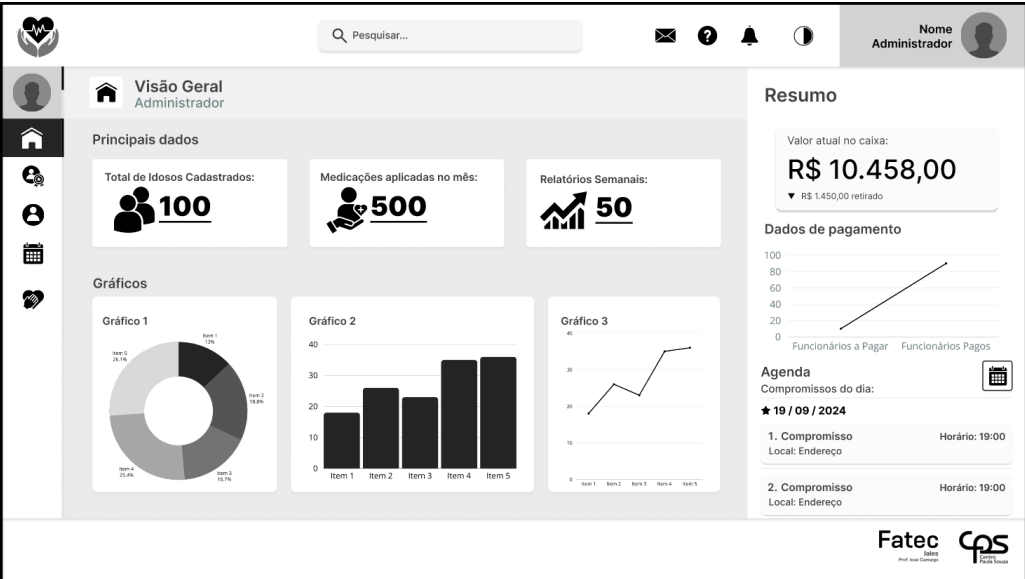


Figura 6 - Tela Principal Administrativa Desktop

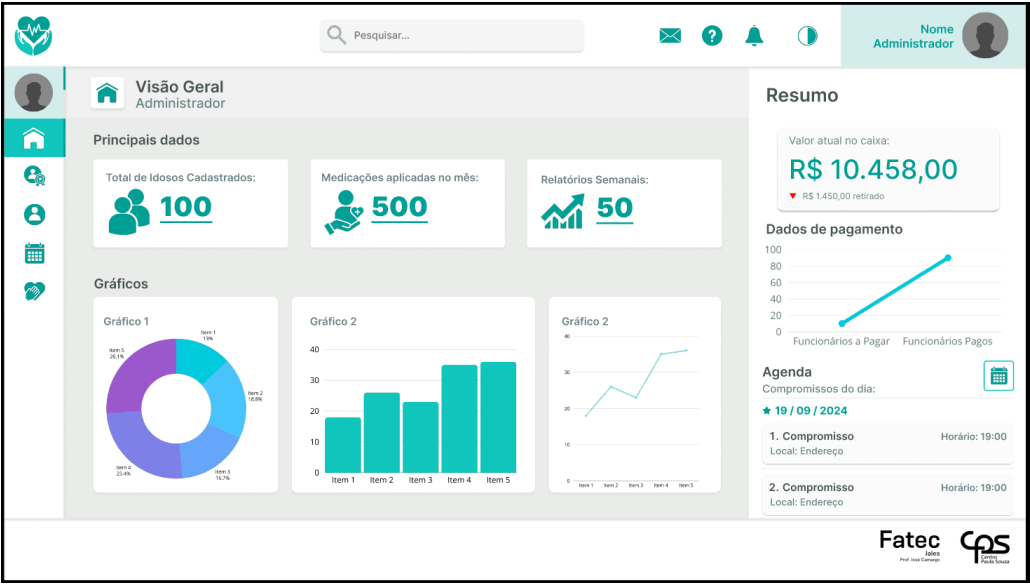


Figura 7 - Menu Desktop (Alto Contraste)

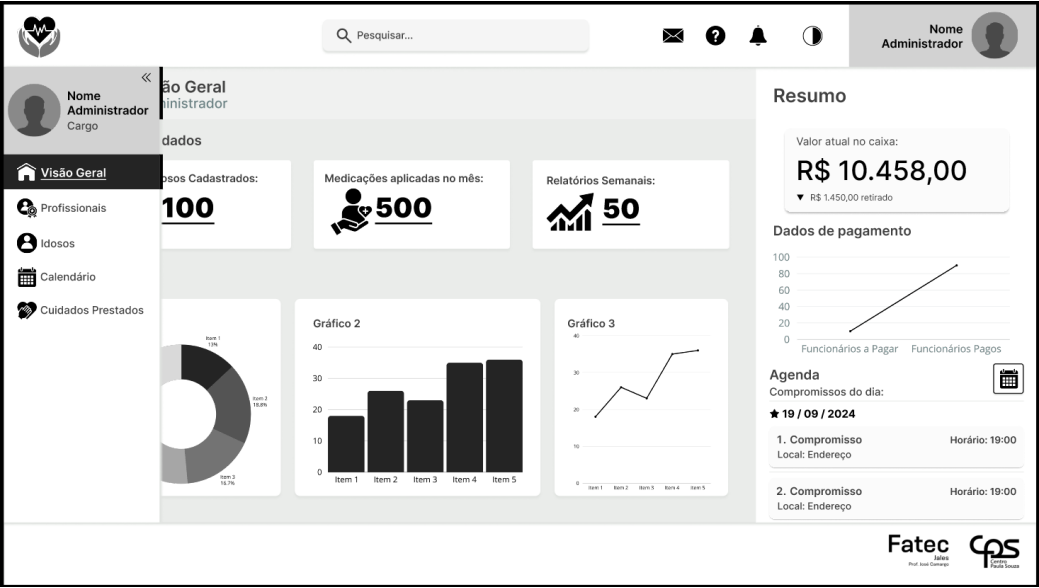


Figura 8 - Menu Desktop

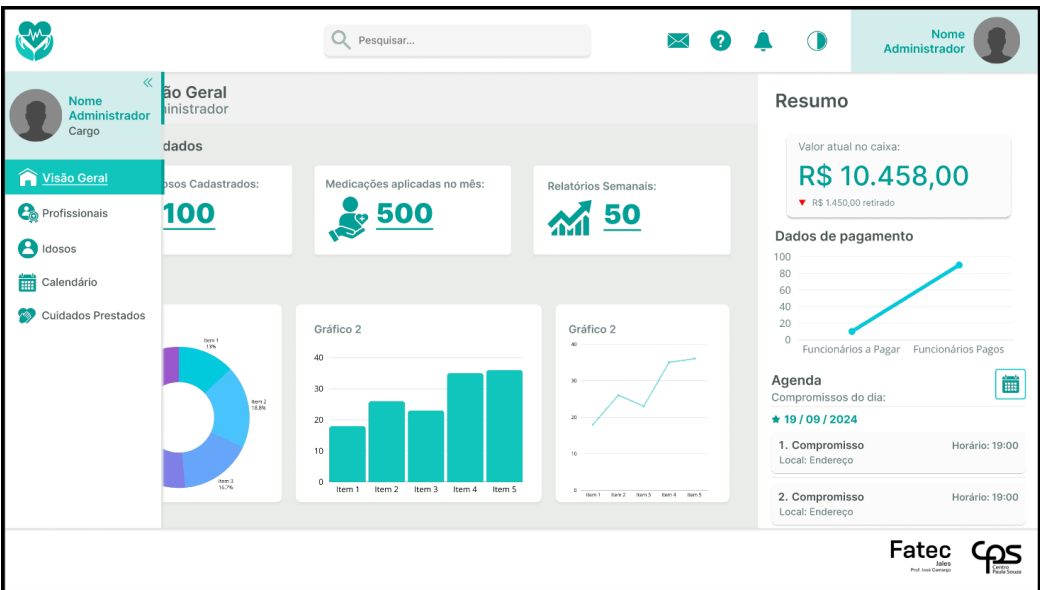


Figura 9 - Calendário Desktop (Alto Contraste)

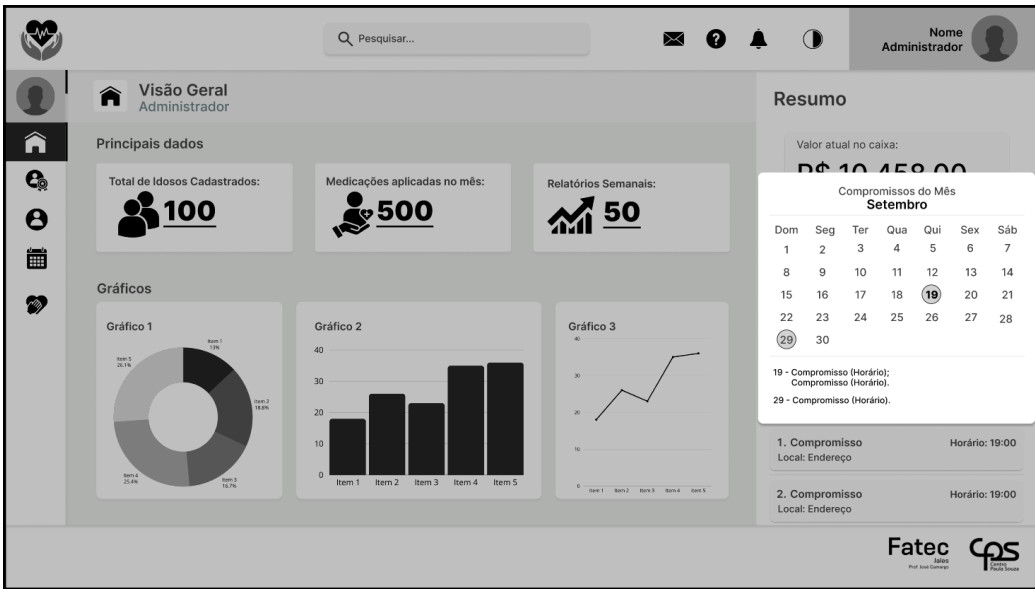
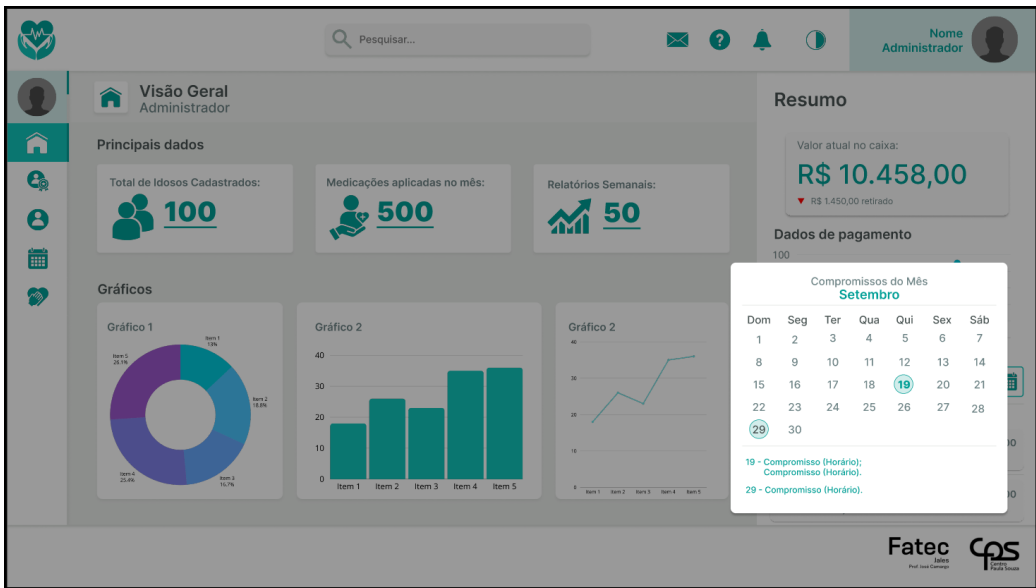


Figura 10 - Calendário Desktop



A funcionalidade de alto contraste também está disponível para dispositivos móveis, garantindo uma experiência de uso acessível e legível em todas as plataformas.

Figura 11 - Tela Inicial Mobile (Alto Contraste)



Figura 12 - Tela Inicial Mobile



Figura 13 - Tela Login Mobile (Alto Contraste)



Figura 14 - Tela Login Mobile

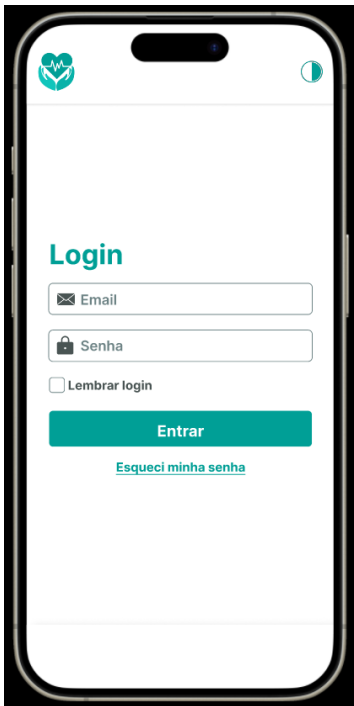


Figura 15 - Tela Principal Administrativa Mobile (Alto Contraste)



Figura 16 - Tela Principal Administrativa Mobile



Figura 17 - Menu Mobile (Alto Contraste)



Figura 18 - Menu Mobile



Figura 19 - Resumo (Alto Contraste)



Figura 20 - Resumo



Figura 21 - Calendário Mobile (Alto Contraste)



Figura 22 - Calendário Mobile



5. **Texto Redimensionável:** Os usuários poderão ajustar o tamanho do texto conforme necessário, sem que isso prejudique a disposição do conteúdo na página. Essa flexibilidade é importante para pessoas com dificuldade de leitura. Segue um exemplo de texto redimensionável:

Figura 23 - Tela Inicial Desktop (Texto Redimensionável)



Figura 24 - Tela Login Desktop (Texto Redimensionável)

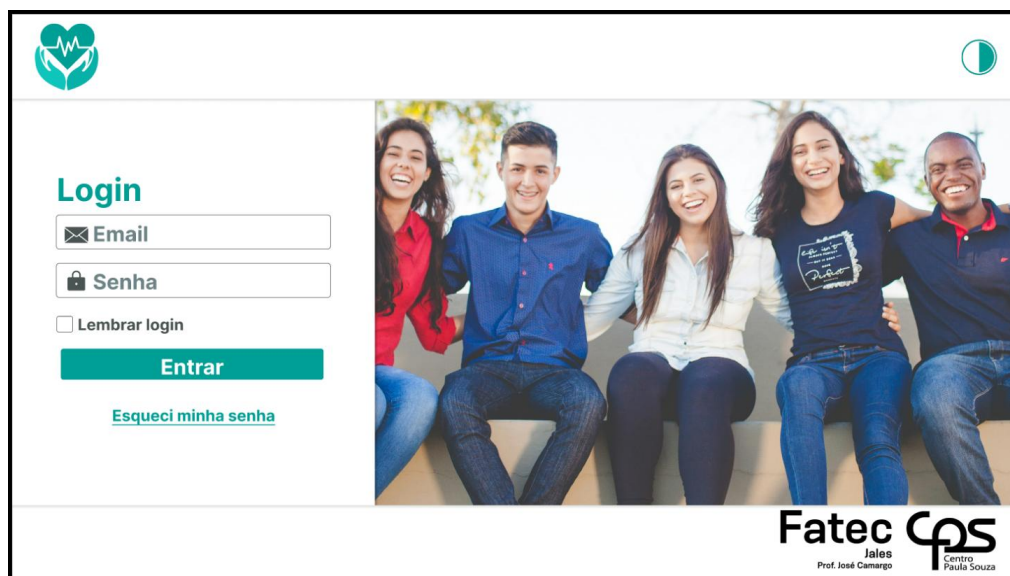


Figura 25 - Tela Principal Administrativa Desktop (Texto Redimensionável)



Figura 26 - Menu Desktop (Texto Redimensionável)



Figura 27 - Calendário Desktop (Texto Redimensionável)



A funcionalidade de texto redimensionável também está disponível para dispositivos móveis, garantindo uma experiência de uso acessível e legível em todas as plataformas.

Figura 28 - Tela Inicial Mobile (Texto Redimensionável)



Figura 29 - Tela Inicial Mobile



Figura 30 - Tela Administrativa Principal Mobile (Texto Redimensionável)



Figura 31 - Menu Mobile (Texto Redimensionável)



Figura 32 - Resumo (Texto Redimensionável)



Figura 33 - Calendário (Texto Redimensionável)



6. **Compatibilidade com Leitores de Tela:** O sistema será otimizado para funcionar de forma eficiente com leitores de tela, garantindo que todos os elementos, como botões e menus, sejam compreendidos e acessíveis para pessoas com deficiência visual.
7. **Foco Visível:** Os elementos interativos, como botões e links, terão um destaque visual quando selecionados por navegação via teclado. Isso ajudará os usuários a identificarem facilmente onde estão interagindo na página.
8. **Evitar Autoplay:** Não será reproduzido automaticamente nenhum vídeo ou áudio. O usuário terá controle total sobre a reprodução de mídias, evitando distrações e ajudando quem utiliza tecnologias assistivas.
9. **Facilidade de Fechamento:** Modais e pop-ups poderão ser fechados de maneira simples, tanto visualmente quanto pelo teclado, garantindo que os usuários não fiquem presos em janelas que possam prejudicar a navegação.
10. **Suporte a Línguas:** A linguagem principal da página será definida corretamente para que tradutores automáticos e leitores de tela interpretem o conteúdo de forma precisa, oferecendo uma leitura e pronúncia corretas.
11. **Breadcrumbs (Migalhas de Pão):** Um sistema de navegação será implementado para mostrar o caminho percorrido pelo usuário até a página atual, facilitando o entendimento da localização dentro do site.
12. **Conteúdo Multimídia Acessível:** Todos os vídeos terão legendas e o conteúdo de áudio será acompanhado de transcrições. Isso garantirá que pessoas com deficiência auditiva possam acessar todas as informações multimídia.
13. **Formatação Clara e Consistente:** O layout do sistema será claro e consistente, utilizando espaçamento adequado e organização de conteúdo de fácil compreensão. Isso beneficiará todos os usuários, especialmente aqueles com deficiência cognitiva.

14. **Feedback Acessível:** As mensagens de erro e feedbacks serão apresentados de forma clara e fácil de entender, utilizando textos explicativos e avisos visuais. Isso garantirá que os usuários saibam como corrigir eventuais problemas.
15. **Desempenho em Dispositivos Móveis:** O sistema será adaptado para funcionar perfeitamente em dispositivos móveis, mantendo as práticas de acessibilidade e garantindo que usuários em celulares ou tablets tenham a mesma experiência inclusiva que em desktops.

Links do Figma para acessar todas as telas mencionadas acima:

[Protótipo Desktop – Texto Redimensionável](#)

[Protótipo Desktop - Alto contraste](#)

[Protótipo Mobile – Texto Redimensionável](#)

[Protótipo Mobile - Alto contraste](#)

Conclusão

Será implementado no sistema diversas práticas de acessibilidade para garantir uma experiência inclusiva, incluindo textos alternativos para imagens, teclas de atalho para facilitar a navegação sem mouse, e navegação semântica para melhorar a compreensão por leitores de tela. Além disso, será assegurado um contraste de cores adequado, opções de alto contraste e texto redimensionável, tudo para atender usuários com baixa visão. A compatibilidade com leitores de tela será otimizada, e elementos interativos irão possuir foco visível para facilitar a navegação via teclado. O sistema também oferecerá facilidade no fechamento de modais, suporte a múltiplas línguas, breadcrumbs para navegação, conteúdo multimídia acessível com legendas e transcrições, formatação clara e consistente do layout, feedback acessível e desempenho otimizado para dispositivos móveis, garantindo que todos os usuários possam acessar e interagir com o sistema de forma eficaz e inclusiva.

Assinaturas

Edson Pereira De Carvalho

Lemuel Prates Sobrinho Ferreira

Maria Clara Borges Cardoso

Rodrigo Yoshida Lombezzi

Vinicius Lúcio Marcolino Da Silva

Orientador: Jefferson Antonio Ribeiro Passerini

7.1.3 Sprint 03 - Task 15



Faculdade de Tecnologia Professor José Camargo

Sprint Documentation		
Sprint #03	Start Date: 21/10/2024	Final Date: 26/10/2024
Project Name:	SeniorCareManager-Frontend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Edson Pereira De Carvalho		
Lemuel Prates Sobrinho Ferreira		
Maria Clara Borges Cardoso		
Rodrigo Yoshida Lombezi		
Vinicius Lúcio Marcolino Da Silva		

Sprint Backlog			
Task#	Description	Start Date	Final date
15	Sprint 03 - Criar projeto front-end React	22/10/2024	26/10/2024

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
15	Sprint 03 - Criar projeto front-end React	Lemuel, Maria Clara	Concluído	2	2

Sprint Description

Durante a Sprint 03, o projeto SeniorCareManagerFrontend foi criado com o Vite, uma poderosa ferramenta que facilita a construção de projetos JavaScript modernos de maneira rápida e eficiente. Abaixo, encontra-se uma tabela detalhada com os comandos utilizados na configuração inicial do projeto, proporcionando uma visão clara do processo.

Sprint Results

Task #15:

Comando Usado	Função
npm create vite@latest	Inicializa um novo projeto utilizando o Vite, uma ferramenta rápida para construção de projetos JavaScript modernos. O @latest garante que a versão mais recente seja usada.

<code>cd .\SeniorCareManagerFrontend\</code>	Navega até o diretório do projeto criado chamado SeniorCareManagerFrontend. No Windows, o caminho é precedido por <code>.\</code> para acessar o diretório.
<code>npm install</code>	Instala todas as dependências listadas no arquivo <code>package.json</code> . Isso inclui o Vite e outras dependências necessárias para o funcionamento do projeto.
<code>npm install -D tailwindcss postcss autoprefixer</code>	Instala o Tailwind CSS, PostCSS e Autoprefixer como dependências de desenvolvimento. Utilizado para configurar um ambiente CSS moderno. (-D indica que são usadas apenas no desenvolvimento).
<code>npx tailwindcss init -p</code>	Inicializa a configuração do Tailwind CSS, criando os arquivos <code>tailwind.config.js</code> (configuração do Tailwind) e <code>postcss.config.js</code> (configuração do PostCSS).

Assinaturas

gov.br Documento assinado digitalmente
LEMUEL PRATES SOBRINHO FERREIRA
Data: 28/05/2025 14:31:10-0300
Verifique em <https://validar.iti.gov.br>

Lemuel Prates Sobrinho Ferreira

gov.br Documento assinado digitalmente
MARIA CLARA BORGES CARDOSO
Data: 26/05/2025 17:38:34-0300
Verifique em <https://validar.iti.gov.br>

Maria Clara Borges Cardoso

Orientador: Jefferson Antonio Ribeiro Passerini

7.1.4 Sprint 04 - Task 16 e 20



Faculdade de Tecnologia Professor José Camargo

Sprint Documentation		
Sprint #4	Start Date: 27/10/2024	Final Date: 09/11/2024
Project Name:	SeniorCareManager-Frontend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Edson Pereira De Carvalho		
Lemuel Prates Sobrinho Ferreira		
Maria Clara Borges Cardoso		
Rodrigo Yoshida Lombezzi		
Vinicius Lúcio Marcolino Da Silva		

Sprint Backlog			
Task#	Description	Start Date	Final date
16	Sprint 04 - Desenvolver tela de entrada do sistema	01/11/2024	09/11/2024
20	Sprint 04 - Definir padronização de configurações de acessibilidade e interface	27/10/2024	01/11/2024

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
16	Sprint 04 - Desenvolver tela de entrada do sistema	Maria Clara, Lemuel	Concluído	7	7
20	Sprint 04 - Definir padronização de configurações de acessibilidade e interface	Maria Clara, Lemuel	Concluído	12	9

Sprint Description

Na Sprint 04, o foco foi aprimorar a experiência do usuário, com a implementação de recursos fundamentais para a acessibilidade e a interface do sistema. A equipe finalizou a criação da tela de entrada, que agora serve como o ponto de acesso inicial para os usuários, sendo projetada de forma intuitiva e atraente, alinhada ao design geral do sistema. Além disso, foram concluídas as melhorias na acessibilidade, com a padronização das configurações visuais, incluindo a introdução do modo de alto contraste e a opção de ajuste de tamanho de fonte. Essas ações garantem uma experiência inclusiva e confortável, atendendo às necessidades de todos os usuários e promovendo maior usabilidade.

Sprint Results

Task #16:

A tela de entrada foi desenvolvida para proporcionar uma introdução clara ao sistema, enfatizando sua finalidade de gerenciar o cuidado e bem-estar de idosos. O layout foi aprimorado para garantir uma apresentação coesa e harmoniosa com o restante do sistema, incluindo um título, uma descrição informativa e um botão de login estilizado que incentiva o usuário a iniciar a sessão. Uma imagem ilustrativa ocupa 60% da largura da tela e alterna entre tons de cinza e normal, conforme o tema selecionado (alto contraste ou não).

Para manter a identidade visual consistente, foi instalada a fonte “Inter”, que será utilizada em todo o sistema. Essa fonte foi escolhida para melhorar a legibilidade e proporcionar uma experiência de leitura agradável.

Além disso, o layout padrão do sistema foi reconfigurado para ajustar o tamanho das páginas, assegurando uma apresentação visual consistente em diferentes dispositivos e promovendo uma experiência de navegação fluida. Sombras foram adicionadas ao cabeçalho e ao rodapé para proporcionar maior profundidade visual e modernizar a apresentação desses elementos. Ajustes significativos foram feitos no componente de layout, garantindo que a proporção visual da tela esteja em conformidade com os padrões estabelecidos.

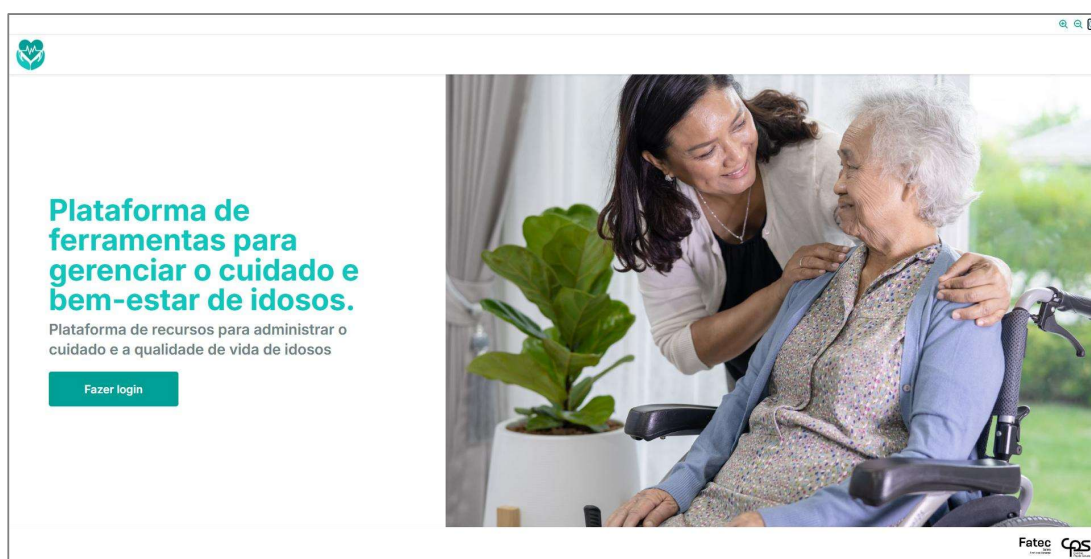


Figura 1 - Tela de abertura do Sistema

- Diagrama de Caso de Uso – Tela de Abertura do Sistema:

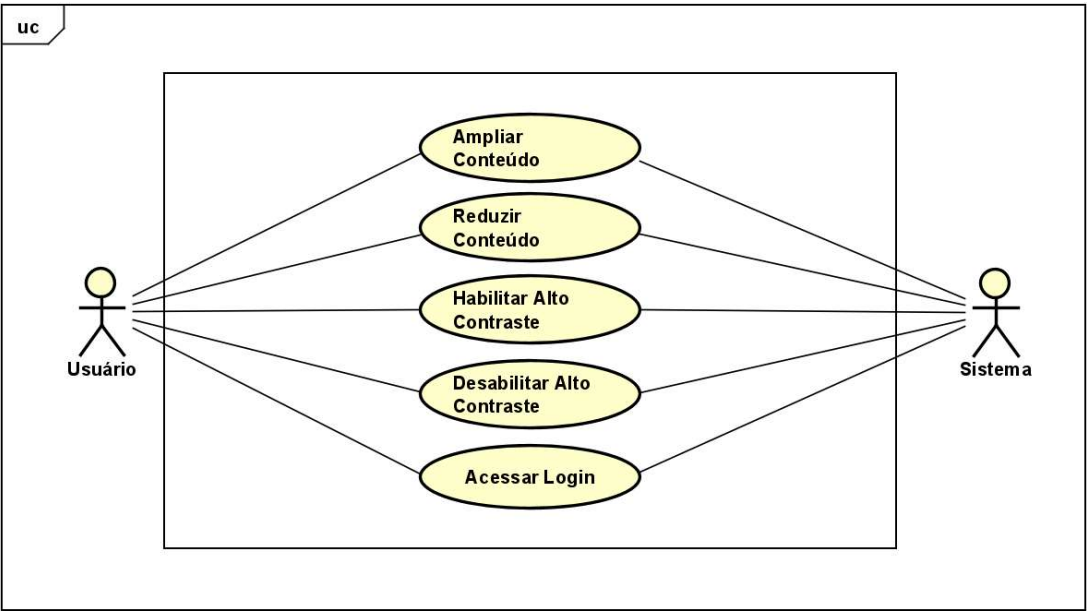


Figura 2 - Diagrama de Caso de Uso: Tela de Abertura do Sistema

Ator Usuário: o **Usuário** representa qualquer pessoa que interage com a tela de entrada do sistema. Este ator tem acesso às funcionalidades de acessibilidade (como ajuste de zoom e modo de alto contraste) e pode iniciar a sessão para acessar as funcionalidades internas do sistema.

Ator Sistema: o Sistema é o ator responsável por processar e responder às solicitações do Usuário, garantindo a funcionalidade das interações na tela de entrada. Ele executa as operações de ampliação e redução de zoom, ativa e desativa o modo de alto contraste conforme configurado e redireciona o Usuário para a tela de login ao iniciar a sessão. O Sistema assegura que todas as funcionalidades de acessibilidade e navegação sejam executadas corretamente e que eventuais erros sejam comunicados de maneira adequada ao Usuário.

Tabela 1 - Documentação Caso de Uso - Ampliar Conteúdo

Nome do Caso de Uso	UC01 – Ampliar Conteúdo
Caso de Uso Geral	
Ator Principal	Usuário
Ator Secundário	Sistema
Resumo	Esse caso de uso descreve as etapas percorridas por um usuário, com auxílio do sistema, para realizar a ampliação do conteúdo visual.
Ações do Ator	Ações do sistema
1. O usuário clica no botão de ampliação de zoom.	
	2. O sistema aumenta o tamanho do conteúdo, conforme solicitado.
Restrições/Validações	
Cenário Alternativo – Ampliar Conteúdo	
Ações do Ator	Ações do sistema
	2.1. Caso o zoom atinja o nível máximo suportado, o sistema não responde mais a ação.
Cenário de Exceção	
Ações do Ator	Ações do sistema

--	--

Tabela 2 - Documentação Caso de Uso - Reduzir Conteúdo

Nome do Caso de Uso	UC02 – Reduzir Conteúdo
Caso de Uso Geral	
Ator Principal	Usuário
Ator Secundário	Sistema
Resumo	Esse caso de uso descreve as etapas percorridas por um usuário, com auxílio do sistema, para realizar a redução do conteúdo visual.
Ações do Ator	Ações do sistema
1. O usuário clica no botão de redução de zoom.	
	2. O sistema reduz o tamanho do conteúdo, conforme solicitado.
Restrições/Validações	
Cenário Alternativo – Ampliar Conteúdo	
Ações do Ator	Ações do sistema
	2.1. Caso o zoom atinja o nível mínimo suportado, o sistema não responde mais a ação.
Cenário de Exceção	
Ações do Ator	Ações do sistema

Tabela 3 - Documentação Caso de Uso - Habilitar Alto Contraste

Nome do Caso de Uso	UC03 – Habilitar Alto Contraste
Caso de Uso Geral	
Ator Principal	Usuário
Ator Secundário	Sistema
Resumo	Esse caso de uso descreve as etapas percorridas por um usuário, com auxílio do sistema, para habilitar o alto contraste da página.
Ações do Ator	Ações do sistema
1. O usuário clica no botão de alto contraste.	
	2. O sistema aplica o tema de alto contraste.
Restrições/Validações	
Cenário Alternativo – Ampliar Conteúdo	
Ações do Ator	Ações do sistema
	2.1. Se o sistema não conseguir aplicar o tema, uma mensagem de erro é exibida ao usuário.
Cenário de Exceção	
Ações do Ator	Ações do sistema

Tabela 4 - Documentação Caso de Uso - Desabilitar Alto Contraste

Nome do Caso de Uso	UC04 – Desabilitar Alto Contraste
Caso de Uso Geral	
Ator Principal	Usuário
Ator Secundário	Sistema

Resumo	Esse caso de uso descreve as etapas percorridas por um usuário, com auxílio do sistema, para desabilitar o alto contraste da página.
Ações do Ator	Ações do sistema
1. O usuário clica no botão de alto contraste para desativar o modo.	
	2. O sistema retorna ao tema padrão.
Restrições/Validações	
Cenário Alternativo – Ampliar Conteúdo	
Ações do Ator	Ações do sistema
	2.1. Caso ocorra algum erro, o sistema exibe uma mensagem de falha ao usuário.
Cenário de Exceção	
Ações do Ator	Ações do sistema

Tabela 5 - Documentação Caso de Uso - Acessar Login

Nome do Caso de Uso	UC05 – Acessar Login
Caso de Uso Geral	
Ator Principal	Usuário
Ator Secundário	Sistema
Resumo	Esse caso de uso descreve as etapas percorridas por um usuário, com auxílio do sistema, para acessar a página de login.
Ações do Ator	Ações do sistema
1. O usuário clica no botão de fazer login.	
	2. O sistema direciona o usuário à tela de login.
Restrições/Validações	
Cenário Alternativo – Ampliar Conteúdo	
Ações do Ator	Ações do sistema
	2.1. Caso o botão de login apresente algum erro, uma mensagem de erro será exibida.
Cenário de Exceção	
Ações do Ator	Ações do sistema

Tabela 6 – Lista de Casos de Uso – Tela de Abertura do Sistema

Nº	Descrição	Caso de Uso
01	Usuário pode ampliar o conteúdo	Ampliar Conteúdo
02	Usuário pode reduzir o conteúdo	Reduzir Conteúdo
03	Usuário pode habilitar o alto contraste	Habilitar Alto Contraste
04	Usuário pode desabilitar o alto contraste	Desabilitar Alto Contraste
05	Usuário acessa a página de login	Acessar Login

Task #20:

Os componentes Footer, Header, Layout e AccessibilityBar foram padronizados, assegurando uma experiência visual e funcional harmoniosa em toda a interface. A definição de rotas foi organizada para garantir uma navegação clara e eficiente entre todas as funcionalidades do sistema.

- **Implementações de acessibilidade:**

Neste sistema, foi implementada uma funcionalidade de alternância entre o tema padrão e o modo de alto contraste, visando melhorar a acessibilidade visual. O tema padrão utiliza uma paleta de cores definida por variáveis CSS, que são armazenadas no seletor `:root` para proporcionar uma experiência harmoniosa. Quando o tema é alternado para o modo de alto contraste, o sistema utiliza o seletor `[data-theme="high-contrast"]` para redefinir essas variáveis com valores de cor ajustados, como tons mais escuros, facilitando a leitura para usuários com baixa visão.

Um contexto global controla o tema e o tamanho da fonte, armazenando esses valores no navegador para que persistam entre sessões. O usuário pode alternar entre os temas, e a escolha é aplicada diretamente ao layout da aplicação, garantindo uma experiência consistente e ajustada conforme as preferências de acessibilidade. Além disso, a interface possibilita a alteração do tamanho da fonte, com o limite de 44 pixels, assegurando conforto visual em diferentes dispositivos. Essa abordagem de variáveis CSS facilita a manutenção e a expansão dos temas, permitindo ajustes rápidos e personalizados para cada elemento de interface.



Figura 3 - Componente Acessibilidade (Alto Contraste)



Figura 4 - Componente Acessibilidade (Formatação de Texto)



Figura 5 - Componente Acessibilidade (Formatação de Texto)

- **Componentes Implementados:**

Foi criada uma pasta chamada “componentes”, na qual foram inseridos diversos componentes fundamentais para a estrutura da aplicação, incluindo o “Footer”, “Header”, “AccessibilityBar” e “Layout”.

- Footer: Este componente é responsável por exibir informações relevantes no final de cada página, como políticas de privacidade.



Figura 6 - Componente Footer

- **Header:** O Header contém o logotipo da aplicação, suas demais funções serão inseridas conforme o necessário.



Figura 7 - Componente Header

- **AccessibilityBar:** Este componente foi projetado para oferecer ferramentas que melhoram a acessibilidade do site, permitindo que os usuários ajustem o contraste, o tamanho da fonte e outras configurações de visualização. A barra de acessibilidade está localizada acima do header.

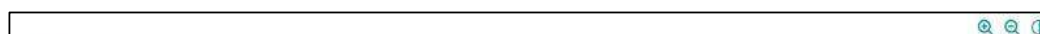


Figura 8 - Componente AccessibilityBar

- **Layout:** O componente de Layout serve como uma estrutura base para as páginas da aplicação. Ele organiza a disposição dos outros componentes, garantindo que o conteúdo seja apresentado de forma consistente. O Layout ajuda a manter a coerência visual e funcional da aplicação.



Figura 9 - Componente Layout

- **Criação Rotas:**

A URL das rotas foi centralizada em um único arquivo, facilitando a manutenção e a gestão das rotas da aplicação. Essa abordagem permite uma organização mais eficiente, permitindo que as alterações nas URLs sejam feitas em um único local, reduzindo a probabilidade de erros e melhorando a legibilidade do código.

- **Bibliotecas Utilizadas:**

Até o momento foram utilizadas duas bibliotecas principais para aprimorar a interface e navegação da aplicação:

- @phosphor-icons/react: Proporciona um conjunto versátil de ícones, personalizáveis em tamanho e cor, ajudando a manter um visual coeso e atraente na interface.
- react-router-dom: Facilita a criação e o gerenciamento das rotas, oferecendo navegação dinâmica e fluida entre as páginas. A centralização das rotas em um único arquivo simplifica a manutenção e melhora a organização da estrutura da aplicação.

Assinaturas

Documento assinado digitalmente
gov.br LEMUEL PRATES SOBRINHO FERREIRA
Data: 28/05/2025 14:33:59-0300
Verifique em <https://validar.iti.gov.br>

Lemuel Prates Sobrinho Ferreira

Documento assinado digitalmente
gov.br MARIA CLARA BORGES CARDOSO
Data: 26/05/2025 17:38:34-0300
Verifique em <https://validar.iti.gov.br>

Maria Clara Borges Cardoso

Orientador: Jefferson Antonio Ribeiro Passerini

7.1.5 Sprint 05 - Task 25



Faculdade de Tecnologia Professor José Camargo

Sprint Documentation		
Sprint #5	Start Date:11/11/2014	Final Date:23/11/2014
Projetc Name:	SeniorCareManager-Frontend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Eduardo Matheus Nardi		
Edson Pereira De Carvalho		
Gabriel Jorge Marta		
Guilherme Takemoto Pereira		
Igor Olhier Aveledo		
Lemuel Prates Sobrinho Ferreira		
Maria Clara Borges Cardoso		
Rodrigo Yoshida Lombezi		
Vinicius Lúcio Marcolino Da Silva		

Sprint Backlog			
Task#	Description	Start Date	Final date
25	Sprint 05 – Desenvolver a tela de cadastro para religião	12/11/2024	23/11/2024

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
25	Sprint 05 - Desenvolver a tela de cadastro para religião	Lemuel, Rodrigo, Vinicius, Edson, Maria Clara, Gabriel Jorge	Concluído	32	32

Sprint Description

Durante a sprint 5, o foco principal foi o desenvolvimento da tela de cadastro de religião e o seu protótipo, conforme descrito na task#25, além da tela de cadastros gerais. A tela de cadastros gerais tem a função de exibir e acessar as informações cadastradas no sistema, enquanto a tela de cadastro de religião permite ao usuário realizar as operações de edição, cadastro, exclusão e visualização das religiões registradas.

Sprint Results

Implementações realizadas:

Na Sprint 04, foi utilizado o Axios para realizar a conexão entre o frontend e o backend. O Axios e responsável **Axios** é uma biblioteca JavaScript para realizar requisições HTTP de forma simples e eficiente, tanto em ambientes de navegador quanto no Node.js. Ela é baseada em promessas e fornece uma API fácil de usar para interagir com servidores e APIs. Axios suporta métodos HTTP como `GET`, `POST`, `PUT`, `DELETE`, além de permitir configuração global de cabeçalhos, tempo de espera, e interceptação de requisições e respostas. Também oferece recursos como cancelamento de requisições e tratamento de erros, tornando a comunicação com servidores mais robusta e flexível.

Comando Usado	Função
<code>npm install axios</code> ou <code>yarn add axios</code>	Responsável para instalar o axios no projeto

A classe `ReligionService` foi criada para gerenciar dados relacionados a religiões. Ela herda funcionalidades de uma classe genérica chamada `GenericService`. A ideia aqui é evitar a repetição de código ao lidar com diferentes tipos de dados, utilizando uma abordagem genérica.

A classe `GenericService` fornece uma implementação genérica para operações CRUD (Create, Read, Update, Delete) em uma API RESTful. Ela utiliza o Axios para realizar as requisições HTTP e permite interagir de forma simplificada com servidores, independentemente do tipo de modelo de dados.

Funcionalidade

A classe `GenericService` permite a criação de um serviço que pode ser reutilizado para qualquer tipo de modelo de dados, como "usuários", "produtos", "pedidos", entre outros, através do uso de um tipo genérico (`T`). A URL base para as requisições é configurada dinamicamente com base no nome do modelo, o que torna a classe flexível e reutilizável.

Comando Usado	Função
<code>getAll(): Promise<T[]></code>	Responsável por realizar uma requisição <code>GET</code> para obter todos os registros do modelo. Retorna um array com os dados dos registros.
<code>getById(id: number): Promise<T></code>	Realiza uma requisição <code>GET</code> para obter um item específico pelo <code>id</code> . Retorna o dado do item solicitado.
<code>create(model: T): Promise<T></code>	Realiza uma requisição <code>POST</code> para criar um novo item no servidor. O <code>model</code> (do tipo <code>T</code>) é enviado no corpo da requisição. Retorna o objeto criado, conforme retornado pela API.
<code>update(id: number, model: T): Promise<T></code>	Realiza uma requisição <code>PUT</code> para atualizar um item existente no servidor. O <code>id</code> do item é utilizado para identificá-lo, e o <code>model</code> atualizado é enviado no corpo da requisição. Retorna o objeto atualizado conforme retornado pela API.
<code>delete(id: number): Promise<string></code>	Realiza uma requisição <code>DELETE</code> para excluir um item, identificado pelo <code>id</code> . Retorna uma mensagem de sucesso ou erro conforme a resposta da API.

A tela de cadastros exibira todas as categorias atualmente registradas no sistema, organizadas e exibidas por cards. Nela, o usuário poderá visualizar e interagir com as categorias. Essa tela oferece uma

visão geral dos cadastrados de categorias, permitindo ao usuário buscar pelo cadastro pelo nome, e permite filtrar pelo tipo de cadastro. A interface foi projetada para ser intuitiva, garantindo que as operações sejam realizadas de forma ágil e sem complicações, facilitando a administração dos dados do sistema.

A Tela de Cadastro de Religião permite ao usuário visualizar, cadastrar, editar e excluir religiões no sistema. Nela, é possível listar todas as religiões cadastradas, com a opção de selecionar uma ou várias para realizar as ações disponíveis.

Funcionalidades Principais:

1. Listagem de Religiões: Exibe uma lista com todas as religiões cadastradas, facilitando a visualização e seleção.
2. Cadastro: Permite adicionar novas religiões ao sistema através de um formulário simples.
3. Edição: Possibilita a atualização de informações de religiões já cadastradas.
4. Exclusão: Permite remover uma ou várias religiões, com confirmação prévia para evitar exclusões acidentais.

O objetivo é oferecer uma gestão fácil e rápida das informações de religiões no sistema.

Ao iniciar o processo de cadastro de uma nova religião, uma caixa de diálogo será exibida no centro da tela. Essa caixa contém um formulário simples para facilitar a inserção de novas religiões no sistema.

Elementos da Caixa de Cadastro:

- Campo de Texto para Nova Religião: Deve ser preenchido com o nome da nova religião a ser cadastrada, garantindo que o nome seja único e adequado.
- Botão "Cadastrar": Após preencher o nome da religião, o usuário deve clicar no botão "Cadastrar" para adicionar a nova religião ao sistema. O sistema realizará uma validação para assegurar que o nome não seja duplicado.
- Botão "Cancelar": Caso o usuário queira desistir do cadastro, o botão "Cancelar" fechará a caixa de diálogo sem realizar nenhuma alteração no sistema.

Confirmação de Cadastro:

Após o cadastro ser concluído com sucesso, o sistema exibirá uma notificação no canto superior da tela, confirmando que a nova religião foi registrada com sucesso.

Elementos da Caixa de Editar:

Ao acessar a seção de Religião, será exibida uma caixa de diálogo centralizada na tela, permitindo que o usuário edite as informações. Dentro dessa caixa, estarão disponíveis dois botões:

1. Cancelar – Esse botão permitirá que o usuário cancele o processo de edição, retornando à página anterior sem realizar alterações.
2. Editar – Ao clicar neste botão, o sistema irá atualizar as informações da religião com os dados fornecidos pelo usuário.

Após a edição, uma notificação será exibida na tela confirmando que a atualização foi realizada com sucesso.

Confirmação de Edição

Após a conclusão da edição, o sistema exibirá uma notificação no canto superior da tela, confirmando que a alteração foi realizada com sucesso.

Elementos da Caixa de Exclusão:

- Botão "Sim, desejo excluir!": O sistema realizará uma exclusão de uma religião do sistema.
- Botão "Cancelar": Caso o usuário queira desistir do cadastro, o botão "Cancelar" fechará a caixa de diálogo sem realizar nenhuma alteração no sistema.

Confirmação de Exclusão:

Após o cadastro ser concluído com sucesso, o sistema exibirá uma notificação no canto superior da tela, confirmando que a religião foi excluída com sucesso.

Figura 1 – Tela de Cadastros

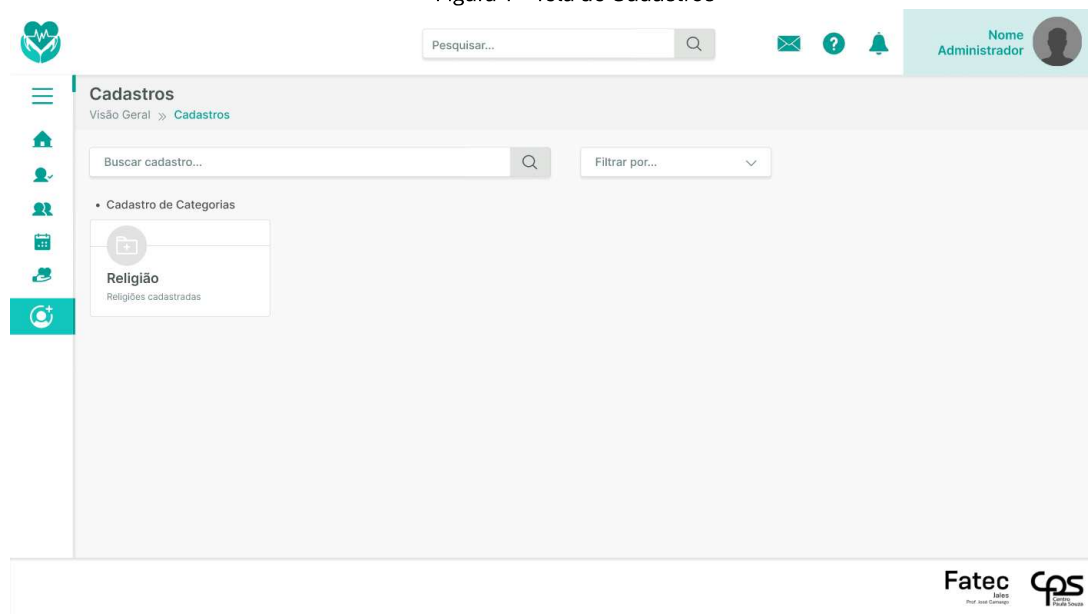


Figura 1 - Tela de Cadastro de Religião

The screenshot displays the 'Religião' (Religion) management interface. At the top, there's a search bar labeled 'Pesquisar...' and a user profile section showing 'Nome Administrador'. The main area is titled 'Religião' with a breadcrumb 'Visão Geral > Cadastros > Religião'. Below this, there's a search bar 'Buscar religião...' and a green button '+ Adicionar religião'. A table lists existing religions with checkboxes and edit/delete icons:

Nome da religião		
☐ Doutrina Espírita		
☐ Igreja Católica		
☐ Budismo		
☐ Judaísmo		

Fatec
Jales
Prof. José Gervásio

CPS
Centro de
Processamento de
Dados

Figura 2 - Tela de Cadastro de Religião(Cadastrar)

This screenshot shows the same 'Religião' page as Figure 1, but with a modal form open for adding a new religion. The modal is titled 'Cadastro Religião' and contains the following elements:

- Label: 'Nome religião:'
- Text input field for the religion name.
- Buttons: 'X Cancelar' (red) and '+ Cadastrar' (green).

The background page remains visible but is dimmed.

Fatec
Jales
Prof. José Gervásio

CPS
Centro de
Processamento de
Dados

Figura 4 - Tela de Cadastro de Religião(Confirmação de exclusão)

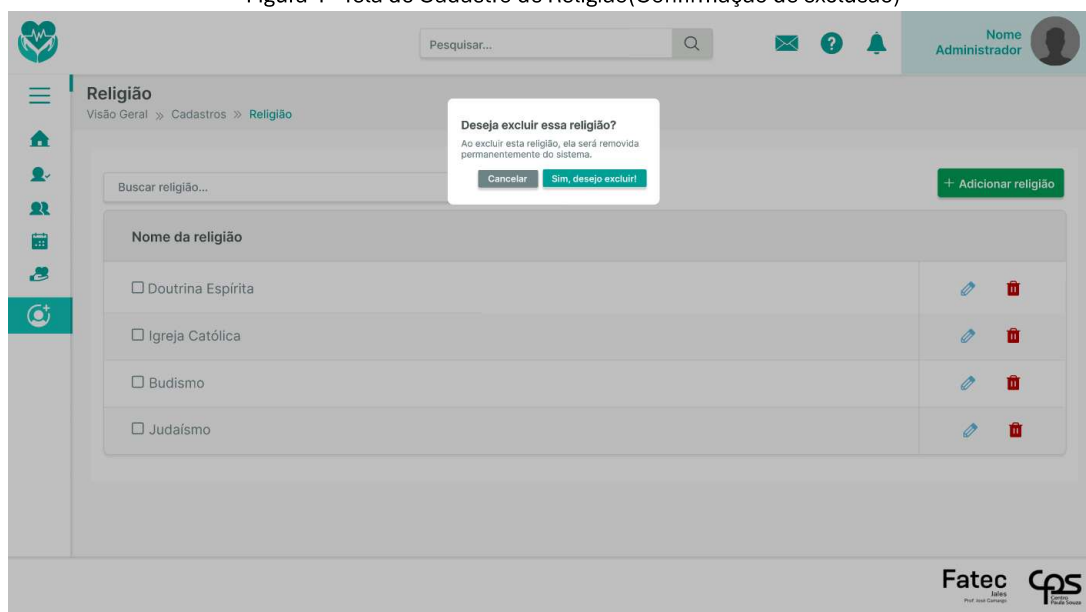


Figura 5 – Mensagem de exclusão com sucesso



- **Componentes Implementados:**

Foram inseridos alguns componentes, como "Tabela", "Menu" e "Botão".

- Tabela: Este componente exibe uma página de cadastro, com ícone, nome e descrição, também fazer a navegação para uma tela de cadastro.

Figura 6 - Componente Acessibilidade (Formatação de Texto)



- Menu: Este componente é responsável pela principal navegação do site.

Figura 7 – Componente Menu(Maximizado)

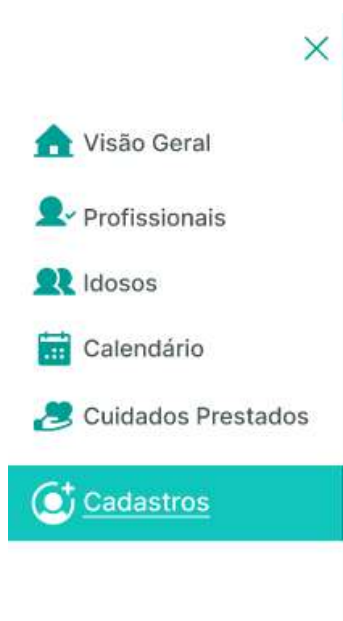
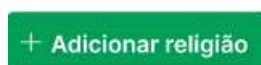


Figura 8 – Componente Menu(Minimizado)



- Botão: O componente botão complementa perfeitamente a estrutura da sua aplicação e pode ser implementado como um elemento reutilizável e estilizado para diversas funcionalidades, mantendo consistência e modularidade.

Figura 9 – Componente Menu(Maximizado)



Assinaturas

Documento assinado digitalmente
 **EDSON PEREIRA DE CARVALHO**
Data: 27/05/2025 15:20:29-0300
Verifique em <https://validar.iti.gov.br>

Edson Pereira De Carvalho

Documento assinado digitalmente
 **GABRIEL JORGE MARTA**
Data: 27/05/2025 14:57:35-0300
Verifique em <https://validar.iti.gov.br>

Gabriel Jorge Marta

Documento assinado digitalmente
 **LEMUEL PRATES SOBRINHO FERREIRA**
Data: 28/05/2025 14:33:59-0300
Verifique em <https://validar.iti.gov.br>

Lemuel Prates Sobrinho Ferreira

Documento assinado digitalmente
 **MARIA CLARA BORGES CARDOSO**
Data: 26/05/2025 17:38:34-0300
Verifique em <https://validar.iti.gov.br>

Maria Clara Borges Cardoso

Documento assinado digitalmente
 **RODRIGO YOSHIDA LOMBEZZI**
Data: 29/05/2025 14:15:22-0300
Verifique em <https://validar.iti.gov.br>

Rodrigo Yoshida Lombezzi

Documento assinado digitalmente
 **VINICIUS LUCIO MARCOLINO DA SILVA**
Data: 29/05/2025 16:29:35-0300
Verifique em <https://validar.iti.gov.br>

Vinicius Lúcio Marcolino Da Silva

Orientador: Jefferson Antonio Ribeiro Passerini

8 Conclusão

O desenvolvimento do sistema *SeniorCareManager* demonstrou a relevância da tecnologia como ferramenta de fortalecimento institucional em organizações sociais. A partir da identificação das fragilidades operacionais do Lar dos Velhinhos São Vicente de Paulo e da APAE de Jales, evidenciou-se a necessidade de uma solução que integrasse informações assistenciais, administrativas e logísticas de forma segura e rastreável. O projeto resultou em uma plataforma robusta, composta por módulos independentes e intercomunicáveis, capazes de automatizar processos essenciais, como prontuário eletrônico, controle de estoque, compras, doações e gestão de vacinas.

A utilização de práticas consolidadas de engenharia de software, aliada à modelagem UML, possibilitou a definição clara de requisitos, fluxos e entidades, garantindo coerência estrutural e aderência às necessidades institucionais. Aspectos como acessibilidade, segurança da informação e conformidade com a LGPD reforçaram o compromisso do projeto com a proteção de dados e a inclusão tecnológica.

Assim, o *SeniorCareManager* representa um avanço significativo na modernização das rotinas internas das instituições atendidas, contribuindo para maior eficiência operacional, qualidade assistencial e otimização de recursos. O projeto reafirma o potencial transformador da extensão acadêmica, ao integrar conhecimento técnico e impacto social.